

```

/**
 * -----
 * Global Constants and Data Mapping
 * -----
 */

const MIN_TAKT_TIME = 2.5;

const BUILD_RATIOS = { super: 0.35, ultra: 0.45, mega: 0.20 };

const ASSEMBLY_LINE_LENGTH = 486;

let isRecalculating = false;

let autoAdjustEnabled = true;


const PRECEDENCE_DATA = [

  { id: 1, predecessors: [] }, { id: 2, predecessors: [1] }, { id: 3, predecessors: [1] }, { id: 4,
predecessors: [1] },

  { id: 5, predecessors: [2, 3] }, { id: 6, predecessors: [1] }, { id: 7, predecessors: [6] }, { id: 8,
predecessors: [1] },

  { id: 9, predecessors: [8] }, { id: 10, predecessors: [1] }, { id: 11, predecessors: [1] }, { id: 12,
predecessors: [10, 11] },

  { id: 13, predecessors: [4, 5, 7, 9, 12] }, { id: 14, predecessors: [13] }, { id: 15,
predecessors: [14] }, { id: 16, predecessors: [15] },

  { id: 17, predecessors: [16] }, { id: 18, predecessors: [14] }, { id: 19, predecessors: [18] },
{ id: 20, predecessors: [19] },

  { id: 21, predecessors: [20] }, { id: 22, predecessors: [18] }, { id: 23, predecessors: [22] },
{ id: 24, predecessors: [23] },

  { id: 25, predecessors: [19, 22] }, { id: 26, predecessors: [19, 22] }, { id: 27, predecessors:
[25, 26] }, { id: 28, predecessors: [27] },

  { id: 29, predecessors: [15] }, { id: 30, predecessors: [17, 21, 24, 27, 29] }, { id: 31,
predecessors: [30] },

```

```
];
```

```
const PERT_LABOR_FALLBACK = {
```

```
  1: 0.8, 2: 1.5, 3: 1.0, 4: 1.3, 5: 0.5, 6: 0.7, 7: 0.7, 8: 0.52, 9: 0.325, 10: 0.96, 11: 0.3, 12: 2.0,  
  13: 2.5, 14: 1.5, 15: 2.2, 16: 1.2, 17: 0.5, 18: 0.8, 19: 1.1, 20: 0.325, 21: 0.325, 22: 1.3, 23:  
  0.14,
```

```
  24: 0.585, 25: 1.3, 26: 1.0, 27: 0.7, 28: 1.0, 29: 0.7, 30: 0.5, 31: 0.4  
};
```

```
const WORKSTATION_CAPACITIES = [
```

```
  { ws: 3, maxDemand: 147 }, { ws: 4, maxDemand: 192 }, { ws: 5, maxDemand: 229 },  
  { ws: 6, maxDemand: 284 }, { ws: 7, maxDemand: 312 }, { ws: 8, maxDemand: 350 },  
  { ws: 9, maxDemand: 407 }, { ws: 10, maxDemand: 419 }, { ws: 11, maxDemand: 499 },  
  { ws: 12, maxDemand: 501 }, { ws: 13, maxDemand: 552 }
```

```
];
```

```
const root = document.documentElement;
```

```
const PERT_PIE_STROKE = getComputedStyle(root).getPropertyValue('--white').trim();
```

```
const PERT_PIE_COLORS = {
```

```
  super: getComputedStyle(root).getPropertyValue('--super-color').trim(),  
  ultra: getComputedStyle(root).getPropertyValue('--ultra-color').trim(),  
  mega: getComputedStyle(root).getPropertyValue('--mega-color').trim(),  
  idle: getComputedStyle(root).getPropertyValue('--idle-color').trim()
```

```
};
```

```
const originalConfigData = {};
```

```
const state = {  
  taskData: new Map(),  
  configData: {}  
};
```

```
const { draw: drawPrecedenceChart, update: updatePrecedenceChart, flatten:  
flattenPrecedenceTree } = PrecedenceTab;
```

```
let sortableInstances = [];  
  
let precedenceChartNodes = null;  
  
let invalidPrecedenceNodes = new Set();  
  
let profitMaximizationCache = { key: null, data: null };  
  
let isProfitCalculating = false;  
  
let animationState = {  
  speedMultiplier: 1.0,  
  layout: { frameId: null, isRunning: false, isPaused: false },  
  schedule: { frameId: null, isRunning: false, isPaused: false },  
  speedo: { currentAngle: 0 }  
};
```

```
/**  
 * -----  
 * DOM ELEMENTS  
 * -----  
 */
```

```
const dailyDemandInput = document.getElementById('dailyDemand');
```

```
const opHoursInput = document.getElementById('opHours');
const numEmployeesInput = document.getElementById('numEmployees');
const employeeCountDisplay = document.getElementById('employeeCountDisplay');
const laborCostInput = document.getElementById('laborCost');
const superSellInput = document.getElementById('superSell');
const superCogsInput = document.getElementById('superCogs');
const ultraSellInput = document.getElementById('ultraSell');
const ultraCogsInput = document.getElementById('ultraCogs');
const megaSellInput = document.getElementById('megaSell');
const megaCogsInput = document.getElementById('megaCogs');
const wipEl = document.getElementById('wip');
const throughputEl = document.getElementById('throughput');
const conveyorSpeedEl = document.getElementById('conveyorSpeed');
const productSpacingEl = document.getElementById('productSpacing');
const grossProfitEl = document.getElementById('grossProfit');
const profitMarginEl = document.getElementById('profitMargin');
const demandStatusEl = document.getElementById('demandStatus');
const avgEfficiencyEl = document.getElementById('avgEfficiency');
const totalIdleTimeEl = document.getElementById('totalIdleTime');
const balanceDelayEl = document.getElementById('balanceDelay');
const idleTimeCvEl = document.getElementById('idleTimeCv');
const leftSidebar = document.getElementById('left-sidebar');
const rightSidebar = document.getElementById('right-sidebar');
const leftToggle = document.getElementById('left-toggle');
const rightToggle = document.getElementById('right-toggle');
const tabs = document.getElementById('tabs');
```

```

const visPanels = document.querySelectorAll('.vis-panel');

const workstationList = document.getElementById('workstation-list');

const precedenceMap = flattenPrecedenceTree();


/**
 * -----
 * Main Initialization
 *
 * These functions are the entry point for the application, handling
 * initial data loading and setup calls.
 * -----
 */


/**
 * Injects custom CSS styles for the sidebars' and main panel's scrollbars.
 */

function injectCustomStyles() {
    const accentColor = getComputedStyle(root).getPropertyValue('--accent').trim();
    const primaryColor = getComputedStyle(root).getPropertyValue('--primary').trim();
    const style = document.createElement('style');
    style.textContent = `
/* Target sidebars and SVG area for Firefox */
#left-sidebar, #right-sidebar, #svg-container {
scrollbar-width: thin;

scrollbar-color: ${accentColor} transparent;
}

```

```

/* Target sidebars and SVG area for Webkit browsers (Chrome, Safari, etc.) */

#left-sidebar::-webkit-scrollbar, #right-sidebar::-webkit-scrollbar, #svg-container::-webkit-scrollbar {

width: 10px;

height: 10px;

}

#left-sidebar::-webkit-scrollbar-track, #right-sidebar::-webkit-scrollbar-track, #svg-container::-webkit-scrollbar-track {

background: transparent;

}

#left-sidebar::-webkit-scrollbar-thumb, #right-sidebar::-webkit-scrollbar-thumb, #svg-container::-webkit-scrollbar-thumb {

background-color: ${accentColor};

border-radius: 10px;

border: 2px solid transparent;

background-clip: content-box;

}

#left-sidebar::-webkit-scrollbar-thumb:hover, #right-sidebar::-webkit-scrollbar-thumb:hover, #svg-container::-webkit-scrollbar-thumb:hover {

background-color: ${primaryColor};

}

`;

document.head.appendChild(style);

}

/**

```

* The main function to initialize the application.

```

*/

async function main() {
    injectCustomStyles();

    await loadData();

    setupEventListeners();

    setupUIEventListeners();

    setupVisibilityListener();

    runProfitCalculation();

    state.invalidPrecedenceMap = validatePrecedence();

    invalidPrecedenceNodes = new Set(Array.from(state.invalidPrecedenceMap.keys()));

    restoreActiveTab();

    updateUI();

    renderActiveTab();

    document.querySelectorAll("input[type='number']").forEach(input => {
        enableMiddleDragNumberInput(input, 1, 1);
    });

    document.querySelectorAll("input[type='range']").forEach(input => {
        enableMiddleDragNumberInput(input, 1, 1);
    });
}

```

```

/**

```

```

* Loads PERT and CONFIGS data from CSV files and populates the global state.

```

```

*/

```

```

async function loadData() {
    try {

```

```

const [pertData, configsRaw] = await Promise.all([
  d3.csv("Data/PERT.csv"),
  d3.csv("Data/CONFIGS.csv")
]);
pertData.forEach(d => {
  state.taskData.set(parseInt(d.Element), {
    laborTime: parseFloat(d.Labor_Time),
    elementTime: parseFloat(d.Element_Time),
    description: d["Description"] || d.Description,
    Super: parseFloat(d.Super),
    Ultra: parseFloat(d.Ultra),
    Mega: parseFloat(d.Mega)
  });
});
for (let i = 3; i <= 13; i++) {
  originalConfigData[i] = {};
}
configsRaw.forEach(row => {
  for (let i = 3; i <= 13; i++) {
    const workstation = row[`${i}_Workstation`];
    const element = parseInt(row[`${i}_Element`]);
    if (workstation && !isNaN(element)) {
      if (!originalConfigData[i][workstation]) {
        originalConfigData[i][workstation] = [];
      }
      originalConfigData[i][workstation].push(element);
    }
  }
});

```



```

        }
    }
});

state.configData = JSON.parse(JSON.stringify(originalConfigData));

console.log("Local CSV data loaded successfully.");

} catch (error) {

    console.error("Fatal Error: Could not load local data files.", error);

    demandStatusEl.innerHTML = "Error: Failed to load data.<br>Please use a local
server.";

}

}

```

```

/**
 * -----
 * UI & DOM Manipulation
 *
 * These functions directly interact with the DOM to update the user
 * interface, render components, and set up event listeners.
 * -----
 */

```

```

/**
 * Halts all running `requestAnimationFrame` loops for the layout and
 * schedule simulations.
 */

function stopAllSimulations() {

```

```

    if (animationState.layout.frameId) {
        cancelAnimationFrame(animationState.layout.frameId);
        animationState.layout.frameId = null;
        animationState.layout.isRunning = false;
    }
    if (animationState.schedule.frameId) {
        cancelAnimationFrame(animationState.schedule.frameId);
        animationState.schedule.frameId = null;
        animationState.schedule.isRunning = false;
    }
}

/**
 * Animates a numeric value in a DOM element from a start to an end value.
 * @param {HTMLElement} element - The DOM element to update.
 * @param {number} start - The starting number.
 * @param {number} end - The ending number.
 * @param {number} [duration=1000] - The animation duration in milliseconds.
 * @param {Function} [formatter] - A function to format the number for display.
 */
function animateValue(element, start, end, duration = 1000, formatter = (val) =>
val.toFixed(1)) {
    if (!element) return;
    if (element._animationId) {
        cancelAnimationFrame(element._animationId);
    }
}

```

```

const startTime = Date.now();

const range = end - start;

function updateValue() {
    const now = Date.now();

    const elapsed = now - startTime;

    const progress = Math.min(elapsed / duration, 1);

    const easeProgress = 1 - Math.pow(1 - progress, 4);

    const current = start + (range * easeProgress);

    element.textContent = formatter(current);

    if (progress < 1) {
        element._animationId = requestAnimationFrame(updateValue);
    } else {
        element._animationId = null;
    }
}

updateValue();
}

/**
 * Parses a numeric value from an element's text content, ignoring
 * currency symbols and other non-numeric characters.
 * @param {HTMLElement} element - The DOM element to parse.
 * @returns {number} The parsed numeric value.
 */
function parseElementValue(element) {

```

```

if (!element || !element.textContent) return 0;

const text = element.textContent;

if (text.includes('$')) {

    const cleanedText = text.replace(/[$,]/g, "");

    const match = cleanedText.match(/-?\d+\.\d*/);

    return match ? parseFloat(match[0]) : 0;

}

const match = text.match(/-?\d+\.\d*/);

return match ? parseFloat(match[0]) : 0;

}

/**
 * Enhances a number or range input to allow value changes via
 * middle-mouse-button drag, mouse wheel scroll, and Ctrl+Click to reset.
 * @param {HTMLInputElement} input - The input element to enhance.
 * @param {number} [step=1] - The step value for dragging.
 * @param {number} [sensitivity=0.1] - The drag sensitivity.
 */
function enableMiddleDragNumberInput(input, step = 1, sensitivity = 0.1) {

    let isDragging = false;

    let startY, startValue;

    const defaultValues = {

        'dailyDemand': 180,

        'opHours': 15.0,

        'numEmployees': 8,

        'laborCost': 25.0,

```

```

'superSell': 1250,
'superCogs': 450,
'ultraSell': 1500,
'ultraCogs': 550,
'megaSell': 1800,
'megaCogs': 650
};

const getConstraints = () => {
  const min = input.hasAttribute('min') ? parseFloat(input.min) : -Infinity;
  const max = input.hasAttribute('max') ? parseFloat(input.max) : Infinity;
  const stepValue = parseFloat(input.step) || 1;
  return { min, max, step: stepValue };
};

input.addEventListener("mousedown", (e) => {
  if (e.button === 1) {
    e.preventDefault();
    e.stopPropagation();
    isDragging = true;
    startY = e.clientY;
    startValue = parseFloat(input.value) || 0;
    const onMouseMove = (ev) => {
      if (!isDragging) return;
      const deltaY = startY - ev.clientY;
      const constraints = getConstraints();
      let newVal = startValue + deltaY * sensitivity * step;
      newVal = Math.max(constraints.min, Math.min(constraints.max, newVal));
    };
  }
});

```

```

    if (input.type === 'range' || constraints.step === 1) {
        input.value = Math.round(newVal).toString();
    } else if (constraints.step < 1) {
        const decimals = Math.max(0, -Math.floor(Math.log10(constraints.step)));
        input.value = newVal.toFixed(decimals);
    } else {
        input.value = newVal.toFixed(2);
    }

    input.dispatchEvent(new Event("input", { bubbles: true }));
};

const onMouseUp = () => {
    isDragging = false;

    document.removeEventListener("mousemove", onMouseMove);
    document.removeEventListener("mouseup", onMouseUp);
};

document.addEventListener("mousemove", onMouseMove);
document.addEventListener("mouseup", onMouseUp);
}

});

input.addEventListener("wheel", (e) => {
    if (document.activeElement === input) {
        e.preventDefault();

        const constraints = getConstraints();
        const direction = e.deltaY > 0 ? -1 : 1;

        let currentValue = parseFloat(input.value) || 0;
        let newVal = currentValue + (direction * constraints.step);
    }
});

```

```

    newVal = Math.max(constraints.min, Math.min(constraints.max, newVal));
    if (input.type === 'range' || constraints.step === 1) {
        input.value = Math.round(newVal).toString();
    } else if (constraints.step < 1) {
        const decimals = Math.max(0, -Math.floor(Math.log10(constraints.step)));
        input.value = newVal.toFixed(decimals);
    } else {
        input.value = newVal.toFixed(2);
    }
    input.dispatchEvent(new Event("input", { bubbles: true }));
}
});
}

/**
 * Restores the active tab from session storage on page load.
 */
function restoreActiveTab() {
    let targetTab = sessionStorage.getItem("activeTab");
    if (!targetTab) {
        targetTab = "overview";
        sessionStorage.setItem("activeTab", targetTab);
    }
    document.querySelectorAll(".tab-btn").forEach(b => b.classList.remove("active"));
    const btn = document.querySelector(`.tab-btn[data-tab="${targetTab}"]`);
    if (btn) btn.classList.add("active");
}

```

```

visPanels.forEach(panel => {
    panel.style.display = panel.id === `${targetTab}-panel` ? "block" : "none";
});
renderActiveTab();
}

```

```

/**

```

```

 * Main UI update function. It recalculates metrics and updates all

```

```

 * output displays and visualizations.

```

```

 */

```

```

function updateUI() {
    employeeCountDisplay.textContent = numEmployeesInput.value;
    renderWorkstationSidebar(parseInt(numEmployeesInput.value));
    setupDragAndDrop();

```

```

    if (invalidPrecedenceNodes.size > 0) {
        demandStatusEl.textContent = "Fails to Meet Precedence";
        demandStatusEl.className = "status failure";
        wipEl.textContent = '---';
        throughputEl.textContent = '---';
        conveyorSpeedEl.textContent = '---';
        productSpacingEl.textContent = '---';
        grossProfitEl.textContent = '---';
        profitMarginEl.textContent = '---';
        avgEfficiencyEl.textContent = '---';
        totalIdleTimeEl.textContent = '---';
    }

```



```
balanceDelayEl.textContent = '---';
idleTimeCvEl.textContent = '---';
} else {
  const opInputs = {
    dailyDemand: parseInt(dailyDemandInput.value),
    opHours: parseFloat(opHoursInput.value),
    numEmployees: parseInt(numEmployeesInput.value)
  };
  const finInputs = {
    laborCost: parseFloat(laborCostInput.value),
    superSell: parseFloat(superSellInput.value),
    superCogs: parseFloat(superCogsInput.value),
    ultraSell: parseFloat(ultraSellInput.value),
    ultraCogs: parseFloat(ultraCogsInput.value),
    megaSell: parseFloat(megaSellInput.value),
    megaCogs: parseFloat(megaCogsInput.value),
  };
  const results = calculateMetrics(opInputs, finInputs);
  if (results) {
    animateValue(wipEl, parseElementValue(wipEl), results.wip, 800, val =>
val.toFixed(1));

    animateValue(throughputEl, parseElementValue(throughputEl),
results.throughputUnitsPerHour, 800, val => `${val.toFixed(1)}/hr`);

    animateValue(conveyorSpeedEl, parseElementValue(conveyorSpeedEl),
results.conveyorSpeed, 800, val => `${val.toFixed(2)} ft/min`);

    animateValue(productSpacingEl, parseElementValue(productSpacingEl),
results.productSpacing, 800, val => `${val.toFixed(2)} ft`);
```

```
    animateValue(grossProfitEl, parseElementValue(grossProfitEl),
results.dailyGrossProfit, 800, val => val.toLocaleString('en-US', { style: 'currency', currency:
'USD', minimumFractionDigits: 0, maximumFractionDigits: 0 }));
```

```
    animateValue(profitMarginEl, parseElementValue(profitMarginEl),
results.grossProfitMargin, 800, val => `${val.toFixed(1)}%`);
```

```
    animateValue(avgEfficiencyEl, parseElementValue(avgEfficiencyEl),
results.averageEfficiency, 800, val => `${val.toFixed(1)}%`);
```

```
    animateValue(totalIdleTimeEl, parseElementValue(totalIdleTimeEl),
results.totalIdleTime / 60, 800, val => `${val.toFixed(2)} hrs`);
```

```
    animateValue(balanceDelayEl, parseElementValue(balanceDelayEl),
results.balanceDelay, 800, val => `${val.toFixed(1)}%`);
```

```
    animateValue(idleTimeCvEl, parseElementValue(idleTimeCvEl), results.idleTimeCv,
800, val => `${val.toFixed(1)}%`);
```

```
    demandStatusEl.textContent = results.meetsDemand ? "Meets Demand" : "Fails to
Meet Demand";
```

```
    demandStatusEl.className = results.meetsDemand ? "status success" : "status
failure";
```

```
    }
```

```
  }
```

```
const activeTab = document.querySelector('.tab-btn.active')?.dataset.tab;

if (activeTab === 'layout' || activeTab === 'schedule' || activeTab === 'efficiency' || activeTab
=== 'profit') {

    stopAllSimulations();

    if (activeTab === 'layout') LayoutTab.draw();

    if (activeTab === 'schedule') ScheduleTab.draw();

    if (activeTab === 'efficiency') EfficiencyTab.draw();

    if (activeTab === 'profit') ProfitTab.draw();

}
```

```
}
```

```
/**
```

```
* Renders the left sidebar, displaying workstations and their assigned
```

```
* elements for a given employee configuration.
```

```
* @param {number} numEmployees - The number of employees/workstations.
```

```
*/
```

```
function renderWorkstationSidebar(numEmployees) {
```

```
  workstationList.innerHTML = "";
```

```
  const config = state.configData[numEmployees];
```

```
  if (!config || Object.keys(config).length === 0) return;
```

```
  const sortedStationIds = Object.keys(config).sort((a, b) => parseInt(a) - parseInt(b));
```

```
  const numWorkstations = sortedStationIds.length;
```

```
  let maxElementTime = 0;
```

```
  sortedStationIds.forEach(stationId => {
```

```
    config[stationId].forEach(taskId => {
```

```
      const task = state.taskData.get(taskId);
```

```
      if (task && task.elementTime > maxElementTime) {
```

```
        maxElementTime = task.elementTime;
```

```
      }
```

```
    });
```

```
  });
```

```
  if (maxElementTime === 0) return;
```

```
const accentColor = getComputedStyle(root).getPropertyValue('--accent').trim();

sortedStationIds.forEach((stationId, stationIndex) => {

  const elementsInStation = config[stationId];

  const elementColorScale = generateElementColorScale(stationIndex,
numWorkstations, elementsInStation.length);

  const workstationDiv = document.createElement('div');
  workstationDiv.className = 'workstation';

  const title = document.createElement('div');
  title.className = 'workstation-title';
  title.textContent = `Workstation ${stationId}`;
  workstationDiv.appendChild(title);

  const elementsContainer = document.createElement('div');
  elementsContainer.className = 'workstation-elements';
  elementsInStation.forEach((taskId, elementIndex) => {
    const task = state.taskData.get(taskId);
    if (task) {
      const elementColor = elementColorScale(elementIndex);
      const elementRow = document.createElement('div');
      elementRow.className = 'element-row';
      elementRow.title = `Element ${taskId}: ${task.description}`;
      elementRow.dataset.taskId = taskId;

      const barWrapper = document.createElement('div');
```

```
barWrapper.className = 'element-bar-wrapper';

const elementTimeBar = document.createElement('div');
elementTimeBar.className = 'element-time-bar';
const elementBarWidth = (task.elementTime / maxElementTime) * 80;
elementTimeBar.style.width = `${elementBarWidth}%`;
elementTimeBar.style.backgroundColor = accentColor;
elementTimeBar.style.border = `2px solid ${accentColor}`;
elementTimeBar.style.borderRadius = '4px';

const laborTimeBar = document.createElement('div');
laborTimeBar.className = 'labor-time-bar';
laborTimeBar.style.backgroundColor = elementColor;
const laborBarRatio = task.elementTime > 0 ? (task.laborTime / task.elementTime) :
0;
laborTimeBar.style.transform = `scaleX(${laborBarRatio})`;

if (task.elementTime > 2 * task.laborTime) {
  const timeText = document.createElement('span');
  timeText.className = 'element-time-text';
  timeText.textContent = taskId;
  elementTimeBar.appendChild(timeText);
} else {
  const laborText = document.createElement('span');
  laborText.className = 'labor-bar-text';
  laborText.textContent = taskId;
```

```

        elementTimeBar.appendChild(laborText);
    }

    elementTimeBar.appendChild(laborTimeBar);
    barWrapper.appendChild(elementTimeBar);
    elementRow.appendChild(barWrapper);
    elementsContainer.appendChild(elementRow);
}

});
workstationDiv.appendChild(elementsContainer);
workstationList.appendChild(workstationDiv);
});

const firstTitle = workstationList.querySelector('.workstation-title');
const svgContainer = document.getElementById('svg-container');
if (firstTitle && svgContainer) {
    const svgTop = svgContainer.getBoundingClientRect().top;
    const titleTop = firstTitle.getBoundingClientRect().top;
    const currentPadding = parseFloat(getComputedStyle(workstationList).paddingTop) ||
0;
    const offset = svgTop - titleTop;
    const newPadding = Math.max(0, currentPadding + offset);
    workstationList.style.paddingTop = `${newPadding}px`;
}
}

```

```

/**
 * Sets up event listeners for the main financial and operational input controls.
 */

/**
 * UI - Set listeners for the variable inputs.
 */

function setupEventListeners() {
  const inputs = [
    dailyDemandInput, opHoursInput, numEmployeesInput, laborCostInput,
    superSellInput, superCogsInput, ultraSellInput, ultraCogsInput,
    megaSellInput, megaCogsInput
  ];
  attachCommitBehavior(inputs, (id, value) => {
    handleInputChange(id);
  });
}

// UPDATE: Improved input commit behavior to reduce accidental commits
// - Focus clears visual contents but keeps committed value until commit
// - Commit on Enter, Escape (revert), Blur (with special empty handling)
function attachCommitBehavior(inputs, onCommit) {
  const timers = new WeakMap();
  const autoFlag = new WeakMap();

  // Clear all auto-commit flags on global mouseup (end of middle-drag)

```

```
const clearAllAutoFlags = () => {
  inputs.forEach(inp => autoFlag.set(inp, false));
};
document.addEventListener('mouseup', clearAllAutoFlags);

inputs.forEach(input => {
  if (!input) return;

  // Ensure we have a committed baseline value stored
  if (!input.dataset.committedValue) input.dataset.committedValue = input.value ?? "";

  autoFlag.set(input, false);
  input.dataset.awaitingInput = 'false';

  // When user focuses the field: show an empty box with cursor but keep committed
  value stored
  input.addEventListener('focus', (e) => {
    input.dataset.preFocusValue = input.dataset.committedValue ?? "";

    // Only clear visual contents if there's something to hide; leave range inputs alone
    if (input.type !== 'range') {
      // If the user already started typing (awaitingInput), don't clear again
      if (input.dataset.awaitingInput !== 'true') {
        input.dataset.awaitingInput = 'true';

        // Clear displayed value but do not overwrite committedValue
        input.value = "";

        // Keep cursor visible; for some browsers move caret to start
```



```
        try { input.setSelectionRange(0, 0); } catch (_) {}  
    }  
}  
});
```

```
// Middle-button drag enables auto-commit behavior (scrub)
```

```
input.addEventListener('mousedown', (e) => {  
    if (e.button === 1) {  
        autoFlag.set(input, true);  
    }  
});
```

```
// Debounced auto-commit, but only if middle-scrub is active
```

```
input.addEventListener('input', () => {  
    const prevTimer = timers.get(input);  
    if (prevTimer) clearTimeout(prevTimer);
```

```
// Immediate commit for range inputs (sliders) so they update instantly
```

```
if (input.type === 'range') {  
    commitInput(input, onCommit);  
    input.dataset.awaitingInput = 'false';  
    return;  
}
```

```
// If middle-scrub is required for auto-commit, ignore otherwise
```

```
if (!autoFlag.get(input)) {
```

```

    // Mark that user did type something (so blur won't restore)
    input.dataset.awaitingInput = 'true';
    return;
  }

  const t = setTimeout(() => {
    const current = (input.value || '').trim();
    const committed = (input.dataset.committedValue || '').trim();
    if (current !== committed) {
      commitInput(input, onCommit);
    }
  }, 200);
  timers.set(input, t);
});

// Also commit on change as a safe fallback (e.g., some browsers fire change reliably)
input.addEventListener('change', () => {
  const prevTimer = timers.get(input);
  if (prevTimer) clearTimeout(prevTimer);
  commitInput(input, onCommit);
  input.dataset.awaitingInput = 'false';
});

// Commit on Enter, revert on Escape (always active)
input.addEventListener('keydown', (e) => {
  if (e.key === 'Enter') {

```

```

    const prevTimer = timers.get(input);
    if (prevTimer) clearTimeout(prevTimer);
    commitInput(input, onCommit);
    input.dataset.awaitingInput = 'false';
    input.blur();
  } else if (e.key === 'Escape') {
    const prevTimer = timers.get(input);
    if (prevTimer) clearTimeout(prevTimer);
    // Restore previous committed value visually and cancel awaiting state
    input.value = input.dataset.committedValue ?? "";
    input.dataset.awaitingInput = 'false';
    input.blur();
  }
});

```

// Commit on blur (always active) with special behavior:

// - If user focused and did not enter a new value (still empty), restore previous committed value

// - Otherwise attempt to commit (commitInput will validate / clamp / persist)

```

input.addEventListener('blur', () => {
  const prevTimer = timers.get(input);
  if (prevTimer) clearTimeout(prevTimer);

```

```

  const awaiting = input.dataset.awaitingInput === 'true';

```

```

  const text = (input.value || "").trim();

```

```

    if (awaiting && text === '') {

        // User focused but didn't type a new value → restore previous committed value

        input.value = input.dataset.committedValue ?? '';

        input.dataset.awaitingInput = 'false';

        // No commit callback needed because value didn't change

    } else {

        // Either user typed something, or this wasn't an "empty-until-typed" session →
attempt commit

        commitInput(input, onCommit);

        input.dataset.awaitingInput = 'false';

    }

});

});

```

// Add Ctrl+Click to reset functionality

```

const defaultValues = {
    'dailyDemand': 180,
    'opHours': 15.0,
    'numEmployees': 8,
    'laborCost': 25.0,
    'superSell': 1250,
    'superCogs': 450,
    'ultraSell': 1500,
    'ultraCogs': 550,
    'megaSell': 1800,
    'megaCogs': 650

```

```

};

inputs.forEach(input => {
  if (!input) return;
  input.addEventListener("click", (e) => {
    if (e.ctrlKey) {
      e.preventDefault();
      const defaultValue = defaultValues[input.id];
      if (defaultValue !== undefined) {
        const min = input.hasAttribute('min') ? parseFloat(input.min) : -Infinity;
        const max = input.hasAttribute('max') ? parseFloat(input.max) : Infinity;
        const step = parseFloat(input.step) || 1;

        input.value = Math.max(min, Math.min(max, defaultValue));
        commitInput(input, onCommit); // This will now work correctly

        input.style.backgroundColor = getComputedStyle(root).getPropertyValue('--
primary').trim();
        setTimeout(() => { input.style.backgroundColor = ""; }, 200);
      }
    }
  });
});
}

function commitInput(input, onCommit) {

```

```
const raw = (input.value || '').trim();
if (raw === '') {
  // Revert to last committed to avoid propagating invalid/empty state
  input.value = input.dataset.committedValue ?? '';
  return;
}
const n = Number(raw);
if (!Number.isFinite(n)) {
  input.value = input.dataset.committedValue ?? '';
  return;
}
const clamped = clampByField(input.id, n);
input.value = String(clamped);
input.dataset.committedValue = input.value;
if (typeof onCommit === 'function') onCommit(input.id, clamped);
}
```

```
function clampByField(id, n) {
  switch (id) {
    case 'opHours':
      return Math.min(Math.max(n, 0), 24);
    case 'dailyDemand':
      return Math.max(0, Math.floor(n));
    case 'numEmployees':
      return Math.max(1, Math.floor(n));
    case 'laborCost':
```

```

        return Math.max(0, n);
    case 'superSell':
    case 'superCogs':
    case 'ultraSell':
    case 'ultraCogs':
    case 'megaSell':
    case 'megaCogs':
        return Math.max(0, n);
    default:
        return n;
    }
}

/**
 * Sets up event listeners for general UI elements like sidebars and tabs.
 */
function setupUIEventListeners() {
    // --- Create the Auto-Adjust Toggle Switch ---
    const switchContainer = document.createElement('div');
    switchContainer.style.display = 'flex';
    switchContainer.style.alignItems = 'center';

    const switchText = document.createElement('span');
    switchText.textContent = ' Auto\nAdjust';
    switchText.style.marginRight = '0.3em';
    switchText.style.marginLeft = '0.3em';

```

```
switchText.style.fontSize = '0.9em';

const switchLabel = document.createElement('label');
switchLabel.className = 'switch';

const switchInput = document.createElement('input');
switchInput.type = 'checkbox';
switchInput.id = 'autoAdjustToggle';
switchInput.checked = autoAdjustEnabled;

const sliderSpan = document.createElement('span');
sliderSpan.className = 'slider';

switchLabel.append(switchInput, sliderSpan);
switchContainer.append(switchText, switchLabel);

switchInput.addEventListener('change', () => {
  autoAdjustEnabled = switchInput.checked;
});

// --- Position the Switch Next to the "Operational Inputs" Title ---
// Note: This robustly finds the title above the 'dailyDemand' input.

const demandInputContainer = dailyDemandInput.closest('.input-group, .form-group,
div');

if (demandInputContainer) {
  const operationalTitle = demandInputContainer.previousElementSibling;
```



```

    if (operationalTitle && (operationalTitle.tagName === 'H3' || operationalTitle.tagName
=== 'H4')) {

        const titleWrapper = document.createElement('div');

        titleWrapper.style.display = 'flex';

        titleWrapper.style.justifyContent = 'space-between';

        titleWrapper.style.alignItems = 'center';

        // Maintain original spacing by moving the title's margin to the wrapper

        titleWrapper.style.marginBottom =
getComputedStyle(operationalTitle).marginBottom;

        operationalTitle.style.marginBottom = '0';

        // Replace the original title with the new wrapper

        operationalTitle.parentNode.insertBefore(titleWrapper, operationalTitle);

        // Move the title and the new switch group into the wrapper

        titleWrapper.appendChild(operationalTitle);

        titleWrapper.appendChild(switchContainer);
    } else {

        // Fallback if the title can't be found: place it at the top.

        rightSidebar.insertBefore(switchContainer, rightSidebar.firstChild);

    }
}

```

```

leftToggle.addEventListener('click', () => {

    const redrawOnTransitionEnd = () => {

        updateUI();
    }
}

```

```

        document.getElementById('left-sidebar').removeEventListener('transitionend',
redrawOnTransitionEnd);

    };

    document.getElementById('left-sidebar').addEventListener('transitionend',
redrawOnTransitionEnd);

    document.getElementById('left-sidebar').classList.toggle('collapsed');

    const isCollapsed = document.getElementById('left-
sidebar').classList.contains('collapsed');

    leftToggle.innerHTML = isCollapsed ? '&raquo;' : '&laquo;';

});

rightToggle.addEventListener('click', () => {

    const redrawOnTransitionEnd = () => {

        updateUI();

        document.getElementById('right-sidebar').removeEventListener('transitionend',
redrawOnTransitionEnd);

    };

    document.getElementById('right-sidebar').addEventListener('transitionend',
redrawOnTransitionEnd);

    document.getElementById('right-sidebar').classList.toggle('collapsed');

    const isCollapsed = document.getElementById('right-
sidebar').classList.contains('collapsed');

    rightToggle.innerHTML = isCollapsed ? '&laquo;' : '&raquo;';

});

tabs.addEventListener('click', (e) => {

    if (e.target.classList.contains('tab-btn')) {

        const targetTab = e.target.dataset.tab;

        const currentActive = tabs.querySelector('.active');

        if (currentActive && currentActive.dataset.tab === targetTab) {

```

```

        return;
    }
    sessionStorage.setItem("activeTab", targetTab);
    if (currentActive) currentActive.classList.remove('active');
    e.target.classList.add('active');
    visPanels.forEach(panel => {
        panel.style.display = panel.id === `${targetTab}-panel` ? 'block' : 'none';
    });
    workstationList.scrollTop = 0;
    stopAllSimulations();
    if (targetTab === 'precedence') {
        drawPrecedenceChart();
    } else {
        renderActiveTab();
    }
}
});
workstationList.addEventListener('scroll', () => {
    const scrollTop = workstationList.scrollTop;
    const schedulePanel = document.getElementById('schedule-panel');
    const contentGroup = schedulePanel.querySelector('.schedule-content-group');
    if (contentGroup) {
        contentGroup.setAttribute('transform', `translate(0, ${-scrollTop})`);
    }
});
}

```

```

/**
 * UI - Controls tab shift visibility
 */
function handleVisibilityChange() {
    if (document.hidden) {
        // Pause animations without resetting state

        if (animationState && animationState.schedule &&
animationState.schedule.isRunning) {
            animationState.schedule.isPaused = true;
        }

        if (animationState && animationState.layout && animationState.layout.isRunning) {
            animationState.layout.isPaused = true;
        }
    } else {
        // Resume animations; reset lastFrameTime to avoid jumps
        if (animationState && animationState.schedule && animationState.schedule.isPaused)
        {
            animationState.schedule.isPaused = false;
            animationState.schedule.lastFrameTime = performance.now();
        }

        if (animationState && animationState.layout && animationState.layout.isPaused) {
            animationState.layout.isPaused = false;
            animationState.layout.lastFrameTime = performance.now();
        }
    }
}

```

```

/**
 * Sets up the event listener for browser tab visibility changes.
 */
function setupVisibilityListener() {
    document.addEventListener('visibilitychange', handleVisibilityChange, false);
}

/**
 * Initializes Sortable.js for drag-and-drop functionality on workstations.
 */
function setupDragAndDrop() {
    sortableInstances.forEach(instance => instance.destroy());
    sortableInstances = [];
    const workstationElements = document.querySelectorAll('.workstation-elements');
    workstationElements.forEach(el => {
        const instance = new Sortable(el, {
            group: 'shared',
            animation: 150,
            onEnd: function (evt) {
                updateWorkstationOrder();
            }
        });
        sortableInstances.push(instance);
    });
}

```

```
/**  
 * Creates a new D3 tooltip element or returns an existing one.  
 * @param {string} className - The class name for the tooltip.  
 * @returns {d3.Selection} The D3 selection for the tooltip div.  
 */
```

```
function createTooltip(className) {  
  let tooltip = d3.select(` body > .d3-tooltip.${className}`);  
  if (tooltip.empty()) {  
    tooltip = d3.select("body").append("div")  
      .attr("class", `d3-tooltip ${className || ''}`)  
      .style("opacity", 0).style("position", "absolute");  
  }  
  return tooltip;  
}
```

```
/**  
 * Creates a standardized SVG control button.  
 * @param {d3.Selection} parent - The parent G element to append the button to.  
 * @param {object} options - Configuration for the button.  
 * @returns {d3.Selection} The created button group selection.  
 */
```

```
function createControlButton(parent, options) {  
  const { className, text, transform = [0, 0] } = options;  
  const btn = parent.append("g")  
    .attr("class", className)
```

```

        .attr("transform", ` translate(${transform[0]}, ${transform[1]}) ` )
        .style("cursor", "pointer");
    btn.append("rect")
        .attr("width", 28)
        .attr("height", 18)
        .attr("fill", getComputedStyle(root).getPropertyValue('--accent').trim())
        .attr("rx", 3);
    btn.append("text")
        .attr("x", 14)
        .attr("y", 13.5)
        .attr("text-anchor", "middle")
        .attr("fill", getComputedStyle(root).getPropertyValue('--secondary1').trim())
        .style("font-size", "14px")
        .text(text);
    return btn;
}

/**
 * Generates a D3 color scale for elements within a single workstation.
 * @param {number} workstationIndex - The index of the workstation.
 * @param {number} numWorkstations - The total number of workstations.
 * @param {number} numElements - The number of elements in this workstation.
 * @returns {d3.ScaleLinear<string, string>} The D3 color scale.
 */
function generateElementColorScale(workstationIndex, numWorkstations, numElements)
{

```

```

const schemeColors = [
  getComputedStyle(root).getPropertyValue('--primary').trim(),
  getComputedStyle(root).getPropertyValue('--secondary1').trim(),
  getComputedStyle(root).getPropertyValue('--secondary2').trim(),
];

const baseColor = d3.hcl(schemeColors[workstationIndex % schemeColors.length]);
const startColor = baseColor.copy();
startColor.l += 15;
const endColor = baseColor.copy();
endColor.l -= 15;
return d3.scaleLinear()
  .domain([0, numElements > 1 ? numElements - 1 : 1])
  .range([startColor.toString(), endColor.toString()])
  .interpolate(d3.interpolateHcl);
}

```

```
/**
```

```
* -----
```

```
* Backend Logic & Calculations
```

```
*
```

```
* These functions handle core data processing, simulations, and
```

```
* business logic without directly manipulating the DOM.
```

```
* -----
```

```
*/
```

```
/**
```


* Handles changes from any of the main input controls, triggering

* recalculations and UI updates.

* @param {string} driverId - The ID of the input element that changed.

*/

```
function handleInputChange(driverId) {
```

```
  if (isRecalculating) return;
```

```
  isRecalculating = true;
```

```
  const isFinancialDriver = ['laborCost', 'superSell', 'superCogs', 'ultraSell', 'ultraCogs',  
  'megaSell', 'megaCogs'].includes(driverId);
```

```
  if (isFinancialDriver) {
```

```
    calculateOptimalProfitData();
```

```
  }
```

```
  try {
```

```
    let dailyDemand = parseInt(dailyDemandInput.value) || 1;
```

```
    let opHours = parseFloat(opHoursInput.value) || 1;
```

```
    let numEmployees = parseInt(numEmployeesInput.value);
```

```
    const isOperationalDriver = ['dailyDemand', 'opHours',  
  'numEmployees'].includes(driverId);
```

```
    if (isOperationalDriver) {
```

```
      workstationList.scrollTop = 0;
```

```
    }
```

```
    if (driverId === 'numEmployees') {
```

```
    state.configData[numEmployees] =  
    JSON.parse(JSON.stringify(originalConfigData[numEmployees]));  
  }
```

```
    if (isOperationalDriver && autoAdjustEnabled) {  
      let currentBottleneck =  
        calculateWorkstationDetails(numEmployees).bottleneckTime;  
      if (currentBottleneck === 0) {  
        console.error(` No valid workstation data for ${numEmployees} employees.  
Aborting.`);  
        isRecalculating = false;  
        return;  
      }  
      let taktTime = (opHours * 60) / dailyDemand;  
      if (taktTime < currentBottleneck) {  
        if (driverId === 'numEmployees') {  
          let requiredHours = (currentBottleneck * dailyDemand) / 60;  
          opHours = requiredHours <= 24 ? roundUpToQuarter(requiredHours) : 24;  
          if (requiredHours > 24) {  
            dailyDemand = Math.floor((24 * 60) / currentBottleneck);  
          }  
        } else {  
          numEmployees = findBestEmployeeFit(taktTime, numEmployees);  
        }  
      }  
      taktTime = (opHours * 60) / dailyDemand;  
      if (taktTime < MIN_TAKT_TIME) {
```

```

    if (driverId === 'dailyDemand') {
        opHours = roundUpToQuarter((MIN_TAKT_TIME * dailyDemand) / 60);
        if (opHours > 24) opHours = 24;
    } else {
        dailyDemand = Math.floor((opHours * 60) / MIN_TAKT_TIME);
    }
}
}

```

```

dailyDemandInput.value = Math.round(dailyDemand);
opHoursInput.value = opHours.toFixed(2);
numEmployeesInput.value = numEmployees;

```

```

    updateUI();
} catch (error) {
    console.error("Error during input handling:", error);
} finally {
    isRecalculating = false;
}
}

```

```

/**

```

```

 * Calculates the cycle time for each workstation and identifies the

```

```

 * line's bottleneck and fastest station times.

```

```

 * @param {number} numEmployees - The number of employees/workstations in the config.

```

```

 * @returns {{workstations: Array<object>, bottleneckTime: number, fastestTime: number}}

```

```

*/

function calculateWorkstationDetails(numEmployees) {

  const config = state.configData[numEmployees];

  if (!config || Object.keys(config).length === 0) return { workstations: [], bottleneckTime: 0,
fastestTime: Infinity };

  let workstations = [],

  bottleneckTime = 0,

  fastestTime = Infinity;

  for (const stationId in config) {

    let totalLaborTime = 0;

    let totalElementTime = 0;

    config[stationId].forEach(taskId => {

      const task = state.taskData.get(taskId);

      if (task) {

        totalLaborTime += task.laborTime;

        totalElementTime += task.elementTime;

      }

    });

    const stationLength = totalElementTime * 15;

    workstations.push({ id: stationId, cycleTime: totalLaborTime, stationLength:
stationLength });

    if (totalLaborTime > bottleneckTime) bottleneckTime = totalLaborTime;

```

```

        if (totalLaborTime < fastestTime && totalLaborTime > 0) fastestTime = totalLaborTime;
    }

    return { workstations, bottleneckTime, fastestTime };
}

/**
 * Calculates all key performance indicators (KPIs) for the assembly line
 * based on the current operational and financial inputs.
 * @param {object} op - Operational inputs { dailyDemand, opHours, numEmployees }.
 * @param {object} fin - Financial inputs { laborCost, ...sell/cogs prices }.
 * @returns {object} An object containing all calculated metrics.
 */
function calculateMetrics(op, fin) {
    const wsDetails = calculateWorkstationDetails(op.numEmployees);
    const fullTotalOpMinutes = Math.floor(op.opHours * 4) * 15;
    const bottleneckCycleTime = wsDetails.bottleneckTime;
    const productSpacing = wsDetails.fastestTime === Infinity ? 0 : wsDetails.fastestTime *
15;

    if (productSpacing <= 0 || bottleneckCycleTime <= 0) {
        return {
            wip: 0, throughputUnitsPerHour: 0, conveyorSpeed: 0, productSpacing: 0,
            dailyGrossProfit: -(op.numEmployees * op.opHours * (fin.laborCost || 0)),
            grossProfitMargin: 0, meetsDemand: false, effectiveCycleTime: Infinity, workstations:
wsDetails.workstations,

```

```
    averageEfficiency: 0, totalIdleTime: fullTotalOpMinutes * op.numEmployees,  
    balanceDelay: 100, idleTimeCv: 0, throughputUnitsPerDay: 0
```

```
};
```

```
}
```

```
let requiredTaktTime;
```

```
const demandIntervals = op.dailyDemand > 1 ? op.dailyDemand - 1 : 0;
```

```
const throughputTimeAsIntervals = ASSEMBLY_LINE_LENGTH / productSpacing;
```

```
if (op.dailyDemand <= 1) {
```

```
    requiredTaktTime = Infinity;
```

```
} else {
```

```
    const totalIntervals = demandIntervals + throughputTimeAsIntervals;
```

```
    requiredTaktTime = fullTotalOpMinutes / totalIntervals;
```

```
}
```

```
const meetsDemand = bottleneckCycleTime <= requiredTaktTime;
```

```
const effectiveCycleTime = meetsDemand ? requiredTaktTime : bottleneckCycleTime;
```

```
const cycleTimeToUseForSpeedCalc = isFinite(effectiveCycleTime) ? effectiveCycleTime :  
bottleneckCycleTime;
```

```
const conveyorSpeed = productSpacing / cycleTimeToUseForSpeedCalc;
```

```
const actualThroughputTime = (ASSEMBLY_LINE_LENGTH / productSpacing) *  
effectiveCycleTime;
```

```
let throughputUnitsPerDay;
```

```
if (fullTotalOpMinutes < actualThroughputTime) {
```

```
    throughputUnitsPerDay = 0;
```

```
} else if (op.dailyDemand <= 1) {
```

```
    throughputUnitsPerDay = 1;
```

```

} else {
    const launchWindowMinutes = fullTotalOpMinutes - actualThroughputTime;
    throughputUnitsPerDay = Math.floor(launchWindowMinutes / effectiveCycleTime) + 1;
}

const wip = ASSEMBLY_LINE_LENGTH / productSpacing;
let actualProductionMinutes;
if (op.dailyDemand <= 0) {
    actualProductionMinutes = 0;
} else if (op.dailyDemand === 1) {
    actualProductionMinutes = actualThroughputTime;
} else {
    actualProductionMinutes = effectiveCycleTime * (demandIntervals) +
actualThroughputTime;
}

const throughputUnitsPerHour = actualProductionMinutes > 0 ? (op.dailyDemand /
actualProductionMinutes) * 60 : 0;

let totalWorkstationCycleTime = 0;
wsDetails.workstations.forEach(ws => {
    totalWorkstationCycleTime += ws.cycleTime;

    ws.efficiency = bottleneckCycleTime > 0 ? (ws.cycleTime / bottleneckCycleTime) * 100 :
0;

    const idleTimePerCycle = bottleneckCycleTime - ws.cycleTime;
    ws.dailyIdleTime = idleTimePerCycle * throughputUnitsPerDay;
});

```

```

const totalAvailableTime = op.numEmployees * fullTotalOpMinutes;

const totalDailyLaborCost = op.numEmployees * op.opHours * (fin.laborCost || 0);

const totalProductiveTime = throughputUnitsPerDay * totalWorkstationCycleTime;

const totalIdleTime = Math.max(0, totalAvailableTime - totalProductiveTime);

const averageEfficiency = totalAvailableTime > 0 ? (totalProductiveTime /
totalAvailableTime) * 100 : 0;


const efficiencies = wsDetails.workstations.map(ws => ws.efficiency);

const balanceActive = efficiencies.length > 0 ? efficiencies.reduce((a, b) => a + b, 0) /
efficiencies.length : 0;

const balanceDelay = 100 - balanceActive;


const idleTimesPerCycle = wsDetails.workstations.map(ws => bottleneckCycleTime -
ws.cycleTime);

const idleMean = idleTimesPerCycle.length > 0 ? idleTimesPerCycle.reduce((a, b) => a +
b, 0) / idleTimesPerCycle.length : 0;

const stdDev = Math.sqrt(idleTimesPerCycle.map(x => Math.pow(x - idleMean,
2)).reduce((a, b) => a + b, 0) / (idleTimesPerCycle.length || 1));

const idleTimeCv = idleMean > 0 ? (stdDev / idleMean) * 100 : 0;


const totalRevenue = throughputUnitsPerDay * ((BUILD_RATIOS.super * (fin.superSell ||
0)) + (BUILD_RATIOS.ultra * (fin.ultraSell || 0)) + (BUILD_RATIOS.mega * (fin.megaSell || 0)));

const totalCogs = throughputUnitsPerDay * ((BUILD_RATIOS.super * (fin.superCogs || 0))
+ (BUILD_RATIOS.ultra * (fin.ultraCogs || 0)) + (BUILD_RATIOS.mega * (fin.megaCogs || 0)));

const dailyGrossProfit = totalRevenue - totalCogs - totalDailyLaborCost;

const grossProfitMargin = totalRevenue > 0 ? (dailyGrossProfit / totalRevenue) * 100 : 0;


return {

```



```

        wip, throughputUnitsPerHour, conveyorSpeed, productSpacing, dailyGrossProfit,
        grossProfitMargin, meetsDemand, effectiveCycleTime, workstations:
wsDetails.workstations,

        averageEfficiency, totalIdleTime, balanceDelay, idleTimeCv, throughputUnitsPerDay
    };
}

```

```
/**
```

```

* Finds the minimum number of employees required to meet a given takt time,
* starting the search from a given employee count.
* @param {number} requiredTaktTime - The target cycle time to meet.
* @param {number} startingCount - The number of employees to start searching from.
* @returns {number} The optimal number of employees.
*/

```

```

function findBestEmployeeFit(requiredTaktTime, startingCount) {
    for (let i = startingCount; i <= 13; i++) {
        if (calculateWorkstationDetails(i).bottleneckTime <= requiredTaktTime) return i;
    }
    return 13;
}

```

```
/**
```

```

* Updates the workstation configuration in the global state based on the
* current DOM structure after a drag-and-drop operation.
*/

```

```

function updateWorkstationOrder() {

```

```

const numEmployees = parseInt(numEmployeesInput.value);

const newConfig = {};

document.querySelectorAll('.workstation').forEach(workstationDiv => {
  const title = workstationDiv.querySelector('.workstation-title').textContent;
  const stationMatch = title.match(/d+/);
  if (stationMatch) {
    const stationId = stationMatch[0];
    const elements = [];
    workstationDiv.querySelectorAll('.element-row').forEach(elRow => {
      elements.push(parseInt(elRow.dataset.taskId));
    });
    newConfig[stationId] = elements;
  }
});

state.configData[numEmployees] = newConfig;

const invalidPrecedenceMap = validatePrecedence();
invalidPrecedenceNodes = new Set(Array.from(invalidPrecedenceMap.keys()));

if (document.querySelector('.tab-btn[data-tab="precedence"].active')) {
  updatePrecedenceChart();
}

setTimeout(updateUI, 0);
}

```

```

/**
 * Validates the current element order against the precedence map,
 * adding error classes to invalid elements in the DOM.
 * @returns {Set<number>} A set of task IDs that violate precedence rules.
 */
function validatePrecedence() {
    const seenTasks = new Set();
    const allElementRows = document.querySelectorAll('element-row');
    const invalidNodes = new Set();

    allElementRows.forEach(row => {
        const taskId = parseInt(row.dataset.taskId);
        const predecessors = precedenceMap.get(taskId) || new Set();
        let isValid = true;

        for (const pId of predecessors) {
            if (!seenTasks.has(pId)) {
                isValid = false;
                break;
            }
        }

        if (!isValid) {
            row.classList.add('precedence-error');
            invalidNodes.add(taskId);
        } else {

```

```

        row.classList.remove('precedence-error');
    }
    seenTasks.add(taskId);
});

return invalidNodes;
}

/**
 * Generates a mixed-model production sequence based on daily demand
 * using the Model-Mix Sequencing Algorithm (MSSA).
 * @param {number} dailyDemand - The total number of units to produce.
 * @returns {Array<number>} An array representing the production queue (1=Super, 2=Ultra,
 3=Mega).
 */
function generateProductionQueue(dailyDemand) {
    const productionQueue = [];
    const modelRatios = Object.values(BUILD_RATIOS);
    let modelDemand = [];
    let sumOfDemands = 0;

    for (let i = 0; i < modelRatios.length - 1; i++) {
        const demand = Math.round(modelRatios[i] * dailyDemand);
        modelDemand.push(demand);
        sumOfDemands += demand;
    }

```

```

modelDemand.push(dailyDemand - sumOfDemands);

let totalDemand = dailyDemand;

const wModel = modelDemand.map(d => (d > 0 ? totalDemand / d : Infinity));

let aModel = wModel.map(w => w / 2);

for (let j = 0; j < dailyDemand; j++) {
    let minIndex = -1;
    let minValue = Infinity;

    for (let k = 0; k < aModel.length; k++) {
        if (modelDemand[k] > 0 && aModel[k] < minValue) {
            minValue = aModel[k];
            minIndex = k;
        }
    }

    if (minIndex === -1) {
        break;
    }

    productionQueue.push(minIndex + 1);
    aModel[minIndex] += wModel[minIndex];
    modelDemand[minIndex]--;
}

return productionQueue;
}

```

```
/**  
 * Generates a unique key based on the current financial inputs  
 * for caching profit calculation results.  
 * @returns {string} The cache key.  
 */
```

```
function getFinancialInputsKey() {  
  const finInputs = {  
    laborCost: parseFloat(laborCostInput.value),  
    superSell: parseFloat(superSellInput.value),  
    superCogs: parseFloat(superCogsInput.value),  
    ultraSell: parseFloat(ultraSellInput.value),  
    ultraCogs: parseFloat(ultraCogsInput.value),  
    megaSell: parseFloat(megaSellInput.value),  
    megaCogs: parseFloat(megaCogsInput.value),  
  };  
  return 'profitDataCache-v1-' + JSON.stringify(finInputs);  
}
```

```
/**  
 * Checks for cached profit data in session storage. If found, loads it.  
 * Otherwise, triggers a new calculation.  
 */
```

```
function runProfitCalculation() {  
  const cacheKey = getFinancialInputsKey();  
  try {
```

```

const cachedData = sessionStorage.getItem(cacheKey);
if (cachedData) {
    console.log("Loading profit data from session cache.");
    profitMaximizationCache = { key: cacheKey, data: JSON.parse(cachedData) };
    if (document.querySelector('.tab-btn.active')?.dataset.tab === 'profit') {
        ProfitTab.draw();
    }
} else {
    console.log("No valid cache found. Calculating optimal profit data for the first time.");
    calculateOptimalProfitData();
}
} catch (e) {
    console.error("Could not access session storage. Recalculating profit data.", e);
    calculateOptimalProfitData();
}
}

/**
 * Finds the most profitable and highest margin configuration for a single demand value.
 * @param {number} demand - The daily demand to analyze.
 * @param {object} finInputs - The financial inputs object.
 * @param {Map<number, number>} maxDemandMap - A pre-calculated map of max
demands per workstation.
 * @returns {{profitResult: object, marginResult: object}} An object containing the results.
 */
function findOptimalConfigForDemand(demand, finInputs, maxDemandMap) {

```

```

let maxProfit = -Infinity;

let maxProfitConfig = { emp: 0, hrs: 0 };

let maxMargin = -Infinity;

let maxMarginConfig = { emp: 0, hrs: 0 };


for (let numEmployees = 3; numEmployees <= 13; numEmployees++) {
  if (demand > (maxDemandMap.get(numEmployees) || 0)) {
    continue;
  }

  if (!originalConfigData[numEmployees] ||
Object.keys(originalConfigData[numEmployees]).length === 0) continue;


  const { bottleneckTime, fastestTime } = calculateWorkstationDetails(numEmployees);

  if (bottleneckTime <= 0 || !isFinite(fastestTime) || fastestTime <= 0) continue;


  const productSpacing = fastestTime * 15;

  const throughputTime = (ASSEMBLY_LINE_LENGTH / productSpacing) *
bottleneckTime;

  const totalRequiredMinutes = (demand > 1 ? (demand - 1) * bottleneckTime : 0) +
throughputTime;

  const minRequiredHours = totalRequiredMinutes / 60;


  if (minRequiredHours > 24) continue;


  const startHours = roundUpToQuarter(minRequiredHours);

  for (let opHours = startHours; opHours <= 24; opHours += 0.25) {

```



```
    const metrics = calculateMetrics({ dailyDemand: demand, opHours,
numEmployees }, finInputs);
```

```
    if (metrics && metrics.throughputUnitsPerDay >= demand) {
        if (metrics.dailyGrossProfit > maxProfit) {
            maxProfit = metrics.dailyGrossProfit;
            maxProfitConfig = { emp: numEmployees, hrs: opHours };
        }
        if (metrics.grossProfitMargin > maxMargin) {
            maxMargin = metrics.grossProfitMargin;
            maxMarginConfig = { emp: numEmployees, hrs: opHours };
        }
        break;
    }
}
```

```
const profitResult = { demand, value: isFinite(maxProfit) ? maxProfit : 0, config:
maxProfitConfig };
```

```
const marginResult = { demand, value: isFinite(maxMargin) ? maxMargin : 0, config:
maxMarginConfig };
```

```
return { profitResult, marginResult };
}
```

```
/**
```

```
* Calculates the optimal profit for every demand level from 50 to 552 for the Profit Tab.
```

```
*/
```

```
async function calculateOptimalProfitData() {
```

```
  if (isProfitCalculating) return;
```

```
  isProfitCalculating = true;
```

```
  const finInputs = {
```

```
    laborCost: parseFloat(laborCostInput.value),
```

```
    superSell: parseFloat(superSellInput.value),
```

```
    superCogs: parseFloat(superCogsInput.value),
```

```
    ultraSell: parseFloat(ultraSellInput.value),
```

```
    ultraCogs: parseFloat(ultraCogsInput.value),
```

```
    megaSell: parseFloat(megaSellInput.value),
```

```
    megaCogs: parseFloat(megaCogsInput.value),
```

```
  };
```

```
  const key = getFinancialInputsKey() + '-demand50plus';
```

```
  if (document.querySelector('.tab-btn.active')?.dataset.tab === 'profit') {
```

```
    ProfitTab.draw(); // Show loading state
```

```
  }
```

```
  setTimeout(() => {
```

```
    const profitData = [];
```

```
    const marginData = [];
```

```
    const originalStateConfig = JSON.parse(JSON.stringify(state.configData));
```

```
    try {
```

```

state.configData = originalConfigData;

const maxDemandMap = new Map(WORKSTATION_CAPACITIES.map(c => [c.ws,
c.maxDemand]));

for (let demand = 50; demand <= 552; demand++) {

    const { profitResult, marginResult } = findOptimalConfigForDemand(demand,
finInputs, maxDemandMap);

    profitData.push(profitResult);

    marginData.push(marginResult);

}

const calculatedData = { profitData, marginData };
profitMaximizationCache = { key, data: calculatedData };
try {
    sessionStorage.setItem(key, JSON.stringify(calculatedData));
} catch (e) {
    console.error("Could not save profit data to session storage.", e);
}
} finally {
    state.configData = originalStateConfig;
    isProfitCalculating = false;
    if (document.querySelector('.tab-btn.active')?.dataset.tab === 'profit') {
        ProfitTab.draw();
    }
}
}, 200);
}

```

```
/**
```

```
* Checks if a specific assembly element is used in the construction
```

```
* of a given refrigerator model.
```

```
* @param {number} elementId - The ID of the assembly element.
```

```
* @param {number} modelId - The ID of the model (1=Super, 2=Ultra, 3=Mega).
```

```
* @returns {boolean} True if the element is used for the model.
```

```
*/
```

```
function doesElementBuildModel(elementId, modelId) {
```

```
    const task = state.taskData.get(elementId);
```

```
    if (!task) return false;
```

```
    const modelMap = { 1: 'Super', 2: 'Ultra', 3: 'Mega' };
```

```
    const modelFieldName = modelMap[modelId];
```

```
    return task[modelFieldName] > 0;
```

```
}
```

```
/**
```

```
* Runs a detailed simulation of the production day to generate task data
```

```
* for the Gantt chart visualization.
```

```
* @returns {{tasks: Array<object>}} An object containing the list of simulated tasks.
```

```
*/
```

```
function runGanttSimulation() {
```

```
    const numEmployees = parseInt(numEmployeesInput.value);
```

```
    const dailyDemand = parseInt(dailyDemandInput.value);
```

```
    const opHours = parseFloat(opHoursInput.value);
```

```
    const config = state.configData[numEmployees];
```

```
if (!config || Object.keys(config).length === 0 || invalidPrecedenceNodes.size > 0) {  
  return { tasks: [] };  
}
```

```
const productionQueue = generateProductionQueue(dailyDemand);  
const metrics = calculateMetrics({ dailyDemand, opHours, numEmployees }, {});  
const launchInterval = metrics.effectiveCycleTime;  
const conveyorSpeed = metrics.conveyorSpeed;
```

```
if (conveyorSpeed <= 0 || !isFinite(conveyorSpeed)) {  
  return { tasks: [] };  
}
```

```
let allFinishedTasks = [];  
let arrivalsForNextStation = productionQueue.map((modelId, index) => {  
  let arrivalTime = index * launchInterval;  
  if (index === 0 && isNaN(arrivalTime)) {  
    arrivalTime = 0;  
  }  
  return {  
    modelId: modelId,  
    arrivalTime: arrivalTime,  
    uniqueId: `${modelId}-${index}`  
  };  
});
```

```

const sortedWorkstationIds = Object.keys(config).sort((a, b) => parseInt(a) - parseInt(b));
const totalElementTimeOfLine = sortedWorkstationIds.reduce((lineSum, stationId) => {
    const elements = config[stationId] || [];

    const stationTime = elements.reduce((stationSum, elId) => stationSum +
(state.taskData.get(elId)?.elementTime || 0), 0);

    return lineSum + stationTime;
}, 0);

const totalPhysicalThroughputTime = ASSEMBLY_LINE_LENGTH / conveyorSpeed;

for (const stationId of sortedWorkstationIds) {
    const elements = config[stationId] || [];

    if (elements.length === 0) continue;

    let workerFreeTime = 0;

    let processedModels = [];

    const totalElementTimeInStation = elements.reduce((sum, elId) => sum +
(state.taskData.get(elId)?.elementTime || 0), 0);

    let travelTimeMinutes = 0;

    if (totalElementTimeOfLine > 0) {
        travelTimeMinutes = (totalElementTimeInStation / totalElementTimeOfLine) *
totalPhysicalThroughputTime;
    }

    arrivalsForNextStation.sort((a, b) => a.arrivalTime - b.arrivalTime);

    for (const model of arrivalsForNextStation) {
        if (!isFinite(model.arrivalTime)) continue;

```

```

const startProcessingTime = Math.max(model.arrivalTime, workerFreeTime);
let currentTaskTime = startProcessingTime;

for (const elementId of elements) {
  if (doesElementBuildModel(elementId, model.modelId)) {
    const task = state.taskData.get(elementId);
    if (task) {
      const taskStartTime = currentTaskTime;
      const taskEndTime = taskStartTime + task.elementTime;
      allFinishedTasks.push({
        workstationId: `WS ${stationId}`,
        modelId: model.modelId,
        taskId: elementId,
        startTime: taskStartTime,
        endTime: taskEndTime,
        uniqueId: model.uniqueId
      });
      currentTaskTime = taskEndTime;
    }
  }
}

const endProcessingTime = currentTaskTime;
workerFreeTime = endProcessingTime;

const endTravelTime = model.arrivalTime + travelTimeMinutes;
const exitTime = Math.max(endProcessingTime, endTravelTime);

```

```

        processedModels.push({ ...model, arrivalTime: exitTime });
    }
    arrivalsForNextStation = processedModels;
}

return { tasks: allFinishedTasks };
}

/**
 * Rounds a number up to the nearest quarter (0.25).
 * @param {number} value - The number to round.
 * @returns {number} The rounded number.
 */
function roundUpToQuarter(value) {
    return Math.ceil(value / 0.25) * 0.25;
}

/**
 * -----
 * Visualization Panels (Tabs)
 *
 * These functions are responsible for rendering the primary content
 * for each of the main application tabs.
 * -----
 */

```



```

/**
 * @tab General
 * Renders the content for the currently active visualization tab.
 */
function renderActiveTab() {
    const activeTab = document.querySelector('.tab-btn.active').dataset.tab;
    if (activeTab === 'overview') drawOverviewPanel();
    else if (activeTab === 'precedence') drawPrecedenceChart();
    else if (activeTab === 'schedule') ScheduleTab.draw();
    else if (activeTab === 'efficiency') EfficiencyTab.draw();
    else if (activeTab === 'layout') LayoutTab.draw();
    else if (activeTab === 'profit') ProfitTab.draw();
    else if (activeTab === 'investment') drawInvestmentPanel();
    else if (activeTab === 'location') LocationTab.draw();
}

```

```

/**
 * @tab Overview
 * Fetches and renders the content for the Overview panel.
 */
async function drawOverviewPanel() {
    const svg = d3.select("#overview-panel");
    svg.selectAll("*").remove();
    const fo = svg.append("foreignObject")
        .attr("x", 0)

```

```

        .attr("y", 0)
        .attr("width", "100%")
        .attr("height", "100%");
const container = fo.append("xhtml:div")
    .attr("class", "overview-container");

try {
    const response = await fetch('Pages/overview.html');
    if (!response.ok) {
        throw new Error(`Failed to load HTML: ${response.statusText}`);
    }
    let html = await response.text();

    const replacements = {
        'overview-num-employees': document.getElementById('numEmployees')?.value || '8',
        'overview-daily-demand': document.getElementById('dailyDemand')?.value || '180',
        'overview-op-hours': document.getElementById('opHours')?.value || '15',
        'overview-labor-cost': parseFloat(document.getElementById('laborCost')?.value ||
'25.00').toFixed(2)
    };

    for (const [id, value] of Object.entries(replacements)) {
        const regex = new RegExp(`(<span id="${id}">)(.*?)(<\\span>)`);
        html = html.replace(regex, ` $1${value}$3 `);
    }

```

```

        container.html(html);
    } catch (error) {
        console.error("Could not render Overview panel:", error);
        container.html(`<p style="padding: 2rem; text-align: center;">Error: Could not load
overview content.</p>`);
    }
}

```

```

// Run the application

```

```

Main();

```

```

/**

```

```

 * =====

```

```

 * EfficiencyTab IIFE Module

```

```

 *

```

```

 * Encapsulates all logic for rendering the multi-chart efficiency

```

```

 * analysis dashboard.

```

```

 * =====

```

```

 */

```

```

const EfficiencyTab = (function () {

```

```

    /**

```

```

    * @tab Efficiency

```

```

    * Draws the efficiency analysis dashboard, including pie charts,

```

```

    * idle time clocks, and summary statistics. This is the main

```

```

    * public method exposed by the IIFE.

```

```

    */

```

```

function draw() {

  // --- INITIAL SETUP ---

  const svg = d3.select("#efficiency-panel");

  // Get the available width and height of the container panel.

  const { clientWidth: panelWidth, clientHeight: panelHeight } =
document.getElementById('svg-container');

  // --- DATA CALCULATION ---

  // Gather current operational inputs and calculate all performance metrics.

  const opInputs = { dailyDemand: +dailyDemandInput.value, opHours:
+opHoursInput.value, numEmployees: +numEmployeesInput.value };

  const finInputs = { laborCost: +laborCostInput.value };

  const results = calculateMetrics(opInputs, finInputs);

  // If results are invalid or empty, clear the panel and display a message.
  if (!results || !results.workstations || results.workstations.length === 0) {

    svg.selectAll("*").remove();

    svg.append("text").attr("x", panelWidth / 2).attr("y", panelHeight / 2).attr("text-
anchor", "middle").text("No data available for efficiency analysis.");

    return;

  }

  // Cancel any previous animation loops to prevent conflicts.

  if (animationState.efficiency && animationState.efficiency.frameId) {

    cancelAnimationFrame(animationState.efficiency.frameId);

    animationState.efficiency.frameId = null;

    animationState.efficiency.isRunning = false;

  }

  // --- ROOT GROUP & RESPONSIVE LAYOUT ---

  // Use a persistent root group ('g') for all elements to help D3 manage transitions.

```

```

const effRoot = svg.selectAll("g#eff-root").data([null]).join("g").attr("id", "eff-root");

// Define padding and calculate available drawing area.

const padding = 20;

const availableWidth = panelWidth - (2 * padding);

const availableHeight = panelHeight - (2 * padding);

// Divide the layout into 4 rows: 1 for the summary, 3 for workstation charts.

const rows = 4;

const rowHeight = availableHeight / rows;

// Calculate a uniform radius for all pie and clock charts for a consistent look.

const maxPieRadius = Math.min(availableWidth / 15, (rowHeight * 0.75) / 2);

const maxClockRadius = Math.min(availableWidth / 40, (rowHeight * 0.75) / 4);

const pieRadius = maxPieRadius;

const clockRadius = maxClockRadius;

/**
 * Calculates the transform (x, y position) for a workstation chart
 * based on its index, arranging them in a 4-5-4 grid pattern.
 * @param {number} i - The zero-based index of the workstation.
 * @returns {string} The SVG transform string.
 */

const layoutTransform = (i) => {
  let row, col, colsInRow;

  if (i < 4) { row = 1; col = i; colsInRow = 4; } // First row of workstations

  else if (i < 9) { row = 2; col = i - 4; colsInRow = 5; } // Second row

  else { row = 3; col = i - 9; colsInRow = 4; } // Third row

  const itemWidth = availableWidth / colsInRow;

```

```
    const x = padding + col * itemWidth + itemWidth / 2; // Center horizontally in its column.
```

```
    const y = padding + row * rowHeight + rowHeight / 2 + rowHeight * 0.05; // Center vertically in its row.
```

```
    return `translate(${x},${y})` ;
  };

  // --- WORKSTATION GROUPS (Data Binding) ---

  // Bind workstation data to groups. Using a key (d.id) allows D3 to track
  // which elements are new, which are being updated, and which are removed.
  const wsSel = effRoot.selectAll("g.ws").data(results.workstations, d => d.id);

  // Create new groups for any new workstations (the 'enter' selection).
  const wsEnter = wsSel.enter()

    .append("g")

    .attr("class", "ws")

    .attr("transform", (d, i) => layoutTransform(i)); // Set initial position.

  // Define offsets for the pie and clock within each workstation group.
  const centerDistance = (pieRadius + clockRadius) * 1.1;
  const chartsGroupOffset = rowHeight * 0.12;
  const pieOffsetX = chartsGroupOffset - centerDistance / 2;
  const clockOffsetX = chartsGroupOffset + centerDistance / 2;

  // Create subgroups for pie and clock charts on new elements only.
  wsEnter.append("g").attr("class", "pie").attr("transform", `translate(${pieOffsetX}, 0)`);
  wsEnter.append("g").attr("class", "clock").attr("transform", `translate(${clockOffsetX}, 0)`);

  // Add the workstation title heading.
  wsEnter.append("text")
```

```

        .attr("class", "ws-heading").attr("x", 0).attr("y", -Math.max(pieRadius + rowHeight *
0.05, rowHeight * 0.35))

        .attr("text-anchor", "middle").style("font-size", `${Math.max(Math.min(rowHeight *
0.08, availableWidth * 0.03), 12)}px` )

        .style("font-weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--
accent').trim());

// Merge the enter selection with the update selection (existing elements).

const wsMerge = wsEnter.merge(wsSel);

// Animate all workstations (new and existing) to their correct positions.

wsMerge.transition().duration(750).attr("transform", (d, i) => layoutTransform(i));

// Update sub-group positions and heading text for all workstations.

wsMerge.select("g.pie").attr("transform", `translate(${pieOffsetX}, 0)`);

wsMerge.select("g.clock").attr("transform", `translate(${clockOffsetX}, 0)`);

wsMerge.select("text.ws-heading")

    .attr("y", -Math.max(pieRadius + rowHeight * 0.05, rowHeight * 0.35))

    .style("font-size", `${Math.max(Math.min(rowHeight * 0.08, availableWidth * 0.03),
12)}px` )

    .text(d => `Workstation ${d.id}`);

// Remove any workstation groups that no longer have data (the 'exit' selection).

wsSel.exit().remove();

// --- PIE CHARTS (Productive vs. Idle Time) ---

const arc = d3.arc().innerRadius(0).outerRadius(pieRadius); // Arc generator for the
pies.

wsMerge.each(function (ws) { // Iterate over each workstation group.

    const pieGroup = d3.select(this).select("g.pie");

    const totalOpMinutes = opInputs.opHours * 60;

    const productiveMinutes = ws.cycleTime * results.throughputUnitsPerDay;

```

```

    const productiveRatio = totalOpMinutes > 0 ? Math.min(1, productiveMinutes /
totalOpMinutes) : 0;

    const productivePercentage = productiveRatio * 100;

    // Define data for two slices: Productive and Idle.

    const workAngle = productiveRatio * 2 * Math.PI;

    const shouldHideIdleSlice = productivePercentage >= 99.5; // Hide idle slice near
100% to avoid visual glitch.

    const data = [

        { label: "Productive", startAngle: 0, endAngle: workAngle, value:
Math.min(productivePercentage, 99.99) },

        { label: "Idle", startAngle: workAngle, endAngle: 2 * Math.PI, value: Math.max(100 -
productivePercentage, 0.01), hidden: shouldHideIdleSlice }

    ];

    const slices = pieGroup.selectAll("path.slice").data(data, d => d.label); // Bind slice
data.

    const slicesEnter = slices.enter().append("path").attr("class", "slice") // Create new
paths.

        .attr("fill", d => d.label === "Productive" ?
getComputedStyle(root).getPropertyValue('--primary') :
getComputedStyle(document.documentElement).getPropertyValue('--secondary1'))

        .attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-
width", 1.5)

        .each(function (d) { this._current = { ...d, startAngle: 0, endAngle: 0 }; }) // Store initial
state for animation.

        .attr("d", function (d) { return arc(this._current); });

    const slicesMerged = slicesEnter.merge(slices);

    // Animate the shape of the slices.

    slicesMerged.transition("shape").duration(750).attrTween("d", function (d) {

        const i = d3.interpolate(this._current || d, d);

```



```

    this._current = i(1);

    return t => arc(i(t));

  });

  // Animate the opacity (fade out idle slice slowly, fade in quickly).
  slicesMerged.each(function (d) {

    const element = d3.select(this);

    const targetOpacity = d.hidden ? 0 : 1;

    const duration = (targetOpacity === 0 && (element.style("opacity") || 1) > 0.5) ?
1600 : 200;

    element.transition("opacity").duration(duration).style("opacity", targetOpacity);

  });

  slices.exit().remove();

  // Percentage text display in the center of the pie.

  const pieTextBg = pieGroup.selectAll("circle.pie-text-bg").data([null]).join("circle") //
Background circle.

    .attr("class", "pie-text-bg").attr("r", pieRadius * 0.33).attr("fill",
getComputedStyle(root).getPropertyValue('--white'))

    .attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-
width", 1.5);

  const pieText = pieGroup.selectAll("text.pie-
text").data([productivePercentage]).join("text") // Text element.

    .attr("class", "pie-text").attr("text-anchor", "middle").attr("dy", "0.35em")

    .style("font-size", `${Math.max(Math.min(pieRadius * 0.2, rowHeight * 0.06), 8)}px`)

    .style("font-weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--
accent'));

  // Animate the text value counting up/down.

  animateValue(pieText.node(), parseElementValue(pieText.node()),
productivePercentage, 800, val => `${val.toFixed(1)}%`);

```

```

    pieText.exit().remove();

  });

  // --- IDLE TIME CLOCKS ---

  wsMerge.each(function (ws) { // Iterate over each workstation group.

    const totalOpMinutes = opInputs.opHours * 60;

    const productiveMinutes = ws.cycleTime * results.throughputUnitsPerDay;

    const idleMinutes = Math.max(0, totalOpMinutes - productiveMinutes);

    const idleHours = idleMinutes / 60;

    const clockGroup = d3.select(this).select("g.clock");

    // Draw clock face and markings.

    const clockFaceMerged =
clockGroup.selectAll("circle.face").data([null]).join("circle").attr("class", "face").attr("r",
clockRadius) // Face.

      .attr("fill", getComputedStyle(root).getPropertyValue('--idle-color')).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent'))

      .attr("stroke-width", Math.max(clockRadius * 0.04, 1));

    const tickOuterRadius = clockRadius * 0.9,

    tickInnerRadius = clockRadius * 0.75,

    majorTickInnerRadius = clockRadius * 0.65;

    clockGroup.selectAll("line.tick").data(d3.range(0, 360, 30)).join("line").attr("class",
"tick") // Ticks.

      .attr("x1", 0).attr("y1", -tickOuterRadius).attr("x2", 0).attr("y2", (d, i) => i % 3 === 0 ? -
majorTickInnerRadius : -tickInnerRadius)

      .attr("stroke", getComputedStyle(root).getPropertyValue('--
accent').trim()).attr("stroke-width", (d, i) => i % 3 === 0 ? Math.max(clockRadius * 0.06,
1.5) : Math.max(clockRadius * 0.04, 1))

      .attr("transform", d => ` rotate(${d}) ` );

```

```

    clockGroup.selectAll("circle.center").data([null]).join("circle").attr("class",
"center").attr("r", Math.max(clockRadius * 0.06, 2)).attr("fill",
getComputedStyle(root).getPropertyValue('--accent')); // Center pin.

    // Calculate the angle for the clock hand based on idle hours.

    const angle = (idleHours / 12) * 2 * Math.PI; // Map 12 hours to 360 degrees.

    const handRadius = clockRadius * 0.8;

    const wsHand = clockGroup.selectAll("line.hand").data([angle]); // Bind angle data.

    // Animate the clock hand rotation.

    wsHand.enter().append("line").attr("class", "hand").attr("x1", 0).attr("y1", 0) // Create
hand if it doesn't exist.

        .attr("x2", 0).attr("y2", -handRadius).attr("stroke",
getComputedStyle(root).getPropertyValue('--secondary2'))

        .attr("stroke-width", Math.max(clockRadius * 0.08, 2)).attr("stroke-linecap",
"round").attr("transform", "rotate(0)")

        .merge(wsHand) // Merge new and existing hands.

        .transition().duration(750)

        .attrTween("transform", function (a) { // Animate from current angle to target angle.

            const currentTransform = d3.select(this).attr('transform') || "rotate(0)";

            const startAngleMatch = /rotate\(\([-.\d]+\)\)/.exec(currentTransform);

            const startAngle = startAngleMatch ? parseFloat(startAngleMatch[1]) : 0;

            const endAngle = (a * 180) / Math.PI;

            const i = d3.interpolate(startAngle, endAngle);

            return t => ` rotate(${i(t)}) ` ;

        });

    wsHand.exit().remove();

    // Add a blinking border if idle time is excessive.

    const clockFace = clockGroup.select("circle.face");

```

```

clockFace.interrupt("blink");

if (idleHours > 12) {

  function blink() {

    clockFace.transition("blink").duration(700).attr("stroke",
getComputedStyle(root).getPropertyValue('--failure-color').trim()).attr("stroke-width", 3.5)
    .transition("blink").duration(700).attr("stroke-width", 1.5).on("end", blink);

  }

  blink();

} else { // Revert to normal border if not excessive.

  clockFace.transition().duration(500).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("stroke-width",
Math.max(clockRadius * 0.04, 1));

}

// Digital display for idle time below the clock.

const idleText = clockGroup.selectAll("text.idle-text").data([idleHours]).join("text")

  .attr("class", "idle-text").attr("text-anchor", "middle").attr("y", clockRadius +
clockRadius * 0.4)

  .style("font-size", `${Math.max(Math.min(clockRadius * 0.4, rowHeight * 0.06),
10)}px`)

  .style("font-weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--
accent'))

  .text(d => `${d.toFixed(1)}h idle`);

idleText.exit().remove();

});

// --- TOP ROW SUMMARY PANEL ---

// Define dimensions and position for the summary area.

const summaryPadding = panelWidth * 0.001;

const summaryWidth = availableWidth - (2 * summaryPadding);

```

```

const summaryHeight = rowHeight - (2 * summaryPadding);

const summaryX = panelWidth / 2; // Center horizontally.

const summaryY = padding + rowHeight / 2; // Center in the first row.

// Create the summary group and its border.

const summary = effRoot.selectAll("g#eff-summary").data([null]).join(enter => {

  const summaryGroup = enter.append("g").attr("id", "eff-summary");

  summaryGroup.append("rect").attr("class", "summary-border").attr("fill", "none") //
Dashed border.

  .attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-
width", 2)

  .attr("stroke-dasharray", "5,5").attr("rx", 10);

  return summaryGroup;

}).attr("transform", ` translate(${summaryX}, ${summaryY}) `);

summary.select("rect.summary-border").attr("x", -summaryWidth / 2).attr("y", -
summaryHeight / 2).attr("width", summaryWidth).attr("height", summaryHeight); // Update
size on redraw.

// --- OVERALL EFFICIENCY PIE CHART (CENTER OF SUMMARY) ---

const arcLine = d3.arc().innerRadius(0).outerRadius(pieRadius);

const clampedEfficiency = Math.min(results.averageEfficiency, 99.99) / 100;

const workAngle = clampedEfficiency * 2 * Math.PI;

const shouldHideSummaryIdleSlice = results.averageEfficiency >= 99.5;

const linePieData = [

  { label: "Work", startAngle: 0, endAngle: workAngle, value:
Math.min(results.averageEfficiency, 99.99) },

  { label: "Idle", startAngle: workAngle, endAngle: 2 * Math.PI, value: Math.max(100 -
results.averageEfficiency, 0.01), hidden: shouldHideSummaryIdleSlice }

];

// Create a group for the pie chart elements, centered horizontally.

```

```

const pieGroup = summary.selectAll("g.pie-group").data([null]).join("g").attr("class",
"pie-group").attr("transform", "translate(0, 15)");

const sumSlices = pieGroup.selectAll("path.sum-slice").data(linePieData, d =>
d.label);

const sumSlicesEnter = sumSlices.enter().append("path").attr("class", "sum-slice")
.attr("fill", d => d.label === "Work" ? getComputedStyle(root).getPropertyValue('--
primary') : getComputedStyle(root).getPropertyValue('--secondary1'))
.attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-
width", 1.5)
.each(function (d) { this._current = { ...d, startAngle: 0, endAngle: 0 }; })
.attr("d", function (d) { return arcLine(this._current); });
const sumSlicesMerged = sumSlicesEnter.merge(sumSlices);
sumSlicesMerged.transition("shape").duration(750).attrTween("d", function (d) {
const i = d3.interpolate(this._current || d, d);
this._current = i(1);
return t => arcLine(i(t));
});
sumSlicesMerged.each(function (d) {
const element = d3.select(this);
const targetOpacity = d.hidden ? 0 : 1;
const duration = (targetOpacity === 0 && (element.style("opacity") || 1) > 0.5) ? 1500 :
200;
element.transition("opacity").duration(duration).style("opacity", targetOpacity);
});
sumSlices.exit().remove();

// Center text for the summary pie chart.

summary.selectAll("circle.summary-pie-text-bg").data([null]).join("circle").attr("class",
"summary-pie-text-bg")

```

```

    .attr("transform", "translate(0, 15)").attr("r", pieRadius * 0.33).attr("fill",
getComputedStyle(root).getPropertyValue('--white'))

    .attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-
width", 1.5);

    const summaryPieText = summary.selectAll("text.summary-pie-
text").data([results.averageEfficiency]).join("text")

    .attr("class", "summary-pie-text").attr("transform", "translate(0, 15)").attr("text-
anchor", "middle").attr("dy", "0.35em")

    .style("font-size", `${Math.max(Math.min(pieRadius * 0.2, rowHeight * 0.06),
10)}px`).style("font-weight", "bold")

    .attr("fill", getComputedStyle(root).getPropertyValue('--accent'));

    animateValue(summaryPieText.node(), parseElementValue(summaryPieText.node()),
results.averageEfficiency, 800, val => `${val.toFixed(1)}%`);

// --- SUMMARY CHARTS (Box Plot & Bar Chart) ---

// Layout for the three charts within the summary panel.

const colWidth = summaryWidth / 3;

const titleAreaHeight = 35;

const chartAreaHeight = summaryHeight - titleAreaHeight;

const chartAreaWidth = colWidth * 1.1;

const labelFontSize = Math.min(summaryHeight * 0.14, 14);

const chartContainerY = -summaryHeight / 2 + titleAreaHeight + chartAreaHeight / 2;

// Create a reusable tooltip.

const tooltip = d3.select("body").selectAll(".efficiency-tooltip").data([null]).join("div")

    .attr("class", "efficiency-tooltip").style("position", "absolute").style("pointer-events",
"none")

    .style("background",
getComputedStyle(document.documentElement).getPropertyValue('--tooltip-
bg').trim()).style("color", "white")

```

```

        .style("padding", "8px 12px").style("border-radius", "6px").style("font-size",
"12px").style("opacity", 0).style("transition", "opacity 0.2s");

    // Define a gradient fill for the charts.

    const defs = effRoot.selectAll("defs").data([null]).join("defs");

    const boxGradient = defs.selectAll("#box-
gradient").data([null]).join("linearGradient").attr("id", "box-gradient").attr("x1",
"0%").attr("y1", "0%").attr("x2", "0%").attr("y2", "100%");

    boxGradient.selectAll("stop").data([
{ offset: "0%", color:
getComputedStyle(root).getPropertyValue('--secondary2').trim() }, { offset: "100%", color:
"#4d337aff" }]).join("stop").attr("offset", d => d.offset).attr("stop-color", d => d.color);

    // --- BOX PLOT (Balance Loss per Cycle) ---

    const boxPlotGroup = summary.selectAll("g.box-plot-
group").data([null]).join("g").attr("class", "box-plot-group").attr("transform", `
translate(${colWidth * 0.8}, ${chartContainerY})`);

    const bottleneckCycleTime = d3.max(results.workstations, d => d.cycleTime) || 0;

    const idleTimesPerCycle = results.workstations.map(ws => bottleneckCycleTime -
ws.cycleTime);

    const q1 = d3.quantile(idleTimesPerCycle, 0.25) || 0,

    median = d3.quantile(idleTimesPerCycle, 0.5) || 0,

    q3 = d3.quantile(idleTimesPerCycle, 0.75) || 0;

    const min = d3.min(idleTimesPerCycle) || 0,

    max = d3.max(idleTimesPerCycle) || 0;

    const xBox = d3.scaleLinear().domain([0, max * 1.1 || 1]).range([-chartAreaWidth / 2,
chartAreaWidth / 2]);

    const boxHeight = chartAreaHeight * 0.4;

    // Draw box plot elements with transitions.

    boxPlotGroup.selectAll("line.center-line").data([null]).join("line").attr("class", "center-
line").attr("y1", 0).attr("y2", 0).attr("stroke", getComputedStyle(root).getPropertyValue('--
accent')).attr("stroke-width", 3).transition().duration(750).attr("x1", xBox(min)).attr("x2",
xBox(max));

```



```

    boxPlotGroup.selectAll("line.whisker").data([
      { val: min, key: 'min' }, { val: max, key: 'max' }
    ], d => d.key).join("line").attr("class", "whisker").attr("y1", -boxHeight / 2).attr("y2", boxHeight / 2).attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 3).attr("stroke-linecap", "round").transition().duration(750).attr("x1", d => xBox(d.val)).attr("x2", d => xBox(d.val));

```

```

    boxPlotGroup.selectAll("rect.box").data([null]).join("rect").attr("class", "box").attr("y", -boxHeight / 2).attr("height", boxHeight).attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 4).style("fill", "url(#box-gradient)").transition().duration(750).attr("x", xBox(q1)).attr("width", xBox(q3) - xBox(q1));

```

```

    boxPlotGroup.selectAll("line.median-line").data([median]).join("line").attr("class", "median-line").attr("y1", -boxHeight / 2).attr("y2", boxHeight / 2).attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 5).attr("stroke-linecap", "round").transition().duration(750).attr("x1", d => xBox(d)).attr("x2", d => xBox(d));

```

```

    const tooltipContent = `<div style="font-weight:bold; margin-bottom: 5px; text-align:center; border-bottom: 1px solid ${getComputedStyle(root).getPropertyValue('--white')}"; padding-bottom: 4px;">Idle Time per Cycle</div><strong>Q1:</strong> ${q1.toFixed(2)} min<br><strong>Median:</strong> ${median.toFixed(2)} min<br><strong>Q3:</strong> ${q3.toFixed(2)} min<br><strong>Max:</strong> ${max.toFixed(2)} min`;

```

```

    boxPlotGroup.selectAll("rect.tooltip-receiver").data([null]).join("rect").attr("class", "tooltip-receiver").attr("x", -chartAreaWidth / 2).attr("y", -chartAreaHeight / 2).attr("width", chartAreaWidth).attr("height", chartAreaHeight).style("fill", "transparent")

```

```

    .on("mouseover", () => tooltip.style("opacity", 1))

```

```

    .on("mousemove", (event) => tooltip.html(tooltipContent).style("left", (event.pageX + 15) + "px").style("top", (event.pageY - 28) + "px"))

```

```

    .on("mouseout", () => tooltip.style("opacity", 0));

```

```

    // --- BAR CHART (Workstation Idle Time) ---

```

```

    const barChartMargin = { top: 10, right: 5, bottom: 35, left: 40 };

```

```

    const barChartInnerWidth = chartAreaWidth - barChartMargin.left - barChartMargin.right;

```

```

    const barChartInnerHeight = chartAreaHeight - barChartMargin.top - barChartMargin.bottom;

```

```

const barChartGroup = summary.selectAll("g.bar-chart-
group").data([null]).join("g").attr("class", "bar-chart-group")

.attr("transform", `translate(${colWidth * 0.8 - chartAreaWidth / 2 +
barChartMargin.left}, ${chartContainerY - chartAreaHeight / 2 + barChartMargin.top})`);

const xBar = d3.scaleBand().domain(results.workstations.map(d => d.id)).range([0,
barChartInnerWidth]).padding(0.2);

const yBar = d3.scaleLinear().domain([0, d3.max(results.workstations, d =>
d.dailyIdleTime) * 1.1 || 1]).range([barChartInnerHeight, 0]);

// X and Y axes for bar chart.

barChartGroup.selectAll(".x-axis").data([null]).join("g").attr("class", "x-
axis").attr("transform", `translate(0,
${barChartInnerHeight})`).call(d3.axisBottom(xBar).tickSizeOuter(0))

.selectAll("text").style("font-size", "12px").style("font-weight", "600").attr("transform",
"rotate(-45)").attr("text-anchor", "end").attr("dx", "-0.8em").attr("dy", "0.15em");

barChartGroup.selectAll(".y-axis").data([null]).join("g").attr("class", "y-
axis").call(d3.axisLeft(yBar).ticks(4).tickFormat(d => `${(d /
60).toFixed(1)}h`)).tickSizeOuter(0))

.selectAll("text").style("font-size", "12px").style("font-weight", "600");

// Draw bars with an animation.

const allBars = barChartGroup.selectAll("rect.bar").data(results.workstations, d =>
d.id)

.join(
  enter => enter.append("rect").attr("class", "bar")

  .attr("x", d => xBar(d.id)).attr("width", xBar.bandwidth())

  .attr("y", yBar(0)).attr("height", 0) // Start from height 0 for animation.

  .style("fill", "url(#box-gradient)").attr("stroke",
getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 1.8)

  .on("mouseover", function (event, d) { tooltip.style("opacity", 1);
d3.select(this).style("opacity", 0.8); })

```

```

.on("mousemove", function (event, d) {

    const tooltipContent = `<div style="font-weight:bold; margin-bottom: 5px; text-align:center; border-bottom: 1px solid ${getComputedStyle(root).getPropertyValue('--white')}"; padding-bottom: 4px;">Workstation ${d.id}</div><strong>Daily Idle Time:</strong> ${((d.dailyIdleTime / 60).toFixed(2))} hours<br><strong>Idle Time:</strong> ${d.dailyIdleTime.toFixed(1)} minutes<br><strong>Efficiency:</strong> ${d.efficiency.toFixed(1)}%`;

    tooltip.html(tooltipContent).style("left", (event.pageX + 15) + "px").style("top", (event.pageY - 28) + "px");

    })

    .on("mouseout", function () { tooltip.style("opacity", 0);
d3.select(this).style("opacity", 1); })

);

allBars.transition().duration(500).ease(d3.easeQuadInOut) // Animate bars growing to their final height.

.attr("y", d => yBar(d.dailyIdleTime)).attr("height", d => barChartInnerHeight - yBar(d.dailyIdleTime));

// --- SUMMARY LABELS ---

// Center Group: Overall Efficiency Title and Total Idle Time value.

const centerLabelGroup = summary.selectAll("g.center-label-group").data([results]).join("g").attr("class", "center-label-group").attr("transform", `translate(0, ${-summaryHeight / 2 + 18})`);

centerLabelGroup.selectAll("text.summary-pie-title").data(["Overall Efficiency"]).join("text").attr("class", "summary-pie-title").attr("text-anchor", "middle").style("font-size", `${labelFontSize}px`).style("font-weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--accent')).text(d => d);

const totalIdleTextGroup = centerLabelGroup.selectAll("g.total-idle-text-group").data([null]).join("g").attr("class", "total-idle-text-group").attr("transform", "translate(-18, 20)").attr("text-anchor", "middle");

totalIdleTextGroup.selectAll("text.total-idle-label").data(["Total Idle Time:"]).join("text").attr("class", "total-idle-label").style("font-size",

```

```

` ${labelFontSize}px` ).style("font-weight", "bold").attr("fill",
getComputedStyle(root).getPropertyValue('--accent')).text(d => d);

    const idleTimeValue = totalIdleTextGroup.selectAll("text.total-idle-
time").data([results]).join("text").attr("class", "total-idle-time").attr("text-anchor",
"start").attr("x", 55).style("font-size", ` ${labelFontSize}px` ).style("font-weight",
"bold").attr("fill", getComputedStyle(root).getPropertyValue('--failure-color'));

    animateValue(idleTimeValue.node(), parseElementValue(idleTimeValue.node()),
results.totalIdleTime / 60, 800, val => ` ${val.toFixed(1)}h` );

    // Left Group: Box Plot Title and Idle Time CV value.

    const boxLabelGroup = summary.selectAll("g.box-label-
group").data([results]).join("g").attr("class", "box-label-group").attr("transform",
` translate(${colWidth * 0.9}, ${summaryHeight / 2 + 18})` );

    boxLabelGroup.selectAll("text.box-plot-title").data(["Balance Loss per
Cycle"]).join("text").attr("class", "box-plot-title").attr("text-anchor", "middle").style("font-
size", ` ${labelFontSize}px` ).style("font-weight", "bold").text(d => d);

    const idleTimeCVTextGroup = boxLabelGroup.selectAll("g.idle-time-cv-text-
group").data([null]).join("g").attr("class", "idle-time-cv-text-group").attr("transform",
"translate(-23, 20)").attr("text-anchor", "middle");

    idleTimeCVTextGroup.selectAll("text.box-idle-time-cv-label").data(["Idle Time CV:
"]).join("text").attr("class", "box-idle-time-cv-label").style("font-size",
` ${labelFontSize}px` ).style("font-weight", "bold").attr("fill",
getComputedStyle(root).getPropertyValue('--accent')).text(d => d);

    const idleTimeCvValue = idleTimeCVTextGroup.selectAll("text.box-idle-cv-
value").data([results]).join("text").attr("class", "box-idle-cv-value").attr("text-anchor",
"start").attr("x", 47).style("font-size", ` ${labelFontSize}px` ).style("font-weight",
"bold").attr("fill", getComputedStyle(root).getPropertyValue('--failure-color'));

    animateValue(idleTimeCvValue.node(), parseElementValue(idleTimeCvValue.node()),
results.idleTimeCv, 800, val => ` ${val.toFixed(1)}%` );

    // Right Group: Bar Chart Title and Balance Loss value.

    const barLabelGroup = summary.selectAll("g.bar-label-
group").data([results]).join("g").attr("class", "bar-label-group").attr("transform",
` translate(${colWidth * 0.9}, ${summaryHeight / 2 + 18})` );

```

```

    barLabelGroup.selectAll("text.bar-chart-title").data(["Total Balance Loss per
Workstation"]).join("text").attr("class", "bar-chart-title").attr("text-anchor",
"middle").style("font-size", `${labelFontSize}px`).style("font-weight", "bold").attr("fill",
getComputedStyle(root).getPropertyValue('--accent')).text(d => d);

```

```

    const balanceLossTextGroup = barLabelGroup.selectAll("g.balance-loss-text-
group").data([null]).join("g").attr("class", "balance-loss-text-group").attr("transform",
"translate(-20, 20)").attr("text-anchor", "middle");

```

```

    balanceLossTextGroup.selectAll("text.bar-balance-delay-label").data(["Workstation
Balance Loss: "]).join("text").attr("class", "bar-balance-delay-label").style("font-size",
`${labelFontSize}px`).style("font-weight", "bold").attr("fill",
getComputedStyle(root).getPropertyValue('--accent')).text(d => d);

```

```

    const balanceDelayValue = balanceLossTextGroup.selectAll("text.bar-balance-delay-
value").data([results]).join("text").attr("class", "bar-balance-delay-value").attr("text-
anchor", "start").attr("x", 90).style("font-size", `${labelFontSize}px`).style("font-weight",
"bold").attr("fill", getComputedStyle(root).getPropertyValue('--failure-color'));

```

```

    animateValue(balanceDelayValue.node(),
parseElementValue(balanceDelayValue.node()), results.balanceDelay, 800, val =>
`${val.toFixed(1)}%`);

```

```

}

```

```

// Expose the public draw method.

```

```

return {

```

```

    draw: draw

```

```

};

```

```

})();

```

```

const drawInvestmentPanel = (function () {

```

```

    /**

```

```

    * @property {object} investmentState - Holds the persistent state for all

```

```

    * user-configurable inputs on the investment panel.

```

```

*/

const investmentState = {
  analysisPeriod: 5,
  marr: 12.0,
  taxRate: 25.0,
  workingDays: 250,
  mfgOverhead: 250000,
  sgaExpenses: 350000,
  freightExpense: 300000,
  costPerFootStraight: 225,
  costPerBend: 450,
  installationCost: 10000,
  salvageValue: 10000,
  runExpansionCase: false,
  // Probabilistic demand parameters
  std: 6750,
  cv: 15.0,
  ciLevel: 95,
  p90Demand: 0,
  p50Demand: 0,
  p10Demand: 0
};

/**
 * @const {object} MACRS_RATES - Standard depreciation rates for 5-year MACRS.
 */

```

```

const MACRS_RATES = {
  '5-year': [0.2000, 0.3200, 0.1920, 0.1152, 0.1152, 0.0576]
};

const Z_SCORE_P90 = 1.28155;

const CI_Z_SCORES = { 90: 1.645, 95: 1.960, 99: 2.576 };

let analysisDebounceTimer;

function formatNumberWithCommas(num) { return (num === null || num ===
undefined) ? '' : num.toString().replace(/\B(?=(\d{3})+(?!\d))/g, ","); }

function parseFormattedNumber(str) { return (typeof str !== 'string') ? str :
(parseFloat(str.replace(/,/g, "")) || 0); }

function updateDemandUI() {
  document.getElementById('inv-std').value =
formatNumberWithCommas(Math.round(investmentState.std));

  document.getElementById('inv-cv').value = investmentState.cv.toFixed(1);

  document.getElementById('inv-p90Demand').value =
formatNumberWithCommas(Math.round(investmentState.p90Demand));

  document.getElementById('inv-p50Demand').textContent =
formatNumberWithCommas(Math.round(investmentState.p50Demand));

  document.getElementById('inv-p10Demand').value =
formatNumberWithCommas(Math.round(investmentState.p10Demand));
}

function updateProbabilisticValues(driver) {
  const meanDemand = (parseFloat(dailyDemandInput.value) || 180) *
investmentState.workingDays;

```

```

investmentState.p50Demand = meanDemand;

let std;

if (driver === 'p90') {
    if (investmentState.p90Demand < meanDemand) investmentState.p90Demand =
meanDemand;

    std = (investmentState.p90Demand - meanDemand) / Z_SCORE_P90;

    investmentState.std = std > 0 ? std : 0;

    investmentState.cv = meanDemand > 0 ? (investmentState.std / meanDemand) *
100 : 0;
} else if (driver === 'p10') {
    if (investmentState.p10Demand > meanDemand) investmentState.p10Demand =
meanDemand;

    std = (meanDemand - investmentState.p10Demand) / Z_SCORE_P90;

    investmentState.std = std > 0 ? std : 0;

    investmentState.cv = meanDemand > 0 ? (investmentState.std / meanDemand) *
100 : 0;
} else {
    if (driver === 'std') {
        std = investmentState.std;

        investmentState.cv = meanDemand > 0 ? (std / meanDemand) * 100 : 0;
    } else {
        std = (investmentState.cv / 100) * meanDemand;

        investmentState.std = std;
    }
}

const z = CI_Z_SCORES[investmentState.ciLevel] || 1.960;

const halfWidth = z * std;

```



```
investmentState.p90Demand = meanDemand + halfWidth;
investmentState.p10Demand = meanDemand - halfWidth;
}
```

```
updateDemandUI();
clearTimeout(analysisDebounceTimer);
analysisDebounceTimer = setTimeout(runFullAnalysis, 0);
}
```

```
function calculateNPV(cashFlows, rate) { return cashFlows.reduce((acc, val, i) => acc +
val / Math.pow(1 + rate, i), 0); }
```

```
function calculateIRR(cashFlows, maxIter = 100, tolerance = 1e-6) {
  if (!cashFlows || cashFlows.length === 0 || cashFlows[0] >= 0) { return NaN; }

  let lowRate = -0.99, highRate = 9999999.0, midRate = 0, npvLow =
  calculateNPV(cashFlows, lowRate), npvHigh = calculateNPV(cashFlows, highRate);

  if (npvLow * npvHigh > 0) return NaN;

  for (let i = 0; i < maxIter; i++) {
    midRate = (lowRate + highRate) / 2;

    const npvMid = calculateNPV(cashFlows, midRate);

    if (Math.abs(npvMid) < tolerance) return midRate;

    if (npvLow * npvMid < 0) { highRate = midRate; } else { lowRate = midRate; }
  }

  return midRate;
}
```

```
function calculatePaybackPeriod(cashFlows) {
```

```

if (!cashFlows || cashFlows.length < 2 || cashFlows[0] >= 0) return 0;

const initialInvestment = Math.abs(cashFlows[0]);

let cumulativeCashFlow = 0;

for (let t = 1; t < cashFlows.length; t++) {

    const lastCumulative = cumulativeCashFlow;

    cumulativeCashFlow += cashFlows[t];

    if (cumulativeCashFlow >= initialInvestment) {

        return (cashFlows[t] <= 0) ? t : (t - 1) + ((initialInvestment - lastCumulative) /
cashFlows[t]);

    }

}

return Infinity;

}

function calculateFinancialScenario(annualUnitDemand) {

    const { analysisPeriod, marr, taxRate, workingDays, runExpansionCase, salvageValue,
installationCost } = investmentState;

    const finInputs = {

        laborCost: parseFloat(laborCostInput.value), superSell:
parseFloat(superSellInput.value), superCogs: parseFloat(superCogsInput.value),

        ultraSell: parseFloat(ultraSellInput.value), ultraCogs:
parseFloat(ultraCogsInput.value), megaSell: parseFloat(megaSellInput.value), megaCogs:
parseFloat(megaCogsInput.value),

    };

    const avgPrice = (finInputs.superSell * BUILD_RATIOS.super) + (finInputs.ultraSell *
BUILD_RATIOS.ultra) + (finInputs.megaSell * BUILD_RATIOS.mega);

    let unitsToProduce = 0, configForReport = {}, initialInvestment = 0,
equipmentCostForDepreciation = 0;

```

```

const currentEmployees = parseInt(numEmployeesInput.value);

const baseOpHours = parseFloat(opHoursInput.value);

if (!runExpansionCase) {

    const metrics = calculateMetrics({ dailyDemand: 9999, opHours: baseOpHours,
numEmployees: currentEmployees }, {});

    const maxAnnualCapacity = metrics.throughputUnitsPerDay * workingDays;

    unitsToProduce = Math.min(annualUnitDemand, maxAnnualCapacity);

    configForReport = { name: `${currentEmployees} Workers, ${baseOpHours} hrs/day`,
empCount: currentEmployees, opHours: baseOpHours };

    equipmentCostForDepreciation = (investmentState.costPerFootStraight *
ASSEMBLY_LINE_LENGTH) + (investmentState.costPerBend * ((4 * currentEmployees) -
(currentEmployees % 2 === 0 ? 2 : 0))) + installationCost;

    initialInvestment = -equipmentCostForDepreciation;

} else {

    const optimalConfigResult = findOptimalNPVConfig(annualUnitDemand, finInputs);

    const optimalConfig = { name: `${optimalConfigResult.emp} Workers,
${optimalConfigResult.hrs.toFixed(2)} hrs/day`, empCount: optimalConfigResult.emp,
opHours: optimalConfigResult.hrs };

    unitsToProduce = annualUnitDemand;

    configForReport = optimalConfig;

    const oldLineCost = (investmentState.costPerFootStraight *
ASSEMBLY_LINE_LENGTH) + (investmentState.costPerBend * ((4 * currentEmployees) -
(currentEmployees % 2 === 0 ? 2 : 0)));

    const newLineCost = (investmentState.costPerFootStraight *
ASSEMBLY_LINE_LENGTH) + (investmentState.costPerBend * ((4 *
optimalConfig.empCount) - (optimalConfig.empCount % 2 === 0 ? 2 : 0)));

    const adjustment = newLineCost < oldLineCost ? -(salvageValue * ((oldLineCost -
newLineCost) / oldLineCost)) : (newLineCost - oldLineCost);

```

```

    equipmentCostForDepreciation = newLineCost < oldLineCost ? 0 : adjustment +
installationCost;

    initialInvestment = -(installationCost + adjustment);

}

const cashFlows = [initialInvestment];

const scaledMfgOverhead = investmentState.mfgOverhead *
(configForReport.opHours > baseOpHours ? configForReport.opHours / baseOpHours : 1);

const scaledSgaExpenses = investmentState.sgaExpenses * (configForReport.opHours
> baseOpHours ? configForReport.opHours / baseOpHours : 1);

const macrsSchedule = MACRS_RATES['5-year'];

for (let t = 1; t <= analysisPeriod; t++) {

    const revenue = unitsToProduce * avgPrice;

    const totalMaterialCost = unitsToProduce * ((finInputs.superCogs *
BUILD_RATIOS.super) + (finInputs.ultraCogs * BUILD_RATIOS.ultra) + (finInputs.megaCogs
* BUILD_RATIOS.mega));

    const laborCost = configForReport.empCount * configForReport.opHours *
finInputs.laborCost * workingDays;

    const taxDepreciation = (t - 1 < macrsSchedule.length &&
equipmentCostForDepreciation > 0) ? equipmentCostForDepreciation * macrsSchedule[t -
1] : 0;

    const ebit = revenue - (totalMaterialCost + laborCost + scaledMfgOverhead +
investmentState.freightExpense + scaledSgaExpenses + taxDepreciation);

    const nopat = ebit - (ebit > 0 ? ebit * (taxRate / 100) : 0);

    cashFlows.push(nopat + taxDepreciation);

}

```

```

    if (equipmentCostForDepreciation > 0 && analysisPeriod > 0)
    { cashFlows[analysisPeriod] += salvageValue * (1 - (taxRate / 100)); }

    const npv = calculateNPV(cashFlows, marr / 100), irr = calculateIRR(cashFlows),
    payback = calculatePaybackPeriod(cashFlows);

    return { annualUnitDemand, requiredConfig: configForReport, metrics: { npv, irr,
    payback, initialInvestment }, cashFlows };

}

function runFullAnalysis() {

    const resultsDisplay = d3.select("#inv-results-display").style("display", "block");

    const resultsColumn = d3.select(".inv-results-column");

    resultsColumn.transition().duration(150).style("opacity", 0.5);

    setTimeout(() => {

        try {

            const results = Object.fromEntries(Object.entries({ 'P90 (Optimistic)':
investmentState.p90Demand, 'P50 (Most Likely)': investmentState.p50Demand, 'P10
(Conservative)': investmentState.p10Demand })).map(([name, demand]) => [name,
calculateFinancialScenario(demand)]);

            d3.select("#inv-results-placeholder").style("display", "none");

            renderInvestmentResults(results);

            resultsColumn.transition().duration(250).style("opacity", 1);

        } catch (error) {

            console.error("Error during investment analysis:", error);

            d3.select("#inv-results-placeholder").html(`<p class="error">An error occurred:
${error.message}</p>`).style("display", "block");

            resultsColumn.style("opacity", 1);

        }

    });
}

```

```
    }, 50);  
  }
```

```
function findOptimalNPVConfig(annualUnitDemand, finInputs) {  
  let maxNPV = -Infinity;  
  
  let bestConfig = { emp: 0, hrs: 0 };  
  
  const dailyDemand = Math.ceil(annualUnitDemand / investmentState.workingDays);  
  
  const currentEmployees = parseInt(numEmployeesInput.value);  
  
  const maxDemandMap = new Map(WORKSTATION_CAPACITIES.map(c => [c.ws,  
c.maxDemand]));  
  
  for (let numEmployees = 3; numEmployees <= 13; numEmployees++) {  
    if (dailyDemand > (maxDemandMap.get(numEmployees) || 0)) continue;  
  
    const tempConfig = { ...state.configData };  
  
    state.configData = originalConfigData;  
  
    const { bottleneckTime, fastestTime } =  
calculateWorkstationDetails(numEmployees);  
  
    state.configData = tempConfig;  
  
    if (bottleneckTime <= 0 || !isFinite(fastestTime) || fastestTime <= 0) continue;  
  
    const productSpacing = fastestTime * 15;  
  
    const throughputTime = (ASSEMBLY_LINE_LENGTH / productSpacing) *  
bottleneckTime;  
  
    const totalRequiredMinutes = (dailyDemand > 1 ? (dailyDemand - 1) *  
bottleneckTime : 0) + throughputTime;  
  
    const minRequiredHours = totalRequiredMinutes / 60;
```

```

    if (minRequiredHours > 24) continue;

    let optimalOpHours = -1;

    for (let opHours = roundUpToQuarter(minRequiredHours); opHours <= 24; opHours
    += 0.25) {

        const metrics = calculateMetrics({ dailyDemand, opHours, numEmployees },
    finInputs);

        if (metrics && metrics.throughputUnitsPerDay >= dailyDemand) {

            optimalOpHours = opHours;

            break;

        }

    }

    if (optimalOpHours === -1) continue;

    const configForAnalysis = { empCount: numEmployees, opHours: optimalOpHours };

    const { analysisPeriod, marr, taxRate, workingDays, salvageValue, installationCost } =
    investmentState;

    const oldLineCost = (investmentState.costPerFootStraight *
    ASSEMBLY_LINE_LENGTH) + (investmentState.costPerBend * ((4 * currentEmployees) -
    (currentEmployees % 2 === 0 ? 2 : 0)));

    const newLineCost = (investmentState.costPerFootStraight *
    ASSEMBLY_LINE_LENGTH) + (investmentState.costPerBend * ((4 *
    configForAnalysis.empCount) - (configForAnalysis.empCount % 2 === 0 ? 2 : 0)));

    const adjustment = newLineCost < oldLineCost ? -(salvageValue * ((oldLineCost -
    newLineCost) / oldLineCost)) : (newLineCost - oldLineCost);

    const equipmentCostForDepreciation = newLineCost < oldLineCost ? 0 : adjustment
    + installationCost;

    const initialInvestment = -(installationCost + adjustment);

    const cashFlows = [initialInvestment];

```

```

    const avgPrice = (finInputs.superSell * BUILD_RATIOS.super) + (finInputs.ultraSell *
BUILD_RATIOS.ultra) + (finInputs.megaSell * BUILD_RATIOS.mega);

    const scaledMfgOverhead = investmentState.mfgOverhead *
(configForAnalysis.opHours > 15 ? configForAnalysis.opHours / 15 : 1);

    const scaledSgaExpenses = investmentState.sgaExpenses *
(configForAnalysis.opHours > 15 ? configForAnalysis.opHours / 15 : 1);

    const macrsSchedule = MACRS_RATES['5-year'];

    for (let t = 1; t <= analysisPeriod; t++) {

        const revenue = annualUnitDemand * avgPrice;

        const totalMaterialCost = annualUnitDemand * ((finInputs.superCogs *
BUILD_RATIOS.super) + (finInputs.ultraCogs * BUILD_RATIOS.ultra) + (finInputs.megaCogs
* BUILD_RATIOS.mega));

        const laborCost = configForAnalysis.empCount * configForAnalysis.opHours *
finInputs.laborCost * workingDays;

        const taxDepreciation = (t - 1 < macrsSchedule.length &&
equipmentCostForDepreciation > 0) ? equipmentCostForDepreciation * macrsSchedule[t -
1] : 0;

        const ebit = revenue - (totalMaterialCost + laborCost + scaledMfgOverhead +
investmentState.freightExpense + scaledSgaExpenses + taxDepreciation);

        const nopat = ebit - (ebit > 0 ? ebit * (taxRate / 100) : 0);

        cashFlows.push(nopat + taxDepreciation);

    }

    if (equipmentCostForDepreciation > 0 && analysisPeriod > 0) {

        cashFlows[analysisPeriod] += salvageValue * (1 - (taxRate / 100));

    }

    const currentNPV = calculateNPV(cashFlows, marr / 100);

```



```

    if (currentNPV > maxNPV) {
      maxNPV = currentNPV;
      bestConfig = { emp: numEmployees, hrs: optimalOpHours };
    }
  }
  return bestConfig;
}

```

```

function renderInvestmentResults(results) {
  const p50Result = results['P50 (Most Likely)'];
  const scorecardData = [
    { label: 'Net Present Value (NPV)', value: p50Result.metrics.npv.toLocaleString('en-US', { style: 'currency', currency: 'USD', maximumFractionDigits: 0 }), isError: p50Result.metrics.npv < 0 },
    { label: 'Internal Rate of Return (IRR)', value: isNaN(p50Result.metrics.irr) ? "No Return" : ` ${ (p50Result.metrics.irr * 100).toFixed(1)}% `, isError: isNaN(p50Result.metrics.irr) },
    { label: 'Payback Period', value: isFinite(p50Result.metrics.payback) ? ` ${Math.ceil(p50Result.metrics.payback * 365.2425)} Days ` : "Net Loss", isError: !isFinite(p50Result.metrics.payback) }
  ];

```

```

  const scorecards = d3.select(".inv-scorecard-container").html("").selectAll(".inv-scorecard").data(scorecardData).join("div").attr("class", "inv-scorecard");

  scorecards.append("div").attr("class", "inv-scorecard-label").text(d => d.label);

  scorecards.append("div").attr("class", "inv-scorecard-value").style("color", d => d.isError ? 'var(--failure-color)' : null).text(d => d.value);

```

```

const chartContainer = d3.select(".inv-chart-container");

chartContainer.html("");

const chartNode = chartContainer.node();

if (!chartNode) return;

const scorecardHeight = 95;

const chartContainerHeight = d3.select('.inv-results-column').node().clientHeight -
scorecardHeight - 15;

chartContainer.style('height', `${chartContainerHeight > 0 ? chartContainerHeight :
0}px`);

const margin = { top: 20, right: 30, bottom: 60, left: 80 };

const width = chartNode.getBoundingClientRect().width - margin.left - margin.right;

const height = chartNode.getBoundingClientRect().height - margin.top -
margin.bottom;

if (width <= 0 || height <= 0) return;

const chartSvg = chartContainer.append("svg").attr("viewBox", `0 0 ${width +
margin.left + margin.right} ${height + margin.top + margin.bottom}`);

const chartG = chartSvg.append("g").attr("transform",
`translate(${margin.left},${margin.top})`);

const cumulativeData = Object.entries(results).map(([name, data]) => ({ name, values:
data.cashFlows.map((cf, i) => ({ year: i, value: data.cashFlows.slice(0, i + 1).reduce((a, b) =>
a + b, 0) }))) ));

const x = d3.scaleLinear().domain([0, investmentState.analysisPeriod]).range([0,
width]);

const y = d3.scaleLinear().domain([d3.min(cumulativeData, d => d3.min(d.values, v =>
v.value)), d3.max(cumulativeData, d => d3.max(d.values, v =>
v.value))]).nice().range([height, 0]);

chartG.append("g").attr("class", "inv-axis").attr("transform",
`translate(0,${height})`).call(d3.axisBottom(x).ticks(investmentState.analysisPeriod).tickF
ormat(d3.format("d"))).selectAll("text").style("font-size", '14px');

```

```

    chartG.append("g").attr("class", "inv-
axis").call(d3.axisLeft(y).tickFormat(d3.format("$,.2s"))).selectAll("text").style("font-size",
'14px');

    const p90Data = cumulativeData.find(d => d.name.includes('P90')).values, p50Data =
cumulativeData.find(d => d.name.includes('P50')).values, p10Data =
cumulativeData.find(d => d.name.includes('P10')).values;

    chartG.append("path").datum(p90Data).attr("fill",
getComputedStyle(root).getPropertyValue('--primary')).attr("class", "inv-area").attr("d",
d3.area().x(d => x(d.year)).y0(d => y(p50Data[d.year].value)).y1(d => y(d.value)));

    chartG.append("path").datum(p50Data).attr("fill",
getComputedStyle(root).getPropertyValue('--secondary2')).attr("class", "inv-area").attr("d",
d3.area().x(d => x(d.year)).y0(d => y(p10Data[d.year].value)).y1(d => y(d.value)));

    const line = d3.line().x(d => x(d.year)).y(d => y(d.value));

    chartG.selectAll(".inv-line").data(cumulativeData).join("path").attr("class", "inv-
line").attr("d", d => line(d.values)).style("stroke", d => d3.scaleOrdinal().domain(['P90
(Optimistic)', 'P50 (Most Likely)', 'P10
(Conservative)']).range([getComputedStyle(root).getPropertyValue('--primary'),
getComputedStyle(root).getPropertyValue('--secondary1'),
getComputedStyle(root).getPropertyValue('--secondary2')])(d.name)).style("stroke-width",
d => d.name.includes('P50') ? '6px' : '2px');

    chartSvg.append("text").attr("class", "inv-axis-label").attr("text-anchor",
"middle").attr("x", margin.left + width / 2).attr("y", height + margin.top + 40).text("Analysis
Period (Years)").style("font-size", "16px").style("font-family", "Arial");

    chartSvg.append("text").attr("class", "inv-axis-label").attr("transform", "rotate(-
90)").attr("text-anchor", "middle").attr("y", margin.left / 4).attr("x", -(margin.top + height /
2)).text("Cumulative Free Cash Flow").style("font-size", "16px").style("font-family", "Arial");

    chartG.append("line").attr("class", "inv-break-even").attr("x1", 0).attr("x2",
width).attr("y1", y(0)).attr("y2", y(0));

    const tooltip = createTooltip("inv-tooltip");

    chartG.selectAll(".inv-hitbox").data(cumulativeData).join("path").attr("class", "inv-
hitbox").attr("d", d => line(d.values)).on("mouseover", (event, d) => {

        tooltip.transition().duration(200).style("opacity", 1);

```

```

    const scenarioResult = results[d.name];

    const FmtdIRR = isNaN(scenarioResult.metrics.irr) ? "No Return" :
` ${scenarioResult.metrics.irr * 100}.toFixed(1)}% `;

    const FmtdPayback = isFinite(scenarioResult.metrics.payback) ?
` ${Math.ceil(scenarioResult.metrics.payback * 365.2425)} Days ` : "Net Loss";

    tooltip.html(`<div class="tooltip-header">${d.name}</div><div class="tooltip-
row"><span>NPV:</span> <strong>${scenarioResult.metrics.npv.toLocaleString('en-US',
{ style: 'currency', currency: 'USD', maximumFractionDigits: 0 })}</strong></div><div
class="tooltip-row"><span>IRR:</span> <strong>${FmtdIRR}</strong></div><div
class="tooltip-row"><span>Payback:</span>
<strong>${FmtdPayback}</strong></div><hr><div class="tooltip-
row"><span>Config:</span>
<strong>${scenarioResult.requiredConfig.name}</strong></div><div class="tooltip-
row"><span>Annual Demand:</span>
<strong>${scenarioResult.annualUnitDemand.toFixed(0).toLocaleString('en-US')}
Units</strong></div>`);

    }).on("mousemove", (event) => tooltip.style("left", (event.pageX + 15) + "px").style("top",
(event.pageY - 28) + "px")).on("mouseout", () =>
tooltip.transition().duration(500).style("opacity", 0));

}

```

```

return async function draw() {

    const svg = d3.select("#investment-panel");

    svg.selectAll("*").remove();

    const container = svg.append("foreignObject").attr("width", "100%").attr("height",
"100%").append("xhtml:div").attr("class", "inv-container");

    const inputColumn = container.append("div").attr("class", "inv-input-column");

    inputColumn.append("h3").attr("class", "inv-column-title").text("Economic
Parameters");

    const inputArea = inputColumn.append("div").attr("class", "inv-inputs");

    try {

```

```
const response = await fetch('Pages/investmentInputs.html');
if (!response.ok) throw new Error(response.statusText);
inputArea.html(await response.text());
setTimeout(() => {
  const tooltips = {
    'inv-analysisPeriod': 'The Number of Years over which the Investment\'s Cash  
Flows are projected.',
    'inv-marr': 'The Minimum Acceptable Rate of Return (MARR) for an Investment to  
be worth it.',
    'inv-taxRate': 'The Corporate Tax Rate applied to Earnings before Tax.',
    'inv-workingDays': 'The Number of Production Days in a Year.',
    'inv-mfgOverhead': 'Annual Fixed Manufacturing Expenses not tied to Production  
(Rent, Utilities).',
    'inv-sgaExpenses': 'Annual Fixed Selling, General, and Administrative Expenses  
(Salaries, Marketing).',
    'inv-freightExpense': 'Annual Variable Cost of Shipping Finished Goods.',
    'inv-costPerFootStraight': 'The Capital Cost for each Linear Foot of the Straight  
Conveyor Belt.',
    'inv-costPerBend': 'The Capital Cost for each 90-Degree Bend in the Conveyor  
System.',
    'inv-installationCost': 'The Fixed Cost to Install the New or Modified Assembly  
Line.',
    'inv-salvageValue': 'The Estimated Resale Value of Equipment at the end of  
Analysis Period.',
    'inv-std': 'Standard Deviation: The Expected Volatility of Annual Demand around  
the Expected Value.',
    'inv-cv': 'Coefficient of Variation: The Ratio of Standard Deviation to the Mean, to  
Normalize Volatility across Means.'
```

'inv-ciLevel': 'Confidence Interval: The Probability that True Annual Demand falls within the Calculated Range to the Right.',

'inv-p10Demand': 'P10 Demand: The Conservative Forecast; there is a 10% Chance of Demand being at least this Low',

'inv-p90Demand': 'P90 Demand: The Optimistic Forecast; there is a 10% Chance of Demand being at least this High.'

};

```
const tooltip = createTooltip("inv-tooltip");
```

```
const containerElement = container.node();
```

```
for (const [id, text] of Object.entries(tooltips)) {
```

```
  const labelElement = containerElement.querySelector(`label[for="${id}"]`);
```

```
  if (labelElement) {
```

```
    d3.select(labelElement)
```

```
      .on("mouseover", function (event) {
```

```
        tooltip.transition().duration(200).style("opacity", 1);
```

```
        tooltip.html(`<div class="tooltip-row">${text}</div>`)
```

```
          .style("left", (event.pageX + 15) + "px")
```

```
          .style("top", (event.pageY - 28) + "px");
```

```
      })
```

```
      .on("mousemove", function (event) {
```

```
        tooltip.style("left", (event.pageX + 15) + "px")
```

```
          .style("top", (event.pageY - 28) + "px");
```

```
      })
```

```
      .on("mouseout", function () {
```

```
        tooltip.transition().duration(500).style("opacity", 0);
```

```
    });
```

```

    }
  }
}, 10);

} catch (e) { inputArea.html('<p class="error">Could not load input form.</p>');
console.error(e); }

container.append("div").attr("class", "inv-results-column").html(`<div id="inv-results-
placeholder" style="display: none;"></div><div id="inv-results-display"><div class="inv-
scorecard-container"></div><div class="inv-chart-container"></div></div>`);

// **CORRECTED**: Update from location tab first when drawing
const summaryCostEl = document.getElementById('summary-cost');
if (summaryCostEl) {
  const costText = summaryCostEl.textContent;
  const parsedCost = parseFloat(costText.replace(/[$,]/g, "")) || 0;
  if (parsedCost > 0) {
    investmentState.freightExpense = parsedCost;
  }
}

Object.keys(investmentState).forEach(key => {
  const el = document.getElementById(`inv-${key}`);
  if (el) el.value = investmentState[key];
});

const fieldsToFormat = ['inv-mfgOverhead', 'inv-sgaExpenses', 'inv-freightExpense', 'inv-
installationCost', 'inv-salvageValue'];

fieldsToFormat.forEach(id => {

```

```

const input = document.getElementById(id);

if (input) {

  const key = id.replace('inv-', '');

  input.value = formatNumberWithCommas(investmentState[key]);

  input.addEventListener('input', () => {

    const rawValue = parseFormattedNumber(input.value);

    if (key in investmentState) investmentState[key] = rawValue;

    input.value = formatNumberWithCommas(rawValue);

  });

}

});

```

```

container.selectAll("input[data-type='currency'], input[type='number'],
select").on("change", (event) => {

  const key = event.target.id.replace('inv-', '');

  if (key in investmentState) {

    investmentState[key] = event.target.dataset.type === 'currency' ?
    parseFormattedNumber(event.target.value) : (event.target.type === 'select-one' ?
    event.target.value : parseFloat(event.target.value)) || 0;

    if ([ 'std', 'cv', 'p90Demand', 'p10Demand', 'ciLevel' ].includes(key)) {

      updateProbabilisticValues(key.replace('Demand', ''));

    } else {

      clearTimeout(analysisDebounceTimer);

      analysisDebounceTimer = setTimeout(runFullAnalysis, 500);

    }

  }

});

```



```

const controlsArea = inputColumn.append("div").attr("class", "inv-analysis-controls");

controlsArea.html(`<div class="inv-button-group"><button id="inv-
baseCaseBtn">Base Case</button><button id="inv-expansionCaseBtn">Expansion
Case</button></div>`);

controlsArea.select('#inv-baseCaseBtn').on('click', () => { if
(investmentState.runExpansionCase) { investmentState.runExpansionCase = false;
runFullAnalysis(); controlsArea.select('#inv-baseCaseBtn').classed('active', true);
controlsArea.select('#inv-expansionCaseBtn').classed('active', false); } });

controlsArea.select('#inv-expansionCaseBtn').on('click', () => { if
(!investmentState.runExpansionCase) { investmentState.runExpansionCase = true;
runFullAnalysis(); controlsArea.select('#inv-baseCaseBtn').classed('active', false);
controlsArea.select('#inv-expansionCaseBtn').classed('active', true); } });

controlsArea.select(investmentState.runExpansionCase ? '#inv-expansionCaseBtn' :
'#inv-baseCaseBtn').classed('active', true);

let investmentTabListenersAttached = false;

if (!investmentTabListenersAttached) {

    const mainInputs = [dailyDemandInput, opHoursInput, numEmployeesInput,
laborCostInput, superSellInput, superCogsInput, ultraSellInput, ultraCogsInput,
megaSellInput, megaCogsInput];

    mainInputs.forEach(input => {

        if (input) {

            input.addEventListener('input', () => {

                if (document.querySelector('.tab-btn.active')?.dataset.tab === 'investment') {

                    updateProbabilisticValues('mean');

                }

            });

        }

    });

    investmentTabListenersAttached = true;

```

```

    }

    setTimeout(() => updateProbabilisticValues('mean'), 0);
  };

})();

const LayoutTab = (function () {

  /**
   * @tab Layout
   * Draws the animated U-shaped factory layout visualization.
   * This is the main public method exposed by the IIFE.
   */
  function draw() {
    // --- Setup & Validation ---

    // Halt any simulations from other tabs that might be running.
    stopAllSimulations();

    const numEmployees = parseInt(numEmployeesInput.value);

    // Select the SVG container for the visualization and clear any existing content.
    const svg = d3.select("#layout-panel");
    svg.selectAll("*").remove();

    // Get the specific workstation configuration for the given number of employees.
    const config = state.configData[numEmployees];

```

```

// If no configuration data exists, display a message and exit.
if (!config || Object.keys(config).length === 0) {
    svg.append("text").attr("x", "50%").attr("y", "50%").attr("text-anchor",
    "middle").attr("fill", getComputedStyle(root).getPropertyValue('--accent')).text("No
    configuration data for this number of workstations.");
    return;
}

// Check if any workstation's calculated length is too short, which makes the layout
invalid.
let isValidLayout = true;
for (const stationId in config) {
    const elements = config[stationId];
    if (!elements || elements.length === 0) continue;
    // Calculate the total physical length of the station in feet.
    const totalElementTime = elements.reduce((sum, elId) => sum +
    (state.taskData.get(elId)?.elementTime || 0), 0);
    const stationLengthFt = totalElementTime * 15; // Convert element time to feet.
    // A station must be at least 13 feet long to be valid.
    if (stationLengthFt > 0 && stationLengthFt < 13) {
        isValidLayout = false;
        break;
    }
}

// If the layout is invalid, display an error message and exit.

```

```

if (!isLayoutValid) {

    demandStatusEl.textContent = "Invalid Spacing"; // Update status display.

    demandStatusEl.className = "status failure";

    svg.append("text").attr("x", "50%").attr("y", "50%").attr("text-anchor",
"middle").attr("fill", getComputedStyle(root).getPropertyValue('--failure-color')).text("Error:
A workstation's length is less than 13 feet.");

    return;

}

// --- Initial Calculations ---

// Gather operational and financial inputs to calculate performance metrics.

const opInputs = { dailyDemand: parseInt(dailyDemandInput.value), opHours:
parseFloat(opHoursInput.value), numEmployees: parseInt(numEmployeesInput.value) };

const finInputs = { laborCost: parseFloat(laborCostInput.value) };

const results = calculateMetrics(opInputs, finInputs);

// --- LAYOUT CONFIGURATION ---

// Define the page layout using an 80/20 split for the visualization and the control panel.

const { clientWidth: containerWidth, clientHeight: containerHeight } =
document.getElementById('svg-container');

const leftPanelWidth = containerWidth * 0.8;

const rightPanelWidth = containerWidth * 0.2;

const rightPanelX = leftPanelWidth;

const uiPadding = containerWidth * 0.01;

```

```

// --- Path and Point Generation ---

// This section calculates the coordinates for each segment of the U-shaped assembly
line.

const isEven = numEmployees % 2 === 0;

const numLeft = isEven ? numEmployees / 2 : Math.floor(numEmployees / 2);

const middleWsld = isEven ? null : numLeft + 1;

let connectionPoint;

const allPaths = [],
      allPoints = [],
      workstationBorders = [];

// Loop through each workstation to define its geometry.
for (let i = 1; i <= numEmployees; i++) {
  const wsld = i;

  const elements = config[wsld];

  if (!elements || elements.length === 0) continue;

  // Calculate total length of the workstation path.

  const totalElementTime = elements.reduce((sum, elld) => sum +
(state.taskData.get(elld)?.elementTime || 0), 0);

  const totalLengthFt = totalElementTime * 15;

  let p; // 'p' will hold the array of points for the current workstation path.

  // Handle the unique geometry of the middle station in an odd-numbered layout.
  if (wsld === middleWsld) {

    const startPt = { x: 0, y: numLeft * 10 },
          endPt = { x: 10, y: numLeft * 10 };

```

```

const horizontal_segment_ft = 10;

const vertical_leg_ft = Math.max(0, (totalLengthFt - horizontal_segment_ft) / 2);

p = [startPt, { x: startPt.x, y: startPt.y + vertical_leg_ft }, { x: endPt.x, y: endPt.y +
vertical_leg_ft }, endPt];

} else {

    let startPt, endPt, out_dx, out_dy;

    if (wsld <= numLeft) { // Left side of the 'U'

        startPt = { x: 0, y: (wsld - 1) * 10 };

        endPt = { x: 0, y: wsld * 10 };

        out_dx = -1;

        out_dy = 0;

    } else { // Right side of the 'U'

        const mirroredIndex = (isEven ? numLeft : numLeft + 1) - (wsld - numLeft - 1);

        startPt = { x: 10, y: mirroredIndex * 10 };

        endPt = { x: 10, y: (mirroredIndex - 1) * 10 };

        out_dx = 1;

        out_dy = 0;

    }

}

// Handle the special connection point for even-numbered layouts.

if (isEven && (wsld === numLeft || wsld === numLeft + 1)) {

    const leg_to_center = 5,

        leg_from_main = 2,

        mouth_ft = 6;

    const extension_ft = Math.max(0, (totalLengthFt - leg_to_center - mouth_ft -
leg_from_main) / 2);

    if (wsld === numLeft) {

```

```

        p = [startPt, { x: startPt.x, y: startPt.y + leg_from_main }, { x: startPt.x -
extension_ft, y: startPt.y + leg_from_main }, { x: startPt.x - extension_ft, y: startPt.y +
leg_from_main + mouth_ft }, { x: startPt.x, y: startPt.y + leg_from_main + mouth_ft }, { x:
startPt.x + leg_to_center, y: startPt.y + leg_from_main + mouth_ft }]];

        connectionPoint = p[p.length - 1]; // Store the connection point for the next
station.

    } else {

        startPt = connectionPoint;

        endPt = { x: 10, y: (numLeft - 1) * 10 };

        p = [startPt, { x: startPt.x + leg_to_center, y: startPt.y }, { x: startPt.x +
leg_to_center + extension_ft, y: startPt.y }, { x: startPt.x + leg_to_center + extension_ft, y:
startPt.y - mouth_ft }, { x: startPt.x + leg_to_center, y: startPt.y - mouth_ft }, endPt];

    }

} else { // Standard U-shaped workstation path.

    const leg1_ft = 2,

        leg2_ft = 2,

        mouth_ft = 6;

    const extension_ft = Math.max(0, (totalLengthFt - leg1_ft - leg2_ft - mouth_ft) / 2);

    const dx = Math.sign(endPt.x - startPt.x),

        dy = Math.sign(endPt.y - startPt.y);

    p = [startPt, { x: startPt.x + dx * leg1_ft, y: startPt.y + dy * leg1_ft }, { x: startPt.x + dx
* leg1_ft + out_dx * extension_ft, y: startPt.y + dy * leg1_ft + out_dy * extension_ft }, { x:
startPt.x + dx * (leg1_ft + mouth_ft) + out_dx * extension_ft, y: startPt.y + dy * (leg1_ft +
mouth_ft) + out_dy * extension_ft }, { x: endPt.x - dx * leg2_ft, y: endPt.y - dy * leg2_ft },
endPt];

    }

}

allPoints.push(...p); // Add generated points to the master list for scaling.

```

```

// Create a border path string for this workstation.
if (p && p.length > 1) {
    let borderPathString = "M " + p[0].x + " " + p[0].y;
    for (let j = 1; j < p.length; j++) { borderPathString += " L " + p[j].x + " " + p[j].y; }
    workstationBorders.push({ wsID: i, path: borderPathString });
}

// Generate sub-paths for each individual element within the workstation.
const elementColorScale = generateElementColorScale(i - 1, numEmployees,
elements.length);
let currentPathPosFt = 0;
elements.forEach((elId, index) => {
    const task = state.taskData.get(elId);
    allPaths.push({
        wsId: i,
        elId: elId,
        path: generateSubPath(p, currentPathPosFt, (task?.elementTime || 0) * 15),
        color: elementColorScale(index),
        lineCap: (index === 0 || index === elements.length - 1) ? 'butt' : 'round'
    });
    currentPathPosFt += (task?.elementTime || 0) * 15;
});
}

if (allPoints.length === 0) return; // Exit if no points were generated.

```



```

// --- Scaling and Translation ---

// Calculate the bounding box of the entire assembly line path.
const minX_ft = d3.min(allPoints, d => d.x),
      maxX_ft = d3.max(allPoints, d => d.x);
const minY_ft = d3.min(allPoints, d => d.y),
      maxY_ft = d3.max(allPoints, d => d.y);

if ((maxX_ft - minX_ft) <= 0 || (maxY_ft - minY_ft) <= 0) return; // Exit if path has no area.

// Determine the scale factor to fit the path within the available SVG panel space.
const lineBBox = { width: maxX_ft - minX_ft, height: maxY_ft - minY_ft };
const availableLineWidth = leftPanelWidth - (uiPadding * 2);
const availableLineHeight = containerHeight - (uiPadding * 2.5);
const scale = Math.min(availableLineWidth / lineBBox.width, availableLineHeight /
lineBBox.height);

// Calculate translation needed to center the scaled path.
const scaledLineWidth = lineBBox.width * scale;
const leftPadding = (leftPanelWidth - scaledLineWidth) / 1.5;
const translateX = (leftPadding - (minX_ft * scale));
const translateY = uiPadding - (minY_ft * scale);

const g = svg.append("g").attr("transform", `translate(${translateX}, ${translateY})
scale(${scale})`).attr("fill", "none"); // Create the main group for the layout.

// --- UI Element Positioning ---

```

```

const clockY = containerHeight * 0.09;

const clockX = rightPanelX + (rightPanelWidth / 2);

const clockRadius = Math.min(rightPanelWidth * 0.5, containerHeight * 0.15 * 0.5);

const speedoY = containerHeight * 0.33;

const speedoX = clockX;

const speedoRadius = Math.min(rightPanelWidth * 0.5, containerHeight * 0.15 * 0.5);

// --- Clock ---

// Draw the clock face and hands.

const clockGroup = svg.append("g").attr("transform", `translate(${clockX},
${clockY})`); // Position the clock group.

clockGroup.append("circle").attr("r", clockRadius).attr("fill",
getComputedStyle(root).getPropertyValue('--idle-color')).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 2); // Clock face.

for (let i = 0; i < 12; i++) { // Clock ticks.

    const angle = (i / 12) * 2 * Math.PI;

    const tickLength = i % 3 === 0 ? 8 : 4;

    clockGroup.append("line").attr("x1", Math.sin(angle) * (clockRadius -
tickLength)).attr("y1", -Math.cos(angle) * (clockRadius - tickLength)).attr("x2",
Math.sin(angle) * clockRadius).attr("y2", -Math.cos(angle) * clockRadius).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", i % 3 === 0 ? 2 :
1);

}

clockGroup.append("line").attr("id", "sim-clock-hour-hand").attr("y2", -clockRadius *
0.5).attr("stroke", getComputedStyle(root).getPropertyValue('--secondary2')).attr("stroke-
width", 4).attr("stroke-linecap", "round"); // Hour hand.

```

```
clockGroup.append("line").attr("id", "sim-clock-minute-hand").attr("y2", -clockRadius * 0.8).attr("stroke", getComputedStyle(root).getPropertyValue('--secondary2')).attr("stroke-width", 2).attr("stroke-linecap", "round"); // Minute hand.
```

```
clockGroup.append("circle").attr("r", 4).attr("fill", getComputedStyle(root).getPropertyValue('--accent')); // Center pin.
```

```
// --- Animation Controls (Play/Pause, Reset) ---
```

```
const controlsGroup = svg.append("g").attr("transform", `translate(${clockX - 30}, ${clockY + clockRadius + 15})`); // Position controls below clock.
```

```
const playPauseBtn = controlsGroup.append("g").attr("class", "play-pause-btn").style("cursor", "pointer");
```

```
playPauseBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill", getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);
```

```
const playPauseIcon = playPauseBtn.append("text").attr("x", 14).attr("y", 13).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--white').trim()).style("font-size", "14px").text("II"); // Default to pause icon.
```

```
const resetBtn = controlsGroup.append("g").attr("class", "reset-btn").attr("transform", "translate(32, 0)").style("cursor", "pointer");
```

```
resetBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill", getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);
```

```
resetBtn.append("text").attr("x", 14).attr("y", 13).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--white').trim()).style("font-size", "13px").text("⌂"); // Reset icon.
```

```
// --- Speed Slider (robust pointer handling) ---
```

```
const sliderTopPadding = uiPadding * 1.1;
```

```
const sliderHeight = (containerHeight / 6) - sliderTopPadding;
```

```

    const sliderGroup = svg.append("g").attr("transform", `translate(${clockX +
clockRadius + 25}, ${sliderTopPadding})`);

    const minVal = 0.1;

    const maxVal = 8.0;

    const speedScale = d3.scaleLinear().domain([maxVal, minVal]).range([0,
sliderHeight]).clamp(true); // Vertical scale: top is faster.

    sliderGroup.append("line").attr("class", "track").attr("y1", 0).attr("y2",
sliderHeight).attr("stroke", getComputedStyle(root).getPropertyValue('--
accent').trim()).attr("stroke-width", 4).attr("stroke-linecap", "round"); // Slider track.

    sliderGroup.append("circle").attr("id", "d3-layout-slider-handle").attr("class",
"handle").attr("r", 8).attr("fill", getComputedStyle(root).getPropertyValue('--
secondary2').trim()).attr("stroke", getComputedStyle(root).getPropertyValue('--
white').trim()).attr("stroke-width", 2).attr("cy",
speedScale(animationState.speedMultiplier)); // Slider handle.

    const interactionArea = sliderGroup.append("rect").attr("y", 0).attr("height",
sliderHeight).attr("x", -10).attr("width", 20).style("fill", "transparent").style("cursor",
"grab").style("touch-action", "none"); // Larger invisible area for easier interaction.

    // Helper to update speed based on pointer position.

    const setFromPointer = (event) => {

        const getLocalY = (evt) => (evt && evt.sourceEvent && typeof evt.y === 'number') ?
evt.y : d3.pointer(evt && evt.sourceEvent ? evt.sourceEvent : evt, sliderGroup.node())[1];

        const localY = Math.max(0, Math.min(sliderHeight, getLocalY(event)));

        const newValue = speedScale.invert(localY);

        animationState.speedMultiplier = newValue;

        sliderGroup.select(".handle").attr("cy", speedScale(newValue));

    };

```

```
// Attach drag and click events to the interaction area.
```

```
interactionArea
```

```
.on("mousedown", function () { d3.select(this).style("cursor", "grabbing"); })
```

```
.on("mouseup", function () { d3.select(this).style("cursor", "grab"); })
```

```
.on("click", function (event) { setFromPointer(event); })
```

```
.call(
```

```
  d3.drag()
```

```
    .container(() => sliderGroup.node())
```

```
    .on("start", function (event) { if (event.sourceEvent &&  
event.sourceEvent.preventDefault) event.sourceEvent.preventDefault();  
setFromPointer(event); })
```

```
    .on("drag", function (event) { if (event.sourceEvent &&  
event.sourceEvent.preventDefault) event.sourceEvent.preventDefault();  
setFromPointer(event); })
```

```
    .on("end", function (event) { if (event.sourceEvent &&  
event.sourceEvent.preventDefault) event.sourceEvent.preventDefault(); })
```

```
);
```

```
// Attach mouse wheel event for fine-tuning speed.
```

```
interactionArea.on("wheel", function (event) {
```

```
  event.preventDefault();
```

```
  const scrollStep = 0.1;
```

```
  const change = event.deltaY > 0 ? -scrollStep : scrollStep;
```

```
  let newValue = animationState.speedMultiplier + change;
```

```
  newValue = Math.max(minVal, Math.min(maxVal, newValue));
```

```
  animationState.speedMultiplier = newValue;
```

```
  d3.select(this.parentNode).select(".handle").transition().duration(50).attr("cy",  
speedScale(newValue));
```

```

});

// Add label for the speed slider.

sliderGroup.append("text").attr("x", 0).attr("y", sliderHeight / 2 + 120).attr("text-anchor",
"middle").style("font-size", "12px").style("fill", getComputedStyle(root).getPropertyValue('--
accent')).text("Speed");

// --- Speedometer ---

const speedoGroup = svg.append("g").attr("transform", `translate(${speedoX},
${speedoY})`); // Position speedometer group.

const speedoDomain = [0, 15]; // ft/min

const colorThresholds = { slow: 4, medium: 10 };

const radianScale = d3.scaleLinear().domain(speedoDomain).range([-Math.PI / 2,
Math.PI / 2]); // Map speed to angle.

const arcGenerator = d3.arc().innerRadius(speedoRadius *
0.7).outerRadius(speedoRadius).cornerRadius(3);

const arcs = [ // Define colored arcs for the speedometer face.

  { start: speedoDomain[0], end: colorThresholds.slow, color:
getComputedStyle(root).getPropertyValue('--secondary2').trim() },

  { start: colorThresholds.slow, end: colorThresholds.medium, color:
getComputedStyle(root).getPropertyValue('--secondary1').trim() },

  { start: colorThresholds.medium, end: speedoDomain[1], color:
getComputedStyle(root).getPropertyValue('--primary').trim() }

];

speedoGroup.selectAll("path.color-arc").data(arcs).join("path").attr("class", "color-
arc").attr("fill", d => d.color).attr("d", d => arcGenerator({ startAngle: radianScale(d.start),
endAngle: radianScale(d.end) })); // Draw the arcs.

const ticks = radianScale.ticks(6);

```

```
speedoGroup.selectAll("text.tick-label").data(ticks).join("text").attr("class", "tick-label").attr("x", d => Math.sin(radianScale(d)) * (speedoRadius + 15)).attr("y", d => -Math.cos(radianScale(d)) * (speedoRadius + 15)).attr("text-anchor", "middle").attr("dominant-baseline", "central").style("font-size", "12px").style("font-weight", "700").attr("fill", getComputedStyle(root).getPropertyValue('--accent')).text(d => d3.format("d")(d)); // Draw tick labels.
```

```
const targetAngleDeg = (radianScale(Math.min(speedoDomain[1], results.conveyorSpeed || 0)) * 180 / Math.PI); // Calculate target angle for the needle.
```

```
const needle = speedoGroup.selectAll("line.speedo-needle").data([targetAngleDeg]);
```

```
// Create needle at previous angle then animate to target.
```

```
needle.enter()

  .append("line")

  .attr("class", "speedo-needle")

  .attr("id", "speedo-needle")

  .attr("y1", 10).attr("y2", -speedoRadius * 0.9)

  .attr("stroke", getComputedStyle(root).getPropertyValue('--accent').trim())

  .attr("stroke-width", 4).attr("stroke-linecap", "round")

  .attr("transform", `rotate(${animationState.speedo.currentAngle})`) // Start at previous angle.

  .merge(needle)

  .transition().duration(750)

  .attrTween("transform", function (d) { // Animate rotation from old angle to new.

    const startAngle = animationState.speedo.currentAngle;

    const i = d3.interpolate(startAngle, d);

    return t => `rotate(${i(t)})`;

  })

  .on("end", () => {
```

```
        animationState.speedo.currentAngle = targetAngleDeg; // Store the new angle for
next update.
```

```
    });
```

```
    needle.exit().remove();
```

```
    speedoGroup.append("circle").attr("r", 8).attr("fill",
getComputedStyle(root).getPropertyValue('--accent')).attr("stroke",
getComputedStyle(root).getPropertyValue('--white')).attr("stroke-width", 2); // Center pin.
```

```
    speedoGroup.append("text").text(` ${ (results.conveyorSpeed || 0).toFixed(1)} `).attr("y",
speedoRadius * 0.45).attr("text-anchor", "middle").style("font-size", "18px").style("font-
weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--accent')); // Digital
speed readout.
```

```
    speedoGroup.append("text").text("ft/min").attr("y", speedoRadius * 0.65).attr("text-
anchor", "middle").style("font-size", "14px").attr("fill",
getComputedStyle(root).getPropertyValue('--accent')); // Units.
```

```
// --- Finished Goods Bin ---
```

```
const binAreaTopY = speedoY + speedoRadius - (uiPadding * 0.75);
```

```
const binAreaHeight = (containerHeight - uiPadding) - binAreaTopY;
```

```
const maxContentWidth = rightPanelWidth - (uiPadding * 2);
```

```
const maxContentHeight = binAreaHeight;
```

```
// Calculate a grid layout for finished products.
```

```
const capacity = 552;
```

```
const aspectRatio = maxContentWidth / maxContentHeight;
```

```
let numRows = Math.round(Math.sqrt(capacity / aspectRatio));
```

```
if (numRows < 1) numRows = 1;
```

```
let numCols = Math.ceil(capacity / numRows);
```



```
if (numCols < 1) numCols = 1;

const itemSizeWithPadding = Math.min(maxContentWidth / numCols,
maxContentHeight / numRows);

const itemPadding = itemSizeWithPadding * 0.1;
const finalItemSize = itemSizeWithPadding - itemPadding;

const finalContentWidth = numCols * itemSizeWithPadding;
const correctedVisualWidth = finalContentWidth - (itemSizeWithPadding);

// Center the bin within the right panel.
const rightPanelCenterX = rightPanelX + (rightPanelWidth / 2);
const binContentStartX = rightPanelCenterX - (correctedVisualWidth / 2);
const finalContentHeight = numRows * itemSizeWithPadding;
const binAreaCenterY = binAreaTopY + (binAreaHeight / 2);
const binContentStartY = binAreaCenterY - (finalContentHeight / 2);

// Store bin configuration for the simulation.
const binConfig = {
  productPixelSize: finalItemSize,
  itemsPerRow: numCols,
  padding: itemPadding,
  binPixelX: rightPanelCenterX - (finalContentWidth / 2),
  binPixelY_bottom: binContentStartY + finalContentHeight,
};
```

```

// Draw the visual rectangle for the bin.

svg.append("rect")

    .attr("x", binContentStartX).attr("y", binContentStartY)

    .attr("width", correctedVisualWidth).attr("height", finalContentHeight)

    .attr("fill", getComputedStyle(root).getPropertyValue('--idle-color'))

    .attr("stroke", getComputedStyle(root).getPropertyValue('--accent'))

    .attr("stroke-width", 1);

// Draw the bin title.

svg.append("text").text("Finished Goods")

    .attr("x", rightPanelCenterX).attr("y", binContentStartY * 1.1)

    .attr("text-anchor", "middle").style("font-size", "14px").style("font-weight", "bold")

    .attr("fill", getComputedStyle(root).getPropertyValue('--accent'));

// --- Legend and Grid ---

// Draw the legend for product shapes.

const legendGroup = svg.append("g").attr("transform", `translate(${uiPadding},
${containerHeight - 130})`);

    legendGroup.append("rect").attr("width", 160).attr("height", 120).attr("fill",
getComputedStyle(root).getPropertyValue('--white')).attr("rx", 5).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent')); // Legend background.

    legendGroup.append("text").text("Built Models").attr("x", 10).attr("y", 20).style("font-
weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--accent')); // Legend
title.

const legendModels = [{ id: 1, name: "Super" }, { id: 2, name: "Ultra" }, { id: 3, name:
"Mega" }];

legendModels.forEach((model, i) => { // Legend items.

```

```

    const item = legendGroup.append("g").attr("transform", `translate(20, ${40 + i * 22})`);

    createProductShape(item, model.id).attr("transform", "scale(8)");

    item.append("text").text(model.name).style('font-weight', 650).attr("x", 20).attr("y", 4).attr("fill", getComputedStyle(root).getPropertyValue('--accent'));

  });

  legendGroup.append("text").text("Grid: 10ft x 10ft").attr("x", 10).attr("y", 110).style("font-style", "italic").attr("fill", getComputedStyle(root).getPropertyValue('--accent')); // Grid note.

  // Draw the background grid for scale reference.

  const gridGroup = g.append("g");

  const gridBounds = { x1: (0 - translateX) / scale, y1: (0 - translateY) / scale, x2: (containerWidth - translateX) / scale, y2: (containerHeight - translateY) / scale };

  for (let x = Math.floor(gridBounds.x1 / 10) * 10; x <= gridBounds.x2; x += 10)
  { gridGroup.append("line").attr("x1", x).attr("y1", gridBounds.y1).attr("x2", x).attr("y2", gridBounds.y2); }

  for (let y = Math.floor(gridBounds.y1 / 10) * 10; y <= gridBounds.y2; y += 10)
  { gridGroup.append("line").attr("x1", gridBounds.x1).attr("y1", y).attr("x2", gridBounds.x2).attr("y2", y); }

  gridGroup.selectAll("line").attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 0.2).attr("opacity", 0.1); // Style the grid lines.

  // --- Draw Layout Paths ---

  // Helper function to draw the styled element paths.

  const drawElementPaths = (selection) => {

    selection.append("path").attr("d", d => d.path).attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-width", 2.25).attr("stroke-linecap", d => d.lineCap); // Outer border.
  }

```

```
selection.append("path").attr("d", d => d.path).attr("stroke", d => d.color).attr("stroke-
width", 1.75).attr("stroke-linecap", d => d.lineCap).append("title").text(d => `Element
${d.elld}\nWorkstation ${d.wslId}`); // Inner colored path with tooltip.
```

```
};
```

```
g.selectAll("g.element-group").data(allPaths, d => `${d.wslId}-
${d.elld}`).join("g").attr("class", "element-group").call(drawElementPaths);
```

```
g.selectAll("path.workstation-border").data(workstationBorders, d =>
d.wslId).join("path").attr("class", "workstation-border").attr("d", d => d.path).attr("fill",
"none").attr("stroke", getComputedStyle(root).getPropertyValue('--accent')).attr("stroke-
width", 0.3).attr("stroke-linecap", "butt").attr("opacity", 0.6); // Faint workstation outlines.
```

```
// --- Simulation Setup and Initialization ---
```

```
const totalDurationMin = (ASSEMBLY_LINE_LENGTH / results.conveyorSpeed);
```

```
const launchDelayMin = (results.productSpacing / results.conveyorSpeed);
```

```
// Only start the simulation if the calculated times are valid.
```

```
if (isFinite(totalDurationMin) && totalDurationMin > 0 && isFinite(launchDelayMin) &&
launchDelayMin > 0) {
```

```
    // Create a single master path for animating the products.
```

```
    let masterPathString = "";
```

```
    allPaths.forEach((pathData, i) => { masterPathString += i === 0 ? pathData.path :
pathData.path.replace('M', ' '); });
```

```
    const masterPathNode = g.append("path").attr("d", masterPathString).node();
```

```
// Map element IDs to their start/end distances along the master path.
```

```
let cumulativeDist = 0;
```

```
const tempPath = document.createElementNS("http://www.w3.org/2000/svg",
"path");
```

```

const elementMap = allPaths.map(p => {
  tempPath.setAttribute('d', p.path);
  const len = tempPath.getTotalLength();
  const segment = { elementId: p.elld, startDist: cumulativeDist, endDist:
cumulativeDist + len };
  cumulativeDist += len;
  return segment;
});

// The main configuration object passed to the animation engine.
const simulationConfig = {
  svg,
  g,
  masterPathNode,
  elementMap,
  opHours: opInputs.opHours,
  productionQueue: generateProductionQueue(opInputs.dailyDemand),
  totalDurationMs: (ASSEMBLY_LINE_LENGTH / results.conveyorSpeed) * 1000,
  launchDelayMs: (results.productSpacing / results.conveyorSpeed) * 1000,
  binConfig,
  scale
};

// Add event listeners for the play/pause and reset buttons.
playPauseBtn.on("click", () => {
  animationState.layout.isPaused = !animationState.layout.isPaused;

```

```

        playPauseBtn.select('text').text(animationState.layout.isPaused ? "▶" : "⏸");
        if (!animationState.layout.isPaused && !animationState.layout.isRunning) {
            svg.selectAll(".product-shape").remove();
            startSimulation(simulationConfig);
        }
    });

    resetBtn.on("click", () => {
        stopAllSimulations();
        animationState.layout.isPaused = false;
        playPauseBtn.select('text').text("⏸");
        svg.selectAll(".product-shape").remove();
        startSimulation(simulationConfig);
    });

    // Start the simulation automatically.
    startSimulation(simulationConfig);
}

// --- INNER HELPER FUNCTIONS ---

/**
 * Creates an SVG shape (circle, square, or triangle) for a product model.
 * @param {d3.Selection} container - The parent D3 selection to append the shape to.
 * @param {number} modelId - The ID of the model (1=Super, 2=Ultra, 3=Mega).
 * @returns {d3.Selection} The created shape selection.

```

```

*/

function createProductShape(container, modelId) {

  // Map model IDs to colors and shapes.

  const modelColors = { 1: getComputedStyle(root).getPropertyValue('--super-color'), 2:
getComputedStyle(root).getPropertyValue('--ultra-color'), 3:
getComputedStyle(root).getPropertyValue('--mega-color') };

  const modelBorders = { 1: getComputedStyle(root).getPropertyValue('--secondary1'), 2:
getComputedStyle(root).getPropertyValue('--secondary2'), 3:
getComputedStyle(root).getPropertyValue('--primary') };

  const modelShapes = { 1: 'square', 2: 'triangle', 3: 'circle' };

  const shapeType = modelShapes[modelId];

  // Define shape-specific dimensions.

  let shapeSize = 1.5;

  let shape;

  if (shapeType === 'circle') {

    shapeSize = 1.55;

    shape = container.append("circle").attr("r", shapeSize / 2);

  } else if (shapeType === 'square') {

    shapeSize = 1.55;

    shape = container.append("rect").attr("x", -shapeSize / 2).attr("y", -shapeSize /
2).attr("width", shapeSize).attr("height", shapeSize);

  } else if (shapeType === 'triangle') {

    shapeSize = 1.47;

    const h = shapeSize * (Math.sqrt(3) / 2);

    shape = container.append("polygon").attr("points", `0,${-h / 1.5} ${shapeSize /
1.5},${h / 2} ${-shapeSize / 1.5},${h / 2}` );

  }

```

```

    // Apply common styles to the created shape.

    if (shape) {

        shape.attr("fill", modelColors[modelId]).attr("stroke",
modelBorders[modelId]).attr("stroke-width", 0.2).attr("class", "product-shape");

    }

    return shape;

}

/**
 * Moves a product shape to its final position in the finished goods bin.
 * @param {d3.Selection} element - The product shape to move.
 * @param {number} count - The zero-indexed count of finished goods.
 * @param {object} binConfig - The configuration object for the bin layout.
 * @param {d3.Selection} svg - The main SVG container.
 * @param {number} scale - The layout's main scale factor.
 */
function placeInBin(element, count, binConfig, svg) {

    const { binPixelX, binPixelY_bottom, itemsPerRow, productPixelSize, padding } =
binConfig;

    const row = Math.floor(count / itemsPerRow);

    const col = count % itemsPerRow;

    // Move the element from the main 'g' group to the top-level 'svg' to escape the scaling
transform.

    svg.node().appendChild(element.node());

```



```

    // Calculate the new pixel coordinates within the bin.

    const newX = binPixelX + (padding / 2) + (col * productPixelSize) + (productPixelSize / 2)
+ (productPixelSize * 0.75); // Adjusted for centering

    const newY = binPixelY_bottom - (padding / 2) - (row * productPixelSize) -
(productPixelSize / 2);

    const newScale = productPixelSize / 1.8; // Scale the shape to fit the bin slot.

    // Animate the element to its final position and scale.

    element.transition().duration(300).attr('transform', `translate(${newX}, ${newY})
rotate(0) scale(${newScale})`);
}

```

```

/**

```

```

* Generates an SVG path string for a portion of a larger path.
* @param {Array<object>} points - The array of {x, y} points defining the full path.
* @param {number} startFt - The starting distance in feet.
* @param {number} lengthFt - The length of the sub-path in feet.
* @returns {string} The SVG path data string.
*/

```

```

function generateSubPath(points, startFt, lengthFt) {
    let pathString = "M ";
    let traveledFt = 0;
    let started = false;
    for (let i = 1; i < points.length; i++) {
        const prev = points[i - 1];
        const curr = points[i];
        const segLenFt = Math.sqrt(Math.pow(curr.x - prev.x, 2) + Math.pow(curr.y - prev.y, 2));

```

```

// Find the start point of the sub-path.
if (!started && traveledFt + segLenFt >= startFt) {
    const ratio = segLenFt > 0 ? (startFt - traveledFt) / segLenFt : 0;
    pathString += `${prev.x + ratio * (curr.x - prev.x)} ${prev.y + ratio * (curr.y - prev.y)}`;
    started = true;
}

// Add points to the path until the desired length is reached.
if (started) {
    if (traveledFt + segLenFt <= startFt + lengthFt) {
        pathString += ` L ${curr.x} ${curr.y}`;
    } else {
        const ratio = segLenFt > 0 ? (startFt + lengthFt - traveledFt) / segLenFt : 0;
        pathString += ` L ${prev.x + ratio * (curr.x - prev.x)} ${prev.y + ratio * (curr.y -
prev.y)}`;
        return pathString; // End the path here.
    }
}

traveledFt += segLenFt;
}

return pathString;
}

/**

```

* Initializes and runs the main animation loop for the layout simulation.

```

* @param {object} config - The master configuration object for the simulation.
*/

function startSimulation(config) {

    stopAllSimulations(); // Ensure no other loops are running.

    let { svg, g, masterPathNode, productionQueue, totalDurationMs, launchDelayMs,
    binConfig, elementMap } = config;

    if (!masterPathNode || totalDurationMs <= 0 || launchDelayMs <= 0) return; // Validate
    inputs.

    // Initialize the animation state object.

    animationState.layout = { ...config, isRunning: true, isPaused: false, lastFrameTime:
    performance.now(), totalSimTimeMs: 0, nextLaunchTime: 0, productsOnLine: [],
    queueIndex: 0, finishedGoodsCount: 0, pathLength: masterPathNode.getTotalLength() };

    // The core animation loop.

    function animationLoop(currentTime) {

        if (!animationState.layout.isRunning) return; // Exit if stopped.

        // Calculate time delta for smooth animation.

        const realDeltaMs = currentTime - animationState.layout.lastFrameTime;

        animationState.layout.lastFrameTime = currentTime;

        // Advance simulation time if not paused.

        if (!animationState.layout.isPaused) {

            animationState.layout.totalSimTimeMs += realDeltaMs *
            animationState.speedMultiplier;

        }
    }

```

```

// Update the clock display.

const elapsedSimTimeMsForClock = animationState.layout.totalSimTimeMs * 60;

const simMinutes = (elapsedSimTimeMsForClock / 1000) / 60;

const simHours = simMinutes / 60;

d3.select("#sim-clock-minute-hand").attr("transform", `rotate(${(simMinutes % 60) /
60 * 360})`);

d3.select("#sim-clock-hour-hand").attr("transform", `rotate(${(simHours % 12) / 12 *
360})`);


// Launch a new product if it's time.

if (animationState.layout.totalSimTimeMs >= animationState.layout.nextLaunchTime
&& animationState.layout.queueIndex < animationState.layout.productionQueue.length) {

    const modelId =
animationState.layout.productionQueue[animationState.layout.queueIndex];

    animationState.layout.productsOnLine.push({ modelId: modelId, launchTime:
animationState.layout.totalSimTimeMs, element: createProductShape(g, modelId) });

    animationState.layout.queueIndex++;

    animationState.layout.nextLaunchTime += animationState.layout.launchDelayMs;
}


// Update the position of each product on the line.

for (let i = animationState.layout.productsOnLine.length - 1; i >= 0; i--) {

    const product = animationState.layout.productsOnLine[i];

    const progress = (animationState.layout.totalSimTimeMs - product.launchTime) /
animationState.layout.totalDurationMs;

    if (progress >= 1) { // Product has reached the end of the line.

```

```

        placeInBin(product.element, animationState.layout.finishedGoodsCount,
animationState.layout.binConfig, svg);

        animationState.layout.finishedGoodsCount++;

        animationState.layout.productsOnLine.splice(i, 1);

    } else { // Product is still on the line.

        const distance = animationState.layout.pathLength * progress;

        const pos = animationState.layout.masterPathNode.getPointAtLength(distance);
// Get current point on path.

        const nextPos =
animationState.layout.masterPathNode.getPointAtLength(distance + 1); // Get next point
to calculate angle.

        const angle = Math.atan2(nextPos.y - pos.y, nextPos.x - pos.x) * 180 / Math.PI; //
Calculate rotation.


// Add shape-specific offset to better center shapes on the path.

const modelShapes = { 1: 'square', 2: 'triangle', 3: 'circle' };

const shapeType = modelShapes[product.modelId];

let offset = 0.1;

if (shapeType === 'circle') offset = 0;

if (shapeType === 'square') offset = 0.01;

if (shapeType === 'triangle') offset = 0.14;


const perpAngle = angle + 90; // Perpendicular angle for offset.

const offsetX = Math.cos(perpAngle * Math.PI / 180) * offset;

const offsetY = Math.sin(perpAngle * Math.PI / 180) * offset;


        product.element.attr('transform', ` translate(${pos.x + offsetX},${pos.y + offsetY})
rotate(${angle}) `);

```

```

        // Change product color to idle if the current element isn't used for its model type.

        const currentSegment = elementMap.find(e => distance >= e.startDist &&
distance < e.endDist);

        product.element.attr('fill', (currentSegment &&
doesElementBuildModel(currentSegment.elementId, product.modelId)) ?
getComputedStyle(root).getPropertyValue(`--${modelShapes[product.modelId]} ===
'square' ? 'super' : modelShapes[product.modelId] === 'triangle' ? 'ultra' : 'mega'`)-
color`).trim() : getComputedStyle(root).getPropertyValue('--idle-color'));

    }

}

// Continue the loop if there are products on the line or in the queue.

if (animationState.layout.productsOnLine.length > 0 ||
animationState.layout.queueIndex < animationState.layout.productionQueue.length) {

    animationState.layout.frameId = requestAnimationFrame(animationLoop);

} else {

    animationState.layout.isRunning = false;

}

}

animationState.layout.frameId = requestAnimationFrame(animationLoop); // Start the
first frame.

}

// Expose the public draw method.

return {

    draw: draw

```

```
};
```

```
})();
```

```
const LocationTab = () => {
```

```
  // --- Constants and State ---
```

```
  const DEMAND_UNIT_LBS = 410;
```

```
  const TRUCK_CAPACITY_UNITS = 54;
```

```
  const FTL_RATE_PER_MILE = 2.1;
```

```
  const majorCities = {
```

```
    "New York, NY": [-74.0060, 40.7128],
```

```
    "Los Angeles, CA": [-118.2437, 34.0522],
```

```
    "Chicago, IL": [-87.6298, 41.8781],
```

```
    "Houston, TX": [-95.3698, 29.7604],
```

```
    "Phoenix, AZ": [-112.0740, 33.4484],
```

```
    "Philadelphia, PA": [-75.1652, 39.9526],
```

```
    "San Antonio, TX": [-98.4936, 29.4241],
```

```
    "San Diego, CA": [-117.1611, 32.7157],
```

```
    "Dallas, TX": [-96.7970, 32.7767],
```

```
    "Columbus, OH": [-82.9988, 39.9612],
```

```
    "Charlotte, NC": [-80.8431, 35.2271],
```

```
    "Indianapolis, IN": [-86.1581, 39.7684],
```

```
    "Jacksonville, FL": [-81.6557, 30.3322],
```

```
    "San Francisco, CA": [-122.4194, 37.7749],
```

```
"Seattle, WA": [-122.3321, 47.6062],  
"Denver, CO": [-104.9903, 39.7392],  
"Washington, D.C.": [-77.0369, 38.9072],  
"Boston, MA": [-71.0589, 42.3601],  
"Detroit, MI": [-83.0458, 42.3314],  
"Memphis, TN": [-90.0490, 35.1495],  
"Salt Lake City, UT": [-111.8910, 40.7608],  
"Las Vegas, NV": [-115.1398, 36.1699],  
"St. Louis, MO": [-90.1994, 38.6270],  
"Miami, FL": [-80.1918, 25.7617],  
"Atlanta, GA": [-84.3880, 33.7490]  
};
```

```
const cityData = new Map();  
let optimalFactoryLocation = null;  
let totalDemandCapacity = { p10: 0, p50: 0, p90: 0, workingDays: 250 };  
let optimizationMode = 'New'; // 'New' or 'Existing'
```

```
// --- Helper and Calculation Functions ---
```

```
const toRadians = (deg) => deg * (Math.PI / 180);
```

```
const greatCircleDistance = (coords1, coords2) => {  
  if (!coords1 || !coords2) return 0;  
  const [lon1, lat1] = coords1.map(toRadians);  
  const [lon2, lat2] = coords2.map(toRadians);  
  const distanceRad = Math.acos(  
    Math.cos(lat1) * Math.cos(lat2) +  
    Math.sin(lat1) * Math.sin(lat2) * Math.cos(lon1 - lon2)  
  )  
}
```



```

    (Math.sin(lat1) * Math.sin(lat2)) +
    (Math.cos(lat1) * Math.cos(lat2) * Math.cos(lon1 - lon2))
  );
  const meanLat = (coords1[1] + coords2[1]) / 2;
  const radius = 3963.34 - (13.35 * Math.sin(toRadians(meanLat)));
  return distanceRad * radius;
};

```

```

const getCircuitryFactor = (distance) => {
  if (distance >= 250) return 1.2;
  return 1.35;
};

```

```

const calculateLTLCost = (distance, shipmentWeightTons) => {
  const q = shipmentWeightTons;
  const d = distance;
  if (q <= 0 || d <= 0) return 0;
  const numerator = 43.78 * q * d;
  const denominator = (q ** (1 / 7) * d ** (15 / 29)) - 3.5;
  if (denominator <= 0) return Infinity;
  return numerator / denominator;
};

```

```

const runOptimization = () => {
  const cities = Array.from(cityData.values());

```

```

if (optimizationMode === 'New') {
  if (cities.length < 2) {
    optimalFactoryLocation = null;
  } else {
    cities.forEach(c => {
      const costPerShipmentPerMile = getShipmentDetails(null, c,
1).costPerShipment;

      const shipmentsPerYear = 365.2425 / c.freq;

      c.monetaryWeight = costPerShipmentPerMile * shipmentsPerYear;
    });

    let sumLon = 0, sumLat = 0, totalMonetaryWeight = 0;

    cities.forEach(c => {
      sumLon += c.coordinates[0] * c.monetaryWeight;
      sumLat += c.coordinates[1] * c.monetaryWeight;
      totalMonetaryWeight += c.monetaryWeight;
    });

    let currentLocation = [sumLon / totalMonetaryWeight, sumLat /
totalMonetaryWeight];

    for (let i = 0; i < 100; i++) {
      let numLon = 0, numLat = 0, den = 0;

      cities.forEach(city => {
        const d = Math.max(0.001, greatCircleDistance(currentLocation,
city.coordinates));

        numLon += (city.coordinates[0] * city.monetaryWeight) / d;
        numLat += (city.coordinates[1] * city.monetaryWeight) / d;

```

```

        den += city.monetaryWeight / d;
    });
    const nextLocation = [numLon / den, numLat / den];
    if (greatCircleDistance(currentLocation, nextLocation) < 0.1) {
        currentLocation = nextLocation;
        break;
    }
    currentLocation = nextLocation;
}
optimalFactoryLocation = currentLocation;
}
} else { // 'Existing' mode
    if (cities.length < 1) {
        optimalFactoryLocation = null;
    } else {
        let bestLocation = null, minCost = Infinity;
        for (const potentialSite of cities) {
            const currentCost = calculateTotalCost(potentialSite.coordinates, cities);
            if (currentCost < minCost) {
                minCost = currentCost;
                bestLocation = potentialSite.coordinates;
            }
        }
        optimalFactoryLocation = bestLocation;
    }
}
}

```

```

    updateOptimalFactoryMarker();

    updateSummaryPanel();

    updateConnectionLines();
};

// --- D3 Drawing and Updating Functions ---

let projection;

let radiusScale;

const draw = () => {

    const svg = d3.select("#location-panel");

    svg.selectAll("*").remove();

    const defs = svg.append("defs");

    defs.append("marker")

        .attr("id", "arrowhead")

        .attr("viewBox", "0 -5 10 10")

        .attr("refX", 5)

        .attr("refY", 0)

        .attr("markerWidth", 4)

        .attr("markerHeight", 4)

        .attr("orient", "auto")

        .append("path")

        .attr("d", "M0,-5L10,0L0,5")

        .attr("class", "arrowhead");

```

```
const svgContainer = d3.select("#svg-container").node();
const width = svgContainer.getBoundingClientRect().width;
const height = svgContainer.getBoundingClientRect().height;

projection = d3.geoAlbersUsa().scale(width * 1.1).translate([width / 2, height / 2]);
const path = d3.geoPath().projection(projection);
radiusScale = d3.scaleSqrt().domain([100, 100000]).range([4, 25]).clamp(true);

const yShift = height * 0.05;
const mainMapGroup = svg.append("g").attr("transform", `translate(0, ${yShift})`);

mainMapGroup.append("g").attr("class", "us-map").on("click", () =>
infoBox.style("display", "none"));

mainMapGroup.append("g").attr("class", "connection-lines");
mainMapGroup.append("g").attr("class", "optimal-factory-container");
mainMapGroup.append("g").attr("class", "city-markers");

const infoBox = svg.append("foreignObject")
  .attr("width", 200).attr("height", 120).attr("class", "city-info-box").style("display",
"none");

const infoDiv = infoBox.append("xhtml:div");
infoDiv.append("h4").attr("id", "info-header");
infoDiv.append("p").attr("id", "info-demand");
infoDiv.append("p").attr("id", "info-annual-cost");
infoDiv.append("button").text("Remove City").attr("id", "info-remove-btn");

d3.json("https://cdn.jsdelivr.net/npm/us-atlas@3/states-10m.json").then(us => {
```

```
const continentalStates = topojson.feature(us, us.objects.states).features.filter(d =>
d.id !== '02' && d.id !== '15');
```

```
mainMapGroup.select(".us-map").selectAll("path")
  .data(continentalStates)
  .enter().append("path")
  .attr("d", path)
  .attr("class", "state-boundary");
});
```

```
const controls = svg.append("foreignObject").attr("x", 15).attr("y", 15).attr("width",
550).attr("height", 100);
```

```
const controlsDiv = controls.append("xhtml:div").attr("class", "location-controls");
```

```
const cityGroup = controlsDiv.append("div").attr("class", "input-group");
```

```
cityGroup.append("label").text("City");
```

```
const citySelect = cityGroup.append("select").attr("id", "city-select");
```

```
Object.keys(majorCities).sort().forEach(city =>
citySelect.append("option").attr("value", city).text(city));
```

```
const demandGroup = controlsDiv.append("div").attr("class", "input-group");
```

```
demandGroup.append("label").text("Shipment Qty");
```

```
const demandInputGroup = demandGroup.append("div").attr("class", "input-with-
unit");
```

```
demandInputGroup.append("input").attr("type", "number").attr("id", "shipment-
qty").attr("value", "200").attr("min", "1");
```

```
demandInputGroup.append("span").attr("class", "unit-label").text("Units");
```

```

const freqGroup = controlsDiv.append("div").attr("class", "input-group");
freqGroup.append("label").text("Frequency");
const freqInputGroup = freqGroup.append("div").attr("class", "input-with-unit");
freqInputGroup.append("input").attr("type", "number").attr("id", "shipment-
freq").attr("value", "7").attr("min", "1");
freqInputGroup.append("span").attr("class", "unit-label").text("Days");

controlsDiv.append("button").text("Add City").on("click", addCity);

const demandBox = svg.append("foreignObject")
    .attr("class", "demand-capacity-box")
    .attr("x", 15).attr("y", height - 180)
    .attr("width", 220).attr("height", 165);
const demandDiv = demandBox.append("xhtml:div");
demandDiv.append("h4").text("Annual Demand");
demandDiv.append("div").attr("class", "demand-row").html(`<span>P10
(Low):</span><span id="demand-p10">0</span>`);
demandDiv.append("div").attr("class", "demand-row").html(`<span>P50
(Likely):</span><span id="demand-p50">0</span>`);
demandDiv.append("div").attr("class", "demand-row").html(`<span>P90
(High):</span><span id="demand-p90">0</span>`);
demandDiv.append("div").attr("class", "demand-
row").html(`<span>Allocated:</span><span id="demand-allocated">0</span>`);
demandDiv.append("div").attr("class", "demand-bar-container")
    .append("div").attr("class", "demand-bar").attr("id", "demand-bar-fill").text("0%");

const summaryPanel = svg.append("foreignObject").attr("class", "summary-panel")

```

```

        .attr("x", width - 235).attr("y", 15)
        .attr("width", 220).attr("height", 140);
const summaryDiv = summaryPanel.append("xhtml:div");
const switchGroup = summaryDiv.append("div").attr("class", "inv-button-group");
switchGroup.append("button").attr("id", "loc-new-btn").text("New");
switchGroup.append("button").attr("id", "loc-existing-btn").text("Existing");
summaryDiv.append("h4").text("Optimal Summary");

summaryDiv.append("div").attr("class", "demand-row").html(`<span>Annual
Cost:</span><span id="summary-cost">$0</span>`);

summaryDiv.append("div").attr("class", "demand-
row").html(`<span>Shipments:</span><span id="summary-shipments">0</span>`);

summaryDiv.append("div").attr("class", "demand-row").html(`<span>Avg
Cost/Unit:</span><span id="summary-avg-cost">$0.00</span>`);

```

```

d3.select("#loc-new-btn").on('click', () => {
    if (optimizationMode === 'Existing') {
        optimizationMode = 'New';
        d3.select("#loc-new-btn").classed('active', true);
        d3.select("#loc-existing-btn").classed('active', false);
        runOptimization();
    }
});

```

```

d3.select("#loc-existing-btn").on('click', () => {
    if (optimizationMode === 'New') {
        optimizationMode = 'Existing';
        d3.select("#loc-new-btn").classed('active', false);
    }
});

```



```
    d3.select("#loc-existing-btn").classed('active', true);  
    runOptimization();  
  }  
});
```

```
d3.select(optimizationMode === 'New' ? "#loc-new-btn" : "#loc-existing-  
btn").classed('active', true);
```

```
fetchDemandData();
```

```
function addCity() {  
  const name = d3.select("#city-select").property("value");  
  const qty = parseFloat(d3.select("#shipment-qty").property("value"));  
  const freq = parseFloat(d3.select("#shipment-freq").property("value"));  
  
  if (name && qty > 0 && freq > 0) {  
    const annualDemand = (qty / freq) * totalDemandCapacity.workingDays;  
    cityData.set(name, { name, coordinates: majorCities[name], annualDemand, qty,  
freq });  
    updateCityMarkers();  
    runOptimization();  
    updateDemandCapacityBox();  
  }  
}
```

```
d3.select("#info-remove-btn").on("click", function () {  
  const cityToRemove = d3.select(this).attr("data-city-name");
```

```

    if (cityToRemove) {
        cityData.delete(cityToRemove);
        infoBox.style("display", "none");
        updateCityMarkers();
        runOptimization();
        updateDemandCapacityBox();
    }
});

```

```

updateCityMarkers();
runOptimization();

```

```

function updateCityMarkers() {
    const tooltip = createTooltip('city-calc-tooltip');
    const markers = d3.select(".city-markers").selectAll(".city-
marker").data(Array.from(cityData.values()), d => d.name);
    markers.exit().transition().duration(300).attr("r", 0).remove();
    markers.enter()
        .append("circle").attr("class", "city-marker").attr("r", 0)
        .merge(markers)
        .on("mouseover", (event, d) => {
            const details = getShipmentDetails(optimalFactoryLocation, d);
            if (!details) return;

            const annualCost = calculateTotalCostForCity(optimalFactoryLocation, d);
            const avgCostPerUnit = d.annualDemand > 0 ? (annualCost / d.annualDemand) :
0;

```

```

        tooltip.style("opacity", 1).html(
            `<div class="tooltip-header">${d.name} Details</div>

            <div class="tooltip-row"><span>Est. Road Dist:</span>
<span>${details.roadDistance.toFixed(0)} mi</span></div>

            <hr>

            <div class="tooltip-row"><span>FTL Trucks/Ship:</span>
<span>${details.numFTL}</span></div>

            <div class="tooltip-row"><span>LTL Weight/Ship:</span>
<span>${details.remainerTons.toFixed(2)} tons</span></div>

            <hr>

            <div class="tooltip-row"><span>FTL Cost/Ship:</span>
<span>${details.costFTL.toLocaleString('en-US', { style: 'currency', currency: 'USD',
maximumFractionDigits: 0 })}</span></div>

            <div class="tooltip-row"><span>LTL Cost/Ship:</span>
<span>${details.costRemainder.toLocaleString('en-US', { style: 'currency', currency: 'USD',
maximumFractionDigits: 0 })} (${details.remainerChoice})</span></div>

            <hr>

            <div class="tooltip-row"><span>Annual Qty:</span>
<span>${Math.round(d.annualDemand).toLocaleString()}</span></div>

            <div class="tooltip-row"><span>Annual Cost:</span>
<span>${annualCost.toLocaleString('en-US', { style: 'currency', currency: 'USD',
maximumFractionDigits: 0 })}</span></div>

            <div class="tooltip-row"><span>Avg Cost/Unit:</span>
<span>${avgCostPerUnit.toLocaleString('en-US', { style: 'currency', currency:
'USD' })}</span></div>`
        );

```

```
const tooltipNode = tooltip.node();
```

```
if (!tooltipNode) return;
```

```

    const { width, height } = tooltipNode.getBoundingClientRect();

    const padding = 15;

    let left = event.pageX + padding;

    let top = event.pageY + padding;

    if (left + width > window.innerWidth) { left = event.pageX - width - padding; }

    if (top + height > window.innerHeight) { top = event.pageY - height - padding; }

    tooltip.style("left", `${left}px`).style("top", `${top}px`);

  })

  .on("mousemove", (event) => {

    const tooltipNode = tooltip.node();

    if (!tooltipNode) return;

    const { width, height } = tooltipNode.getBoundingClientRect();

    const padding = 15;

    let left = event.pageX + padding;

    let top = event.pageY + padding;

    if (left + width > window.innerWidth) { left = event.pageX - width - padding; }

    if (top + height > window.innerHeight) { top = event.pageY - height - padding; }

    tooltip.style("left", `${left}px`).style("top", `${top}px`);

  })

  .on("mouseout", () => tooltip.style("opacity", 0))

  .on("click", (event, d) => {

    event.stopPropagation();

    const [x, y] = projection(d.coordinates);

    d3.select("#info-header").html(`<span style="color: var(--secondary2);">${d.name}</span>`);

    d3.select("#info-demand").html(`<strong>Demand:</strong>
    ${Math.round(d.annualDemand).toLocaleString()} Units/Yr`);

```

```

        d3.select("#info-annual-cost").html(`<strong>Annual Cost:</strong>
        ${calculateTotalCostForCity(optimalFactoryLocation, d).toLocaleString('en-US', { style:
        'currency', currency: 'USD', maximumFractionDigits: 0 })}`);

        d3.select("#info-remove-btn").attr("data-city-name", d.name);

        infoBox.attr("x", x + 15 + "px").attr("y", y + yShift - 15 + "px").style("display",
        "block");

    })

    .transition().duration(500)

    .attr("r", d => radiusScale(d.annualDemand))

    .attr("transform", d => `translate(${projection(d.coordinates)})`);

}

};

```

```

function fetchDemandData() {

    const p50Display = document.getElementById('inv-p50Demand');

    let p10, p50, p90, workingDays;

    if (p50Display && p50Display.textContent && p50Display.textContent.replace(/,/g,
    '') !== "0") {

        p10 = parseFloat(document.getElementById('inv-p10Demand').value.replace(/,/g, ''))
        || 0;

        p50 = parseFloat(p50Display.textContent.replace(/,/g, '')) || 0;

        p90 = parseFloat(document.getElementById('inv-p90Demand').value.replace(/,/g, ''))
        || 0;

        workingDays = parseFloat(document.getElementById('inv-workingDays')?.value ||
        250);

    } else {

        const daily = parseFloat(document.getElementById('dailyDemand')?.value || 180);
    }
}

```

```
workingDays = 250;

const std = 6750;

p50 = daily * workingDays;

const halfWidth = 1.28155 * std;

p90 = p50 + halfWidth;

p10 = p50 - halfWidth;

}
```

```
totalDemandCapacity = { p10, p50, p90, workingDays };

updateDemandCapacityBox();

}
```

```
function updateDemandCapacityBox() {

  const allocated = Array.from(cityData.values()).reduce((sum, city) => sum +
city.annualDemand, 0);
```

```
  d3.select("#demand-p10")

    .text(Math.round(totalDemandCapacity.p10).toLocaleString())

    .style("font-weight", allocated > totalDemandCapacity.p10 ? "bold" : null)

    .style("color", allocated > totalDemandCapacity.p10 ? "var(--failure-color)" : null);
```

```
  d3.select("#demand-p50")

    .text(Math.round(totalDemandCapacity.p50).toLocaleString())

    .style("font-weight", allocated > totalDemandCapacity.p50 ? "bold" : null)

    .style("color", allocated > totalDemandCapacity.p50 ? "var(--failure-color)" : null);
```

```

d3.select("#demand-p90")
    .text(Math.round(totalDemandCapacity.p90).toLocaleString())
    .style("font-weight", allocated > totalDemandCapacity.p90 ? "bold" : null)
    .style("color", allocated > totalDemandCapacity.p90 ? "var(--failure-color)" : null);

d3.select("#demand-allocated").text(Math.round(allocated).toLocaleString());

const percent = totalDemandCapacity.p50 > 0 ? (allocated /
totalDemandCapacity.p50) * 100 : 0;

const bar = d3.select("#demand-bar-fill");
bar.style("width", `${Math.min(percent, 100)}%`).text(`${Math.round(percent)}%`);
bar.style("background-color", percent > 100 ? "var(--failure-color)" : "var(--primary)");
}

```

```

function updateSummaryPanel() {
    let totalCost = 0;
    let totalShipments = 0;
    let totalAllocatedDemand = 0;
    const cities = Array.from(cityData.values());

    if (optimalFactoryLocation && cities.length > 0) {
        totalCost = calculateTotalCost(optimalFactoryLocation, cities);
        totalShipments = cities.reduce((sum, city) => {
            const shipmentsPerYear = 365.2425 / city.freq;
            const numFTL = Math.floor(city.qty / TRUCK_CAPACITY_UNITS);
            const remainderUnits = city.qty % TRUCK_CAPACITY_UNITS;

```

```

    const totalShipmentsForCity = shipmentsPerYear * (numFTL + (remainderUnits >
0 ? 1 : 0));

    return sum + totalShipmentsForCity;

}, 0);

totalAllocatedDemand = cities.reduce((sum, city) => sum + city.annualDemand, 0);

}

```

```

const avgCostPerUnit = totalAllocatedDemand > 0 ? totalCost /
totalAllocatedDemand : 0;

```

```

d3.select("#summary-cost").text(totalCost.toLocaleString('en-US', { style: 'currency',
currency: 'USD', maximumFractionDigits: 0 }));

d3.select("#summary-shipments").text(Math.round(totalShipments).toLocaleString());

d3.select("#summary-avg-cost").text(avgCostPerUnit.toLocaleString('en-US', { style:
'currency', currency: 'USD' }));

}

```

```

function updateOptimalFactoryMarker() {

  if (!projection) return;

  const container = d3.select(".optimal-factory-container");

  const tooltip = createTooltip('factory-tooltip');

  const data = optimalFactoryLocation ? [optimalFactoryLocation] : [];

  const marker = container.selectAll(".optimal-factory-marker").data(data);

  marker.exit().transition().duration(300).style("opacity", 0).remove();

  marker.enter()

    .append("path")

    .attr("class", "optimal-factory-marker")

```



```

.attr("d", d3.symbol(d3.symbolStar, 250))

.style("opacity", 0)

.merge(marker)

.on("mouseover", (event, d) => {

  tooltip.style("opacity", 1).html(

    `<div class="tooltip-header">Optimal Location</div>

    <div class="tooltip-row">

      <span class="tooltip-key">Est. Yearly Cost:</span>

      <span>${calculateTotalCost(d,
Array.from(cityData.values()).toLocaleString('en-US', { style: 'currency', currency: 'USD',
maximumFractionDigits: 0 })}</span>

    </div>`

  );

})

.on("mousemove", (event) => {

  const tooltipNode = tooltip.node();

  if (!tooltipNode) return;

  const { width, height } = tooltipNode.getBoundingClientRect();

  const padding = 15;

  let left = event.pageX + padding;

  let top = event.pageY + padding;

  if (left + width > window.innerWidth) { left = event.pageX - width - padding; }

  if (top + height > window.innerHeight) { top = event.pageY - height - padding; }

  tooltip.style("left", `${left}px`).style("top", `${top}px`);

})

.on("mouseout", () => tooltip.style("opacity", 0))

.transition().duration(500)

```

```
.attr("transform", d => `translate(${projection(d)})`)
.style("opacity", 1);
}
```

```
function updateConnectionLines() {
  if (!projection || !radiusScale) return;
  const lineGroup = d3.select(".connection-lines");
  const cities = Array.from(cityData.values());

  if (!optimalFactoryLocation || cities.length < 1) {
    lineGroup.selectAll(".connection-group").remove();
    return;
  }
```

```
  const costs = cities.map(city => calculateTotalCostForCity(optimalFactoryLocation,
city));
  const maxCost = d3.max(costs);

  const widthScale = d3.scaleLinear().domain([0, maxCost || 1]).range([1,
8]).clamp(true);

  const dashScale = d3.scaleLinear().domain([1, TRUCK_CAPACITY_UNITS * 3]).range([5,
30]).clamp(true);

  const gapScale = d3.scaleLinear().domain([1, 30]).range([15, 100]).clamp(true);

  const groups = lineGroup.selectAll(".connection-group").data(cities, d => d.name);
  groups.exit().remove();
```

```
const enterGroups = groups.enter()

  .append("g")

  .attr("class", "connection-group");

enterGroups.append("line").attr("class", "connection-line-bg");
enterGroups.append("line").attr("class", "connection-line");

enterGroups.merge(groups)

  .each(function (d) {

    const group = d3.select(this);

    const startPoint = projection(optimalFactoryLocation);

    const endPoint = projection(d.coordinates);

    const radius = radiusScale(d.annualDemand) + 3;

    const dx = endPoint[0] - startPoint[0];

    const dy = endPoint[1] - startPoint[1];

    const lineLength = Math.sqrt(dx * dx + dy * dy);

    if (lineLength === 0) {

      group.selectAll('line').style('display', 'none');

      return;

    } else {

      group.selectAll('line').style('display', null);

    }

    const newEndPointX = endPoint[0] - (dx / lineLength) * radius;

    const newEndPointY = endPoint[1] - (dy / lineLength) * radius;
```

```
const strokeWidth =  
widthScale(calculateTotalCostForCity(optimalFactoryLocation, d));
```

```
group.select(".connection-line-bg")  
  .attr("x1", startPoint[0]).attr("y1", startPoint[1])  
  .attr("x2", newEndPointX).attr("y2", newEndPointY)  
  .attr("marker-end", "url(#arrowhead)")  
  .style("stroke-width", strokeWidth);
```

```
const animLine = group.select(".connection-line")  
  .attr("x1", startPoint[0]).attr("y1", startPoint[1])  
  .attr("x2", newEndPointX).attr("y2", newEndPointY)  
  .style("stroke-width", strokeWidth)  
  .style("stroke", "var(--secondary1)")  
  .attr("stroke-dasharray", `${dashScale(d.qty)} ${gapScale(d.freq)}`)  
  .attr("marker-end", "url(#arrowhead)");
```

```
animLine.transition();
```

```
function repeat() {  
  const totalLength = dashScale(d.qty) + gapScale(d.freq);  
  animLine.attr("stroke-dashoffset", totalLength)  
    .transition()  
    .ease(d3.easeLinear)  
    .duration(d.freq * 100)
```

```

        .attr("stroke-dashoffset", 0)
        .on("end", repeat);
    }
    repeat();
});
}

```

```

function getShipmentDetails(factoryCoords, city, overrideDistance = null) {
    if (!city) return null;
    if (!factoryCoords && !overrideDistance) return null;

    const distance = overrideDistance || greatCircleDistance(factoryCoords,
city.coordinates);

    if (distance <= 10 && !overrideDistance) {
        return { distance, roadDistance: 0, numFTL: 0, costFTL: 0, remainderUnits: city.qty,
remainderTons: 0, costRemainder: 0, remainderChoice: 'Local', costPerShipment: 0 };
    }

    const roadDistance = distance * getCircuitryFactor(distance);
    const numFTL = Math.floor(city.qty / TRUCK_CAPACITY_UNITS);
    const remainderUnits = city.qty % TRUCK_CAPACITY_UNITS;
    const remainderTons = (remainderUnits * DEMAND_UNIT_LBS) / 2000;
    const costFTL = numFTL * FTL_RATE_PER_MILE * roadDistance;
    let costRemainder = 0, remainderChoice = "N/A";

    if (remainderTons > 0) {

```

```

    const ltlCost = calculateLTLCost(roadDistance, remainderTons);
    const ftlCostForRemainder = FTL_RATE_PER_MILE * roadDistance;
    costRemainder = Math.min(ltlCost, ftlCostForRemainder);
    remainderChoice = ltlCost < ftlCostForRemainder ? "LTL" : "FTL";
  }

  return { distance, roadDistance, numFTL, costFTL, remainderUnits, remainderTons,
    costRemainder, remainderChoice, costPerShipment: costFTL + costRemainder };
}

function calculateTotalCostForCity(factoryCoords, city) {
  if (factoryCoords && city.coordinates[0] === factoryCoords[0] && city.coordinates[1]
    === factoryCoords[1]) {
    return 0;
  }
  const details = getShipmentDetails(factoryCoords, city);
  if (!details) return 0;
  const shipmentsPerYear = 365.2425 / city.freq;
  return details.costPerShipment * shipmentsPerYear;
}

function calculateTotalCost(factoryCoords, cities) {
  return cities.reduce((total, city) => total + calculateTotalCostForCity(factoryCoords,
    city), 0);
}

const setupListeners = () => {

```

```

    const idsToWatch = ['inv-p10Demand', 'inv-p90Demand', 'dailyDemand', 'inv-
workingDays'];

    idsToWatch.forEach(id => {

        const input = document.getElementById(id);

        if (input) {

            const eventType = (input.type === 'range' || input.id === 'dailyDemand') ? 'input' :
'change';

            input.addEventListener(eventType, () => {

                if (document.querySelector('.tab-btn.active')?.dataset.tab === 'location') {

                    fetchDemandData();

                }

            });

        }

    });

};

```

```

setTimeout(setupListeners, 1000);

```

```

    return { draw };

})();

```

```

/**

```

```

 * -----

```

```

 * Precedence Chart Tab (IIFE)

```

```

 * -----

```

```

 * Handles the interactive PERT/precedence chart with zoom/pan.

```

```

*/

const PrecedenceTab = (function () {

  // --- MODULE-LEVEL STATE ---

  let precedenceChartNodes = null;

  let pertTooltip = null;


  // --- HELPER FUNCTIONS ---

  function flatten() {

    const directPredecessors = new Map();

    PRECEDENCE_DATA.forEach(el => {

      directPredecessors.set(el.id, new Set(el.predecessors));

    });

    const fullPredecessorMap = new Map();

    const memo = new Map();

    function getAllPredecessors(taskId) {

      if (memo.has(taskId)) return memo.get(taskId);

      const preds = directPredecessors.get(taskId) || new Set();

      const allPreds = new Set(preds);

      preds.forEach(pld => {

        const grandPreds = getAllPredecessors(pld);

        grandPreds.forEach(gpId => allPreds.add(gpId));

      });

      memo.set(taskId, allPreds);

      return allPreds;

    }

    PRECEDENCE_DATA.forEach(el => {

```



```

    fullPredecessorMap.set(el.id, getAllPredecessors(el.id));
  });
  return fullPredecessorMap;
}

```

```

function updatePrecedenceChartColors() {
  if (!precedenceChartNodes) return;
  precedenceChartNodes.selectAll('circle')
    .each(function (d) {
      const circle = d3.select(this);
      const isError = invalidPrecedenceNodes.has(d.id);
      circle.interrupt("blink");
      if (isError) {
        function blink() {
          circle.transition("blink").duration(700)
            .attr("stroke", getComputedStyle(root).getPropertyValue('--failure-
color').trim())
            .attr("stroke-width", 30)
            .style("fill", getComputedStyle(root).getPropertyValue('--failure-color').trim())
            .transition("blink").duration(700)
            .attr("stroke", getComputedStyle(root).getPropertyValue('--failure-
color').trim())
            .attr("stroke-width", 10)
            .style("fill", getComputedStyle(root).getPropertyValue('--failure-color').trim())
            .on("end", blink);
        }
        blink();
      }
    });
}

```

```

    } else {
      circle.transition().duration(500)
        .attr("stroke", getComputedStyle(root).getPropertyValue('--accent').trim())
        .attr("stroke-width", 1.5)
        .style("fill", getComputedStyle(root).getPropertyValue('--white').trim());
    }
  });
}

```

```

function updatePrecedenceChartLinks() {
  if (!precedenceChartNodes) return;
  const allLinks = d3.select("#precedence-panel").selectAll('g > line');

  if (invalidPrecedenceNodes.size === 0) {
    allLinks.transition().duration(300)
      .attr('stroke', getComputedStyle(root).getPropertyValue('--accent').trim())
      .attr('stroke-width', 2.5)
      .attr('marker-end', 'url(#arrowhead)');
    return;
  }
}

```

```

const elementOrderMap = new Map();
let orderIndex = 0;
document.querySelectorAll('.element-row').forEach(row => {
  const taskId = parseInt(row.dataset.taskId);
  elementOrderMap.set(taskId, orderIndex++);
});

```

```

});

const violatingPathNodes = new Set();
for (const violatingNodeid of invalidPrecedenceNodes) {
  const allPredecessors = precedenceMap.get(violatingNodeid) || new Set();
  for (const predecessorid of allPredecessors) {
    if (elementOrderMap.get(predecessorid) > elementOrderMap.get(violatingNodeid))
  {
    violatingPathNodes.add(violatingNodeid);
    violatingPathNodes.add(predecessorid);
  }
}
}

allLinks.each(function (d) {
  const isHighlighted = violatingPathNodes.has(d.source.id) &&
violatingPathNodes.has(d.target.id);
  d3.select(this)
    .transition().duration(300)
    .attr('stroke', isHighlighted ? getComputedStyle(root).getPropertyValue('--failure-
color').trim() : getComputedStyle(root).getPropertyValue('--accent').trim())
    .attr('stroke-width', isHighlighted ? 5.5 : 2.5)
    .attr('marker-end', isHighlighted ? 'url(#arrowhead-highlight)' : 'url(#arrowhead)');
});
}

// --- PUBLIC FUNCTIONS ---

```

```

function update() {
    if (!precedenceChartNodes) return;
    updatePrecedenceChartColors();
    updatePrecedenceChartLinks();
}

/**
 * Draw the interactive precedence network graph.
 */
function draw() {
    // Data

    const nodes = PRECEDENCE_DATA.map(d => ({ id: d.id }));
    const links = [];
    PRECEDENCE_DATA.forEach(d => {
        d.predecessors.forEach(pld => links.push({ source: pld, target: d.id }));
    });

    // Base SVG

    const svg = d3.select("#precedence-panel");
    svg.selectAll("*").remove();

    // Size + viewBox for consistent zooming/panning

    const width = document.getElementById('svg-container').clientWidth;
    const height = document.getElementById('svg-container').clientHeight;
    svg.attr("viewBox", `0 0 ${width} ${height}`);

```

```

// Markers

svg.append('defs').selectAll('marker')
  .data(['arrowhead', 'arrowhead-highlight'])
  .join('marker')
  .attr('id', d => d)
  .attr('viewBox', '0 -5 10 10')
  .attr('refX', 10)
  .attr('orient', 'auto')
  .attr('markerWidth', 6)
  .attr('markerHeight', 6)
  .append('path')
  .attr('d', 'M0,-5L10,0L0,5')
  .attr('fill', d => d === 'arrowhead-highlight'
    ? getComputedStyle(root).getPropertyValue('--failure-color').trim()
    : getComputedStyle(root).getPropertyValue('--accent').trim());

// --- IMPORTANT: zoom catcher (behind everything) ---
const zoomPane = svg.append("rect")
  .attr("class", "zoom-pane")
  .attr("x", 0).attr("y", 0)
  .attr("width", width).attr("height", height)
  .style("fill", "none")
  .style("pointer-events", "all"); // ensures zoom events are captured

// Main group (transformed by zoom)
const mainGroup = svg.append("g");

```

```

// Tooltip

pertTooltip = createTooltip('pert-tooltip').style("position", "fixed");


// Force layout

const simulation = d3.forceSimulation(nodes)

  .force("link", d3.forceLink(links).id(d => d.id).distance(40))

  .force("charge", d3.forceManyBody().strength(-500))

  .force("center", d3.forceCenter(width / 2, height / 2).strength(0.1))

  .force("collide", d3.forceCollide().radius(d => (d.r || 50) + 8).strength(1));


const link = mainGroup.append("g").selectAll("line").data(links).join("line")

  .attr("class", "precedence-link")

  .attr("marker-end", "url(#arrowhead)");


precedenceChartNodes = mainGroup.append("g").selectAll("g").data(nodes).join("g");


// --- CLAMP HELPERS ---

const CLAMP_PAD = 12;

const clamp = (v, a, b) => Math.max(a, Math.min(b, v));

function clampToViewport(d) {

  const r = (d.r || 12);

  const minX = CLAMP_PAD + r;

  const maxX = width - CLAMP_PAD - r;

  const minY = CLAMP_PAD + r;

  const maxY = height - CLAMP_PAD - r;

```

```

d.x = clamp(d.x, minX, maxX);
d.y = clamp(d.y, minY, maxY);
if (d.fx !== null) d.fx = clamp(d.fx, minX, maxX);
if (d.fy !== null) d.fy = clamp(d.fy, minY, maxY);
}

// TICK (clamp every tick so sim can't push nodes out)
simulation.on("tick", () => {
  nodes.forEach(clampToViewport);

  link.each(function (d) {
    const targetRadius = (d.target.r || 12) + 3;
    const dx = d.target.x - d.source.x;
    const dy = d.target.y - d.source.y;
    const distance = Math.sqrt(dx * dx + dy * dy);
    let x2 = d.target.x, y2 = d.target.y;
    if (distance > 0) {
      const ratio = (distance - targetRadius) / distance;
      x2 = d.source.x + dx * ratio;
      y2 = d.source.y + dy * ratio;
    }
    d3.select(this)
      .attr("x1", d.source.x).attr("y1", d.source.y)
      .attr("x2", x2).attr("y2", y2);
  });
  precedenceChartNodes.attr("transform", d => `translate(${d.x}, ${d.y})`);

```

```
});
```

```
// ----- LEGEND (same look, just moved to bottom-right corner) -----
```

```
function renderPrecedenceLegend() {
```

```
    // Layout constants
```

```
    const legendPadding = 12;
```

```
    const swatch = { w: 14, h: 14 };
```

```
    const colGap = 14; // space between the two columns
```

```
    const rowGap = 10; // space between the two rows
```

```
    const labelOffsetX = 8; // text offset from swatch
```

```
    const topGap = 20; // space below title to the first row
```

```
    const bottomGap = 2;
```

```
    // Compute legend width and height
```

```
    const colWidth = swatch.w + labelOffsetX + 56;
```

```
    const legendWidth = legendPadding * 2 + colWidth * 2;
```

```
    const legendHeight = legendPadding * 2 + topGap + (swatch.h + rowGap) * 2 +  
bottomGap + 14;
```

```
    // Move legend to bottom-right corner with small margin
```

```
    const legendX = width - legendWidth - 20;
```

```
    const legendY = height - legendHeight - 20;
```

```
    const g = svg.append('g')
```

```
        .attr('id', 'precedence-legend')
```

```
        .attr('transform', `translate(${legendX}, ${legendY})` )
```



```
.style('pointer-events', 'none');
```

```
// Card
```

```
g.append('rect')
```

```
.attr('width', legendWidth)
```

```
.attr('height', legendHeight)
```

```
.attr('rx', 10)
```

```
.attr('fill', getComputedStyle(root).getPropertyValue('--white').trim())
```

```
.attr('stroke', getComputedStyle(root).getPropertyValue('--accent').trim());
```

```
const centerX = legendWidth / 2;
```

```
// Title (centered)
```

```
g.append('text')
```

```
.text('Build Ratios')
```

```
.attr('x', centerX)
```

```
.attr('y', 18)
```

```
.attr('text-anchor', 'middle')
```

```
.style('font-weight', 700)
```

```
.style('font-size', '13px')
```

```
.attr('fill', getComputedStyle(root).getPropertyValue('--accent').trim());
```

```
// Grid data
```

```
const itemsGrid = [
```

```
[
```

```
  { label: 'Super', color: PERT_PIE_COLORS.super },
```

```

    { label: 'Ultra', color: PERT_PIE_COLORS.ultra },
  ],
  [
    { label: 'Mega', color: PERT_PIE_COLORS.mega },
    { label: 'Idle', color: PERT_PIE_COLORS.idle },
  ],
];

```

```

const gridOriginX = legendPadding;
const gridOriginY = legendPadding + topGap;

```

```

// Render grid

```

```

itemsGrid.forEach((rowItems, rowIndex) => {
  rowItems.forEach((item, colIndex) => {
    const gx = gridOriginX + colIndex * (colWidth + colGap);
    const gy = gridOriginY + rowIndex * (swatch.h + rowGap);

    const row = g.append('g').attr('transform', `translate(${gx}, ${gy})`);
    row.append('rect')
      .attr('width', swatch.w).attr('height', swatch.h)
      .attr('fill', item.color)
      .attr('stroke', getComputedStyle(root).getPropertyValue('--white').trim())
      .attr('stroke-width', 1);

    row.append('text')
      .text(item.label)

```

```

        .attr('x', swatch.w + labelOffsetX)
        .attr('y', swatch.h - 2)
        .style('font-size', '12px')
        .style('font-weight', 650)
        .attr('fill', getComputedStyle(root).getPropertyValue('--accent').trim());
    });
});

// Caption (centered)
g.append('text')
    .text('Node size = Labor time')
    .attr('x', centerX)
    .attr('y', legendHeight - legendPadding)
    .attr('text-anchor', 'middle')
    .style('font-size', '12px')
    .style('font-weight', 600)
    .attr('fill', getComputedStyle(root).getPropertyValue('--accent').trim());
}

```

```

// Label helpers
function addPERTLabelBackgrounds() {
    if (!precedenceChartNodes) return;
    precedenceChartNodes.each(function (d) {
        if (!d || d.id == null || !d.r) return;

```

```

const g = d3.select(this);

g.insert('circle', 'text').attr('class', '__pert_label_bg')

  .style('pointer-events', 'none')

  .attr('r', Math.max(11, d.r * 0.48))

  .attr('fill', getComputedStyle(root).getPropertyValue('--white').trim())

  .attr('fill-opacity', 0.95)

  .attr('stroke', getComputedStyle(root).getPropertyValue('--accent').trim())

  .attr('stroke-opacity', 0.20)

  .attr('stroke-width', 1);

});

}

function restylePERTNodeLabelsStrong() {

  if (!precedenceChartNodes) return;

  precedenceChartNodes.each(function (d) {

    if (!d || d.id == null || !d.r) return;

    const fs = Math.max(15, Math.min(26, d.r * 0.42));

    d3.select(this).select('text').raise()

      .attr('text-anchor', 'middle').attr('dy', '0.35em')

      .style('font-family', 'sans-serif').style('font-weight', '800')

      .style('font-size', fs + 'px')

      .style('fill', getComputedStyle(root).getPropertyValue('--accent').trim())

      .style('stroke', getComputedStyle(root).getPropertyValue('--white').trim())

      .style('stroke-width', '4px')

      .style('paint-order', 'stroke')

      .style('pointer-events', 'none');

  });

```

```
}
```

```
// PERT pies
```

```
function getPertLaborTime(id) {
```

```
    const t = state?.taskData?.get?.(id)?.laborTime;
```

```
    return Number.isFinite(t) ? t : (PERT_LABOR_FALLBACK[id] || 0);
```

```
}
```

```
function drawPERTNodePiesOnce() {
```

```
    if (!precedenceChartNodes || precedenceChartNodes.empty()) return;
```

```
    const times = nodes.map(d => getPertLaborTime(+d.id));
```

```
    if (!times.length) return;
```

```
    const rScale = d3.scaleLinear().domain(d3.extent(times)).range([14, 56]).nice();
```

```
    const arc = d3.arc().innerRadius(0);
```

```
    const pie = d3.pie().sort(null).value(d => d.value);
```

```
precedenceChartNodes.each(function (d) {
```

```
    const g = d3.select(this);
```

```
    const id = +d.id;
```

```
    const r = rScale(getPertLaborTime(id));
```

```
    d.r = r;
```

```
    g.select('circle').remove();
```

```
    g.append('circle')
```

```
        .attr('r', r)
```

```
        .attr('fill', 'transparent')
```

```
        .style('pointer-events', 'all');
```

```

const row = state.taskData.get(id);

if (!row) return;

const { elementTime: ET, Super: sup, Mega: meg, Ultra: ult } = row;

const slices = [
  { key: 'super', value: ET * sup, color: PERT_PIE_COLORS.super },
  { key: 'mega', value: ET * meg, color: PERT_PIE_COLORS.mega },
  { key: 'ultra', value: ET * ult, color: PERT_PIE_COLORS.ultra },
  { key: 'idle', value: Math.max(0, ET * (1 - (sup + meg + ult))), color:
PERT_PIE_COLORS.idle }
].filter(s => s.value > 1e-6);

const arcGen = arc.outerRadius(r);

g.selectAll('path.__pert_pie')
  .data(pie(slices))
  .join('path')
  .attr('class', '__pert_pie')
  .attr('d', arcGen)
  .style('fill', a => a.data.color)
  .style('stroke', PERT_PIE_STROKE)
  .style('stroke-width', '0.9px');

g.selectAll('text').data([d]).join('text').text(d => d.id);

g.on('mouseenter', (event) => {

```

```

    pertTooltip.style('opacity', 1).html(
      `<div class="tooltip-header">Element ${id}</div>
      <div class="tooltip-row"><span>Labor Time:</span>
<b>${getPertLaborTime(id).toFixed(2)}</b></div>
      <div class="tooltip-row">Super: <b>${(sup * 100).toFixed(0)}%</b></div>
      <div class="tooltip-row">Ultra: <b>${(ult * 100).toFixed(0)}%</b></div>
      <div class="tooltip-row">Mega: <b>${(meg * 100).toFixed(0)}%</b></div>`
    );
  }).on('mousemove', (event) => {
    pertTooltip.style('left', (event.clientX + 14) + 'px')
      .style('top', (event.clientY + 14) + 'px');
  }).on('mouseleave', () => {
    pertTooltip.style('opacity', 0);
  });
});

addPERTLabelBackgrounds();
restylePERTNodeLabelsStrong();
}

// --- ZOOM / PAN ---
const zoom = d3.zoom()
  .scaleExtent([0.1, 8])
  .on("zoom", (event) => {
    mainGroup.attr("transform", event.transform);
  });

```

```
// Attach zoom to the svg and its zoomPane
svg.call(zoom);

zoomPane.call(zoom); // ensure the catcher gets events


// Default: start zoomed out and centered
const DEFAULT_ZOOM = 0.95; // < 1 = zoom out
const tx = (width - width * DEFAULT_ZOOM) / 2;
const ty = (height - height * DEFAULT_ZOOM) / 2;
const initialTransform = d3.zoomIdentity.translate(tx, ty).scale(DEFAULT_ZOOM);
svg.call(zoom.transform, initialTransform);


// --- DRAG (kept inside viewport, zoom-aware) ---
function dragstarted(event, d) {
  if (event.sourceEvent && event.sourceEvent.stopPropagation) {
    event.sourceEvent.stopPropagation();
  }
  if (!event.active) simulation.alphaTarget(0.3).restart();
  const t = d3.zoomTransform(svg.node());
  const [lx, ly] = t.invert([event.x, event.y]);
  d.fx = lx;
  d.fy = ly;
}

function dragged(event, d) {
  const t = d3.zoomTransform(svg.node());
  const [lx, ly] = t.invert([event.x, event.y]);
```



```

const r = (d.r || 12);
const minX = CLAMP_PAD + r;
const maxX = width - CLAMP_PAD - r;
const minY = CLAMP_PAD + r;
const maxY = height - CLAMP_PAD - r;

d.fx = Math.max(minX, Math.min(maxX, lx));
d.fy = Math.max(minY, Math.min(maxY, ly));
}
function dragended(event, d) {
  if (!event.active) simulation.alphaTarget(0);
  d.fx = d.x;
  d.fy = d.y;
}

precedenceChartNodes.call(
  d3.drag()
    .on("start", dragstarted)
    .on("drag", dragged)
    .on("end", dragended)
);

// --- FINAL RENDERING ---
renderPrecedenceLegend();
drawPERTNodePiesOnce();

```

```

    updatePrecedenceChartColors();
    updatePrecedenceChartLinks();
  }

  return { draw, update, flatten };
})();

const ProfitTab = (function () {
  function draw() {
    // --- INITIAL SETUP ---

    const svg = d3.select("#profit-panel");

    const { clientWidth: width, clientHeight: height } = document.getElementById('svg-container');

    svg.selectAll("*").remove();

    const data = profitMaximizationCache.data;
    if (!data) {
      svg.append("text")
        .attr("x", width / 2)
        .attr("y", height / 2)
        .attr("text-anchor", "middle")
        .text("Calculating profit data, please wait...");
      return;
    }
  }

```

```

// --- LAYOUT & SCALES ---

const margin = { top: 40, right: 60, bottom: 60, left: 80 };

const breakdownWidth = Math.max(280, width * 0.32);
const chartsWidth = width - breakdownWidth;
const chartWidth = chartsWidth - margin.left - margin.right;
const chartHeight = (height / 2) - margin.top - margin.bottom;

const chartsGroup = svg.append("g");
const breakdownGroup = svg.append("g").attr("transform",
`translate(${chartsWidth},0)`);

const op = {
  dailyDemand: +dailyDemandInput.value,
  opHours: +opHoursInput.value,
  numEmployees: +numEmployeesInput.value
};

const fin = {
  laborCost: +laborCostInput.value,
  superSell: +superSellInput.value,
  superCogs: +superCogsInput.value,
  ultraSell: +ultraSellInput.value,
  ultraCogs: +ultraCogsInput.value,
  megaSell: +megaSellInput.value,
  megaCogs: +megaCogsInput.value,
};

```

```
const m = calculateMetrics(op, fin);
```

```
const x = d3.scaleLinear().domain([50, 552]).range([0, chartWidth]);
```

```
const currentProfit = m.dailyGrossProfit;
```

```
const yProfit = d3.scaleLinear()  
  .domain([  
    Math.min(currentProfit, d3.min(data.profitData, d => d.value)),  
    Math.max(currentProfit, d3.max(data.profitData, d => d.value))  
  ])  
  .nice()  
  .range([chartHeight, 0]);
```

```
const filteredMarginData = data.marginData.filter(d => d.demand > 50);
```

```
const yMargin = d3.scaleLinear()  
  .domain([  
    Math.min(m.grossProfitMargin, d3.min(filteredMarginData, d => d.value)),  
    Math.max(m.grossProfitMargin, d3.max(filteredMarginData, d => d.value))  
  ])  
  .nice()  
  .range([chartHeight, 0]);
```

```
// --- HELPERS & FORMATTERS ---
```

```
const bisect = d3.bisector(d => d.demand).left;
```

```
const fmtMoney = d3.format("$,.0f");
```

```
const fmtPct = v => `${d3.format(".1f")(v)}%`;
```

```

const reqX = x(op.dailyDemand);

const actX = x(m.throughputUnitsPerDay);


const tooltip = createTooltip('profit-tooltip').style("position", "fixed");

const showTT = (html, ev) => tooltip.html(html).style("opacity", 1)

    .style("left", (ev.clientX + 14) + "px")

    .style("top", (ev.clientY - 24) + "px");

const hideTT = () => tooltip.style("opacity", 0);


function drawAxesWithGrid(g, xScale, yScale) {

    // grids

    g.append("g")

        .attr("class", "grid-major")

        .call(d3.axisLeft(yScale).ticks(8).tickSize(-chartWidth).tickFormat(""));

    g.append("g")

        .attr("class", "grid-major")

        .attr("transform", `translate(0,${chartHeight})`)

        .call(d3.axisBottom(xScale).ticks(12).tickSize(-chartHeight).tickFormat(""));


    // axes

    g.append("g")

        .attr("class", "axis")

        .attr("transform", `translate(0,${chartHeight})`)

        .call(d3.axisBottom(xScale).ticks(12).tickFormat(d3.format("d")));

    g.append("g")

```

```

        .attr("class", "axis")

        .call(d3.axisLeft(yScale).ticks(6).tickFormat(yScale === yProfit ? fmtMoney : d =>
fmtPct(d)));
    }

    // --- PROFIT CHART (TOP) ---

    const gP = chartsGroup.append("g").attr("transform",
`translate(${margin.left},${margin.top})`);

    drawAxesWithGrid(gP, x, yProfit);

    const vGuideP = gP.append("line").attr("class", "crosshair").style("display", "none");
    const hGuideP = gP.append("line").attr("class", "crosshair-h").style("display", "none");

    gP.append("path")

        .datum(data.profitData.filter(d => d.demand > 50))

        .attr("class", "line-profit")

        .attr("fill", "none")

        .attr("stroke-width", 2.6)

        .attr("d", d3.line().x(d => x(d.demand)).y(d => yProfit(d.value)));

    const y_at_act_profit = yProfit(data.profitData[Math.max(0, bisect(data.profitData,
m.throughputUnitsPerDay, 1) - 1)].value);

    const y_current_profit = yProfit(m.dailyGrossProfit);

    gP.append("line")

        .attr("stroke", getComputedStyle(root).getPropertyValue('--accent').trim())

        .attr("stroke-width", 1.5)

```

```

.attr("x1", actX).attr("x2", actX)

.attr("y1", y_at_act_profit).attr("y2", y_current_profit);

if (op.dailyDemand > m.throughputUnitsPerDay) {

    const y_at_req = yProfit(data.profitData[Math.max(0, bisect(data.profitData,
op.dailyDemand, 1) - 1)].value);

    gP.append("path")

        .attr("d", `M ${actX},${y_at_act_profit} L ${reqX},${y_at_req} L
${reqX},${y_current_profit} L ${actX},${y_current_profit} Z`)

        .attr("class", "lost-profit-area");

    gP.append("line")

        .attr("stroke", getComputedStyle(root).getPropertyValue('--failure-color').trim())

        .attr("stroke-width", 1.5)

        .attr("x1", reqX).attr("x2", reqX)

        .attr("y1", y_at_req).attr("y2", y_current_profit);

}

gP.append("circle").attr("class", "point-now").attr("cx", actX).attr("cy",
y_current_profit).attr("r", 5);

gP.append("text")

    .attr("x", chartWidth / 2)

    .attr("y", -14)

    .attr("text-anchor", "middle")

    .style("font-weight", 800)

    .text("Max Gross Profit vs Daily Demand");

gP.append("rect")

```

```

.attr("width", chartWidth)

.attr("height", chartHeight)

.attr("fill", "transparent")

.style("pointer-events", "all")

.on("mousemove", (ev) => {

    const d = data.profitData[Math.max(0, bisect(data.profitData,
Math.round(x.invert(d3.pointer(ev)[0])), 1) - 1)];

    if (!d) return;

    vGuideP.style("display", null).attr("x1", x(d.demand)).attr("x2",
x(d.demand)).attr("y1", 0).attr("y2", chartHeight);

    hGuideP.style("display", null).attr("x1", 0).attr("x2", chartWidth).attr("y1",
yProfit(d.value)).attr("y2", yProfit(d.value));

    showTT(

        `
```



```

const vGuideM = gM.append("line").attr("class", "crosshair").style("display", "none");
const hGuideM = gM.append("line").attr("class", "crosshair-h").style("display", "none");

gM.append("path")
  .datum(filteredMarginData)
  .attr("class", "line-margin")
  .attr("fill", "none")
  .attr("stroke-width", 2.6)
  .attr("d", d3.line().x(d => x(d.demand)).y(d => yMargin(d.value)));

const y_at_act_margin = yMargin(data.marginData[Math.max(0,
bisect(data.marginData, m.throughputUnitsPerDay, 1) - 1)].value);
const y_current_margin = yMargin(m.grossProfitMargin);

gM.append("line")
  .attr("stroke", getComputedStyle(root).getPropertyValue('--accent').trim())
  .attr("stroke-width", 1.5)
  .attr("x1", actX).attr("x2", actX)
  .attr("y1", y_at_act_margin).attr("y2", y_current_margin);

if (op.dailyDemand > m.throughputUnitsPerDay) {
  const y_at_req_margin = yMargin(data.marginData[Math.max(0,
bisect(data.marginData, op.dailyDemand, 1) - 1)].value);
  gM.append("path")
    .attr("d", ` M ${actX},${y_at_act_margin} L ${reqX},${y_at_req_margin} L
    ${reqX},${y_current_margin} L ${actX},${y_current_margin} Z ` )

```

```
.attr("class", "lost-profit-area");  
gM.append("line")  
    .attr("stroke", "red")  
    .attr("stroke-width", 1.5)  
    .attr("x1", reqX).attr("x2", reqX)  
    .attr("y1", y_at_req_margin).attr("y2", y_current_margin);  
}
```

```
gM.append("circle").attr("class", "point-now").attr("cx", actX).attr("cy",  
y_current_margin).attr("r", 5);
```

```
gM.append("text")  
    .attr("x", chartWidth / 2)  
    .attr("y", -14)  
    .attr("text-anchor", "middle")  
    .style("font-weight", 800)  
    .text("Max Gross Profit Margin vs Daily Demand");
```

// Place the x-axis label using bottom margin space.

```
gM.append("text")  
    .attr("class", "axis-label")  
    .attr("x", chartWidth / 2)  
    .attr("y", chartHeight + (margin.bottom - 12))  
    .attr("text-anchor", "middle")  
    .text("Daily Demand (units)");
```

```

gM.append("rect")

.attr("width", chartWidth)

.attr("height", chartHeight)

.attr("fill", "transparent")

.style("pointer-events", "all")

.on("mousemove", (ev) => {

    const d = data.marginData[Math.max(0, bisect(data.marginData,
Math.round(x.invert(d3.pointer(ev)[0])), 1) - 1)];

    if (!d) return;

    vGuideM.style("display", null).attr("x1", x(d.demand)).attr("x2",
x(d.demand)).attr("y1", 0).attr("y2", chartHeight);

    hGuideM.style("display", null).attr("x1", 0).attr("x2", chartWidth).attr("y1",
yMargin(d.value)).attr("y2", yMargin(d.value));

    showTT(

        `<div class="tooltip-header">Demand: ${d.demand}</div>

        <div class="tooltip-row"><span class="tooltip-key">Optimal
Margin</span><span>${fmtPct(d.value)}</span></div>

        <div class="tooltip-row"><span class="tooltip-key">#
Workstations</span><span>${d.config.emp}</span></div>

        <div class="tooltip-row"><span class="tooltip-key">Oper
Hours</span><span>${d.config.hrs}</span></div>`, ev);

    })

    .on("mouseleave", () => { vGuideM.style("display", "none"); hGuideM.style("display",
"none"); hideTT(); });

// --- BREAKDOWN PANEL (RIGHT SIDE) ---

const totalLabor = op.numEmployees * op.opHours * fin.laborCost;

```

```
const perModel = ["super", "ultra", "mega"].map(key => {  
  const units = m.throughputUnitsPerDay * BUILD_RATIOS[key];  
  const sales = units * fin[`${key}Sell`];  
  const cogs = units * fin[`${key}Cogs`];  
  const labor = totalLabor * BUILD_RATIOS[key];  
  const profit = sales - cogs - labor;  
  return {  
    label: key[0].toUpperCase() + key.slice(1),  
    sales, cogs, labor, profit,  
    margin: sales > 0 ? (profit / sales) * 100 : 0  
  };  
});
```

```
const totalProfit = d3.sum(perModel, d => d.profit);  
const pad = Math.max(16, breakdownWidth * 0.10);
```

```
// --- PIE CHARTS (top half) ---
```

```
const topHalf = breakdownGroup.append("g");
```

```
topHalf.append("rect")
```

```
  .attr("class", "breakdown-border")
```

```
  .attr("x", pad / 2).attr("y", pad / 2)
```

```
  .attr("width", breakdownWidth - pad)
```

```
  .attr("height", height / 2 - pad);
```

```
// ADD TITLE: Top (pies)
```

```
topHalf.append("text")
```

```

.attr("x", pad / 2 + (breakdownWidth - pad) / 2)
.attr("y", pad / 2 + 20)
.attr("text-anchor", "middle")
.attr("class", "panel-title")
.style("font-weight", 800)
.text("Cost & Profit Composition");

const titleOffset = 8;

const pies = [
  { title: "Overall", data: { Profit: totalProfit, Labor: totalLabor, Material:
d3.sum(perModel, d => d.cogs) } },
  ...perModel.map(d => ({ title: d.label, data: { Profit: d.profit, Labor: d.labor, Material:
d.cogs } }))
];

const pieColors = {
  Profit: getComputedStyle(root).getPropertyValue('--primary').trim(),
  Labor: getComputedStyle(root).getPropertyValue('--secondary1').trim(),
  Material: getComputedStyle(root).getPropertyValue('--secondary2').trim(),
  Loss: getComputedStyle(root).getPropertyValue('--failure-color').trim()
};

const R = Math.min((breakdownWidth - 2 * pad) / 4, (height / 2 - 2 * pad) / 4) * 0.78;
const pie = d3.pie().value(d => d.value).sort(null);
const arc = d3.arc().innerRadius(0).outerRadius(R);
const ttPie = createTooltip('profit-pie-tooltip');

```

```

pies.forEach((pd, i) => {

  const g = topHalf.append("g")

  .attr("transform", ` translate(${pad + ((breakdownWidth - 2 * pad) * (i % 2 === 0 ?
0.25 : 0.75))}, ${pad + ((height / 2 - 2 * pad) * (i < 2 ? 0.3 : 0.78)) - 10 + titleOffset}) `);

  const isLoss = pd.data.Profit < 0;

  const chartData = Object.entries(

    isLoss

    ? { Loss: -pd.data.Profit, Labor: pd.data.Labor, Material: pd.data.Material }

    : { Profit: pd.data.Profit, Labor: pd.data.Labor, Material: pd.data.Material }

  )

  .map(([k, v]) => ({ label: k, value: v }))

  .filter(d => d.value > 1e-6);

  const total = d3.sum(chartData, r => r.value);

  g.selectAll("path")

    .data(pie(chartData))

    .join("path")

    .attr("d", arc)

    .attr("fill", d => pieColors[d.data.label])

    .attr("stroke", getComputedStyle(root).getPropertyValue('--accent'))

    .attr("stroke-width", 1.5)

    .on("mouseenter", () => ttPie.style("opacity", 1))

    .on("mouseleave", () => ttPie.style("opacity", 0))

```

```

.on("mousemove", (ev, d) => {
    const amountValue = d.data.label === 'Loss' ? `-${fmtMoney(d.data.value)}` :
    fmtMoney(d.data.value);

    ttPie.html(
        `
```

```

}).map(([label, color]) => ({ label, color }));

// width measurement for flow layout

const measurer = svg.append("text").style("font-size", "12px").style("font-weight",
700).style("opacity", 0);

const itemWidths = legData.map(d => {

  measurer.text(d.label);

  return 12 + 4 + measurer.node().getBBox().width + 8; // rect + gap + text + trailing pad
});

measurer.remove();

// --- CONSISTENT ROW HEIGHT & CUSTOM EXTRA GAPS ---

const legendRowHeight = 22;    // base row height

const extraGap12 = 8;          // extra space between rows 1 -> 2

const extraGap34 = 8;          // extra space between rows 3 -> 4

let xCursor = legendXStart;

let yCursor = legendYBase;

let currentRow = 0;             // 0-based row index (0=first row)

const rectY = (legendRowHeight - 12) / 2; // center 12px swatch in the row

const textY = rectY + 10.5;      // text baseline aligned to swatch

const li = legend.selectAll(".li")

  .data(legData)

  .join("g")

```



```

.attr("class", "li")

.attr("transform", (d, i) => {
  const w = itemWidths[i];

  if (xCursor + w > legendXStart + innerWidth) {
    xCursor = legendXStart;
    currentRow += 1; // wrapped to next row

    // add base row height + targeted extra gaps
    let extra = 0;
    if (currentRow === 1) extra = extraGap12; // after first row
    if (currentRow === 3) extra = extraGap34; // after third row
    yCursor += legendRowHeight + extra;
  }

  const tx = xCursor;
  xCursor += w;
  return `translate(${tx},${yCursor})`;
});

```

```

li.append("rect")
  .attr("y", rectY)
  .attr("width", 12)
  .attr("height", 12)
  .attr("fill", d => d.color)
  .attr("rx", 2);

```

```
li.append("text")  
  .attr("x", 16)  
  .attr("y", textY)  
  .style("font-size", "12px")  
  .style("font-weight", 700)  
  .text(d => d.label);
```

```
// --- BAR CHART (bottom half) ---
```

```
const bottomHalf = breakdownGroup.append("g").attr("transform",  
`translate(0,${height / 2})`);
```

```
bottomHalf.append("rect")  
  .attr("class", "breakdown-border")  
  .attr("x", pad / 2).attr("y", pad / 2)  
  .attr("width", breakdownWidth - pad)  
  .attr("height", height / 2 - pad);
```

```
// ADD TITLE: Bottom (bars)
```

```
bottomHalf.append("text")  
  .attr("x", pad / 2 + (breakdownWidth - pad) / 2)  
  .attr("y", pad / 2 + 20)  
  .attr("text-anchor", "middle")  
  .attr("class", "panel-title")  
  .style("font-weight", 800)  
  .text("Profit by Model");
```

```
const barM = { top: 30, right: 22, bottom: 40, left: 20 };

const barH = height / 2 - 2 * pad - barM.top - barM.bottom;

const yBar = d3.scaleLinear()

  .domain([Math.min(0, d3.min(perModel, d => d.profit)), d3.max(perModel, d =>
d.profit)])

  .nice()

  .range([barH, 0]);

let maxLabelWidth = 0;

const tempText = svg.append("text").attr("class", "axis").style("opacity", 0);

yBar.ticks(5).forEach(tick => {

  maxLabelWidth = Math.max(maxLabelWidth,
tempText.text(fmtMoney(tick)).node().getBBox().width);

});

tempText.remove();

const yAxisSpace = maxLabelWidth + 10;

const barW = breakdownWidth - 2 * pad - barM.right - yAxisSpace;

const gB = bottomHalf.append("g").attr("transform", `translate(${pad},${pad +
barM.top})`);

const xBand = d3.scaleBand()

  .domain(perModel.map(d => d.label))

  .range([yAxisSpace, yAxisSpace + barW])

  .padding(0.25);
```

```
gB.append("g").attr("class", "axis").attr("transform",  
`translate(${yAxisSpace},0)`).call(d3.axisLeft(yBar).ticks(5).tickFormat(fmtMoney));
```

```
gB.append("g").attr("class", "axis").attr("transform",  
`translate(0,${barH})`).call(d3.axisBottom(xBand));
```

```
if (yBar.domain()[0] < 0) {
```

```
gB.append("line")
```

```
.attr("x1", yAxisSpace).attr("x2", yAxisSpace + barW)
```

```
.attr("y1", yBar(0)).attr("y2", yBar(0))
```

```
.attr("stroke", getComputedStyle(root).getPropertyValue('--accent').trim())
```

```
.attr("stroke-width", 1.5)
```

```
.attr("stroke-dasharray", "3,3");
```

```
}
```

```
gB.append("text").attr("class", "axis-label").attr("transform", "rotate(-90)").attr("x", -barH  
/ 2).attr("y", 0).attr("text-anchor", "middle").text("Gross Profit");
```

```
gB.append("text").attr("class", "axis-label").attr("x", yAxisSpace + barW / 2).attr("y",  
barH + barM.bottom - 6).attr("text-anchor", "middle").text("Model");
```

```
const modelColor = {
```

```
Super: getComputedStyle(root).getPropertyValue('--super-color').trim(),
```

```
Ultra: getComputedStyle(root).getPropertyValue('--ultra-color').trim(),
```

```
Mega: getComputedStyle(root).getPropertyValue('--mega-color').trim()
```

```
};
```

```
const ttBar = createTooltip('profit-bar-tooltip');
```

```

gB.selectAll("rect")
  .data(perModel)
  .join("rect")
  .attr("x", d => xBand(d.label))
  .attr("width", xBand.bandwidth())
  .attr("y", d => d.profit < 0 ? yBar(0) : yBar(d.profit))
  .attr("height", d => Math.abs(yBar(d.profit) - yBar(0)))
  .attr("rx", 4)
  .attr("fill", d => modelColor[d.label] || getComputedStyle(root).getPropertyValue('--accent'))
  .attr("stroke", getComputedStyle(root).getPropertyValue('--accent'))
  .attr("stroke-width", 1.5)
  .on("mouseenter", () => ttBar.style("opacity", 1))
  .on("mouseleave", () => ttBar.style("opacity", 0))
  .on("mousemove", (ev, d) => {
    ttBar.html(
      `<div class="tooltip-header">${d.label}</div>
      <div class="tooltip-row"><span class="tooltip-key">Profit</span><span>${fmtMoney(d.profit)}</span></div>
      <div class="tooltip-row"><span class="tooltip-key">Margin</span><span>${fmtPct(d.margin)}</span></div>`
    ).style("left", (ev.clientX + 14) + "px").style("top", (ev.clientY - 24) + "px");
  });
}

return { draw };

```

```
})();
```

```
/**
```

```
* =====
```

```
* ScheduleTab IIFE Module
```

```
*
```

```
* Encapsulates all logic for rendering and animating the Schedule
```

```
* Gantt chart visualization.
```

```
* =====
```

```
*/
```

```
const ScheduleTab = (function () {
```

```
  /**
```

```
    * @tab Schedule
```

```
    * Draws the animated Gantt chart for the production schedule.
```

```
    * This is the main public method exposed by the IIFE.
```

```
  */
```

```
  function draw() {
```

```
    // --- INITIAL SETUP ---
```

```
    // Filter state for toggling product visibility on the chart.
```

```
    let activeProductFilters = {
```

```
      1: true, // Super (modelId 1)
```

```
      2: true, // Ultra (modelId 2)
```

```
      3: true // Mega (modelId 3)
```

```
    };
```

```
    // The default duration of the animated view window in simulation minutes.
```

```

const VIEW_WINDOW_MINS = 10;

// State for managing the view's zoom level and pause state.

let zoomLevel = 1.0; // 1.0 = normal, >1 = zoom in, <1 = zoom out.

let isPaused = false;

let currentViewWindow = VIEW_WINDOW_MINS; // The current view window, adjusted
by zoom.

// --- SVG & DATA PREPARATION ---

// Select the SVG container and clear any previous renderings.

const svg = d3.select("#schedule-panel");

svg.selectAll("*").remove();

svg.selectAll(".workstation-schedule-label").remove(); // Clear any lingering labels.

// Run the Gantt simulation to get task data.

const simulationResult = runGanttSimulation();

const { clientWidth: containerWidth, clientHeight: containerHeight } =
document.getElementById('svg-container');

// If the simulation returns no tasks, display a message and exit.

if (!simulationResult || simulationResult.tasks.length === 0) {

    svg.append("text").attr("x", containerWidth / 2).attr("y", containerHeight / 2).attr("text-
anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--accent').trim())

    .text("No data to display. Check configuration or inputs.");

    return;

}

const { tasks } = simulationResult;

const opHours = parseFloat(opHoursInput.value);

// Define chart margins and dimensions.

const margin = { top: 40, right: 20, bottom: 40, left: 100 };

const width = containerWidth - margin.left - margin.right;

```

```

const height = containerHeight - margin.top - margin.bottom;

// Create the main chart group, translated by the margin.

const chart = svg.append("g").attr("transform",
`translate(${margin.left},${margin.top})`);

const controlsY = height + margin.top - 35;

const controlsStartX = margin.left;

// --- UI CONTROLS & DISPLAYS ---

// Timer display for the simulation clock.

const clockGroup = svg.append("g").attr("transform", `translate(${controlsStartX},
${controlsY})`);

clockGroup.append("rect").attr("x", 10).attr("y", -10).attr("width", 70).attr("height",
18).attr("fill", getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3); //
Background rect

const clockDisplay = clockGroup.append("text").attr("id", "sim-clock-display").attr("x",
26).attr("y", 3).attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "14px").style("font-family", "monospace").text("00:00"); //
Time text

// Production counters for each model type.

const superCounter = svg.append("text").attr("id", "super-counter").attr("x",
controlsStartX + 14).attr("y", controlsY + -60).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).style("font-size",
"16px").text("Super: 0");

const ultraCounter = svg.append("text").attr("id", "ultra-counter").attr("x",
controlsStartX + 14).attr("y", controlsY + -40).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).style("font-size",
"16px").text("Ultra: 0");

const megaCounter = svg.append("text").attr("id", "mega-counter").attr("x",
controlsStartX + 14).attr("y", controlsY + -20).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).style("font-size",
"16px").text("Mega: 0");

```



```
svg.append("text").attr("class", "product-title").attr("x", controlsStartX + 14).attr("y",
controlsY + -80).attr("text-anchor", "start").attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).style("font-size",
"14px").style("font-weight", "bold").text("Product"); // Section title
```

```
// Product type filter controls (checkboxes).
```

```
const superFilterGroup = svg.append("g").attr("class", "super-filter").attr("transform",
`translate(${controlsStartX + 120}, ${controlsY - 71})`).style("cursor", "pointer"); // Super
filter
```

```
superFilterGroup.append("rect").attr("width", 12).attr("height", 12).attr("fill",
activeProductFilters[1] ? getComputedStyle(root).getPropertyValue('--super-color').trim() :
getComputedStyle(root).getPropertyValue('--white').trim()).attr("stroke",
getComputedStyle(root).getPropertyValue('--super-color').trim()).attr("stroke-width",
2).attr("rx", 2);
```

```
if (activeProductFilters[1]) { superFilterGroup.append("text").attr("x", 6).attr("y",
9).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "10px").style("font-weight", "bold").text("✓"); } // Checkmark
```

```
const ultraFilterGroup = svg.append("g").attr("class", "ultra-filter").attr("transform",
`translate(${controlsStartX + 120}, ${controlsY - 51})`).style("cursor", "pointer"); // Ultra
filter
```

```
ultraFilterGroup.append("rect").attr("width", 12).attr("height", 12).attr("fill",
activeProductFilters[2] ? getComputedStyle(root).getPropertyValue('--ultra-color').trim() :
getComputedStyle(root).getPropertyValue('--white').trim()).attr("stroke",
getComputedStyle(root).getPropertyValue('--ultra-color').trim()).attr("stroke-width",
2).attr("rx", 2);
```

```
if (activeProductFilters[2]) { ultraFilterGroup.append("text").attr("x", 6).attr("y",
9).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "10px").style("font-weight", "bold").text("✓"); } // Checkmark
```

```
const megaFilterGroup = svg.append("g").attr("class", "mega-filter").attr("transform",
`translate(${controlsStartX + 120}, ${controlsY - 31})`).style("cursor", "pointer"); // Mega
filter
```

```
megaFilterGroup.append("rect").attr("width", 12).attr("height", 12).attr("fill",
activeProductFilters[3] ? getComputedStyle(root).getPropertyValue('--mega-color').trim() :
getComputedStyle(root).getPropertyValue('--white').trim()).attr("stroke",
```

```

getComputedStyle(root).getPropertyValue('--mega-color').trim()).attr("stroke-width",
2).attr("rx", 2);

    if (activeProductFilters[3]) { megaFilterGroup.append("text").attr("x", 6).attr("y",
9).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "10px").style("font-weight", "bold").text("✓"); } // Checkmark

    svg.append("text").attr("class", "filter-title").attr("x", controlsStartX + 110).attr("y",
controlsY - 80).attr("text-anchor", "start").attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).style("font-size",
"14px").style("font-weight", "bold").text("Filter"); // Section title

    // Animation control buttons (Play/Pause, Reset).

    const controlsGroup = svg.append("g").attr("transform", `translate(${controlsStartX +
85}, ${controlsY - 10})`);

    const playPauseBtn = controlsGroup.append("g").attr("class", "play-pause-
btn").style("cursor", "pointer"); // Play/Pause button

    playPauseBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);

    const playPauseIcon = playPauseBtn.append("text").attr("x", 14).attr("y", 13).attr("text-
anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "14px").text(" || ");

    const resetBtn = controlsGroup.append("g").attr("class", "reset-btn").attr("transform",
"translate(32, 0)").style("cursor", "pointer"); // Reset button

    resetBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);

    resetBtn.append("text").attr("x", 14).attr("y", 13).attr("text-anchor", "middle").attr("fill",
getComputedStyle(root).getPropertyValue('--white').trim()).style("font-size",
"13px").text("⌂");

    // Filter reset button.

    const filterResetBtn = controlsGroup.append("g").attr("class", "filter-reset-
btn").attr("transform", "translate(64, 0)").style("cursor", "pointer");

    filterResetBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);

```

```
filterResetBtn.append("text").attr("x", 14).attr("y", 12).attr("text-anchor",
"middle").attr("fill", getComputedStyle(root).getPropertyValue('--white').trim()).style("font-
size", "10px").text("RST");
```

```
// Zoom controls.
```

```
const zoomGroup = svg.append("g").attr("transform", ` translate(${controlsStartX +
150}, ${controlsY - 10}) ` );
```

```
const zoomInBtn = zoomGroup.append("g").attr("class", "zoom-in-btn").style("cursor",
"pointer"); // Zoom In button
```

```
zoomInBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);
```

```
zoomInBtn.append("text").attr("x", 13.5).attr("y", 13).attr("text-anchor",
"middle").attr("fill", getComputedStyle(root).getPropertyValue('--white').trim()).style("font-
size", "14px").text("+");
```

```
const zoomOutBtn = zoomGroup.append("g").attr("class", "zoom-out-
btn").attr("transform", "translate(32, 0)").style("cursor", "pointer"); // Zoom Out button
```

```
zoomOutBtn.append("rect").attr("width", 28).attr("height", 18).attr("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("rx", 3);
```

```
zoomOutBtn.append("text").attr("x", 13.5).attr("y", 13).attr("text-anchor",
"middle").attr("fill", getComputedStyle(root).getPropertyValue('--white').trim()).style("font-
size", "14px").text("-");
```

```
// --- CONTROL EVENT LISTENERS ---
```

```
// Play/Pause button functionality.
```

```
playPauseBtn.on("click", function () {
```

```
    isPaused = !isPaused;
```

```
    animationState.schedule.isPaused = isPaused;
```

```
    playPauseIcon.text(isPaused ? "▶" : "||"); // Toggle icon.
```

```
    // If resuming, restart the animation loop if it's not already running.
```

```
    if (!isPaused && !animationState.schedule.isRunning) {
```

```
        animationState.schedule.isRunning = true;
```

```

        animationState.schedule.lastFrameTime = performance.now();

        animationState.schedule.frameId = requestAnimationFrame(animationLoop);
    }

});

// Reset button functionality.

resetBtn.on("click", function () {

    animationState.schedule.totalSimTimeMins = 0; // Reset time to zero.

    animationState.schedule.lastFrameTime = performance.now();

    isPaused = false;

    animationState.schedule.isPaused = false;

    playPauseIcon.text(" || ");

    clockDisplay.text("00:00");

    // Restart animation loop if it was stopped.

    if (!animationState.schedule.isRunning) {

        animationState.schedule.isRunning = true;

        animationState.schedule.frameId = requestAnimationFrame(animationLoop);

    }

});

// Zoom In/Out functionality.

zoomInBtn.on("click", function () {

    zoomLevel = Math.min(zoomLevel * 1.5, 4.0); // Increase zoom, max 4x.

    currentViewWindow = VIEW_WINDOW_MINS / zoomLevel; // Decrease view window
duration.

});

zoomOutBtn.on("click", function () {

    zoomLevel = Math.max(zoomLevel / 1.5, 0.25); // Decrease zoom, min 0.25x.

```

```
        currentViewWindow = VIEW_WINDOW_MINS / zoomLevel; // Increase view window
duration.
```

```
});
```

```
// Filter reset functionality.
```

```
filterResetBtn.on("click", function () {
```

```
    // Reset all product filters to active.
```

```
    activeProductFilters[1] = true;
```

```
    activeProductFilters[2] = true;
```

```
    activeProductFilters[3] = true;
```

```
    updateFilterUI(); // Update checkbox visuals.
```

```
    updateTaskVisibility(); // Update task bar visibility.
```

```
});
```

```
// Individual product filter functionality.
```

```
superFilterGroup.on("click", function () {
```

```
    activeProductFilters[1] = !activeProductFilters[1];
```

```
    updateFilterUI();
```

```
    updateTaskVisibility();
```

```
});
```

```
ultraFilterGroup.on("click", function () {
```

```
    activeProductFilters[2] = !activeProductFilters[2];
```

```
    updateFilterUI();
```

```
    updateTaskVisibility();
```

```
});
```

```
megaFilterGroup.on("click", function () {
```

```
    activeProductFilters[3] = !activeProductFilters[3];
```

```
    updateFilterUI();
```

```

        updateTaskVisibility();

    });

    // --- FILTER HELPER FUNCTIONS ---

    /**
     * Updates the visual state of the filter checkboxes.
     */

    function updateFilterUI() {

        // Update Super filter checkbox and checkmark.

        superFilterGroup.select("rect").attr("fill", activeProductFilters[1] ?
getComputedStyle(root).getPropertyValue('--super-color').trim() :
getComputedStyle(root).getPropertyValue('--white').trim());

        superFilterGroup.selectAll("text").remove();

        if (activeProductFilters[1]) { superFilterGroup.append("text").attr("x", 6).attr("y",
9).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "10px").style("font-weight", "bold").text("✓"); }

        // Update Ultra filter checkbox and checkmark.

        ultraFilterGroup.select("rect").attr("fill", activeProductFilters[2] ?
getComputedStyle(root).getPropertyValue('--ultra-color').trim() :
getComputedStyle(root).getPropertyValue('--white').trim());

        ultraFilterGroup.selectAll("text").remove();

        if (activeProductFilters[2]) { ultraFilterGroup.append("text").attr("x", 6).attr("y",
9).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "10px").style("font-weight", "bold").text("✓"); }

        // Update Mega filter checkbox and checkmark.

        megaFilterGroup.select("rect").attr("fill", activeProductFilters[3] ?
getComputedStyle(root).getPropertyValue('--mega-color').trim() :
getComputedStyle(root).getPropertyValue('--white').trim());

        megaFilterGroup.selectAll("text").remove();

```

```

        if (activeProductFilters[3]) { megaFilterGroup.append("text").attr("x", 6).attr("y",
9).attr("text-anchor", "middle").attr("fill", getComputedStyle(root).getPropertyValue('--
white').trim()).style("font-size", "10px").style("font-weight", "bold").text("✓"); }

    }

    /**
     * Animates the visibility of task bars based on the active filters.
     */

    function updateTaskVisibility() {

        // Select bars that should be visible.

        const visibleBars = contentGroup.selectAll(".bar").filter(d =>
activeProductFilters[d.modelId]);

        // Select bars that should be hidden.

        const hiddenBars = contentGroup.selectAll(".bar").filter(d
=> !activeProductFilters[d.modelId]);

        // Animate visible bars into view.

        visibleBars.style("display",
"block").transition().duration(300).ease(d3.easeQuadOut).style("opacity",
0.9).style("transform", "scale(1)");

        // Animate hidden bars out of view.

        hiddenBars.transition().duration(300).ease(d3.easeQuadIn).style("opacity",
0.0).style("transform", "scale(0.95)").on("end", function () { d3.select(this).style("display",
"none"); });

    }

    // --- CHART & ANIMATION SETUP ---

    // Timeline scrubbing: click on the chart to jump to a specific time.

    chart.append("rect")

        .attr("class", "timeline-scrubber").attr("width", width).attr("height", height)

        .attr("fill", "transparent").style("cursor", "crosshair")

```

```

.on("click", function (event) {

  const [mouseX] = d3.pointer(event);

  const clickedTime = xScale.invert(mouseX); // Convert pixel position to simulation
time.

  // Update simulation time if the click is within valid bounds.

  if (clickedTime >= 0 && clickedTime <= totalSimDurationMinutes) {

    animationState.schedule.totalSimTimeMins = clickedTime;

    const h = String(Math.floor(clickedTime / 60)).padStart(2, '0');

    const m = String(Math.floor(clickedTime % 60)).padStart(2, '0');

    clockDisplay.text(` ${h}:${m} `); // Update clock display immediately.

  }

});

// Speed slider.

const sliderWidth = Math.max(40, Math.min(100, containerWidth * 0.12));

const sliderGroup = svg.append("g").attr("transform", ` translate(${controlsStartX +
220}, ${controlsY}) `);

const speedScale = d3.scaleLinear().domain([0.1, 8.0]).range([0,
sliderWidth]).clamp(true); // Map speed value to pixel position.

sliderGroup.append("text").attr("x", sliderWidth / 2).attr("y", -8).attr("text-anchor",
"middle").style("font-size", "12px").style("font-weight", "bold").style("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).text("Speed"); // Label.

sliderGroup.append("line").attr("class", "track").attr("x1", 0).attr("x2",
sliderWidth).attr("stroke", getComputedStyle(root).getPropertyValue('--
accent').trim()).attr("stroke-width", 3).attr("stroke-linecap", "round"); // Track.

sliderGroup.append("circle").attr("id", "d3-schedule-slider-handle").attr("class",
"handle").attr("r", 6).attr("fill", getComputedStyle(root).getPropertyValue('--
secondary2').trim()).attr("stroke", getComputedStyle(root).getPropertyValue('--
white').trim()).attr("stroke-width", 2).attr("cx",
speedScale(animationState.speedMultiplier)); // Handle.

```



```

    const speedInteractionArea = sliderGroup.append("rect").attr("x", -10).attr("width",
sliderWidth + 20).attr("y", -10).attr("height", 20).style("fill", "transparent").style("cursor",
"grab").style("touch-action", "none"); // Interaction area.

    // Speed slider event listeners (drag, click, wheel).

    speedInteractionArea

        .on("mousedown", function () { d3.select(this).style("cursor", "grabbing"); })

        .on("mouseup", function () { d3.select(this).style("cursor", "grab"); })

        .on("click", (event) => {

            const localX = Math.max(0, Math.min(sliderWidth, d3.pointer(event,
sliderGroup.node())[0]));

            animationState.speedMultiplier = speedScale.invert(localX);

            sliderGroup.select(".handle").attr("cx",
speedScale(animationState.speedMultiplier));

        })

        .call(d3.drag().on("drag", (event) => {

            const localX = Math.max(0, Math.min(sliderWidth, event.x));

            animationState.speedMultiplier = speedScale.invert(localX);

            sliderGroup.select(".handle").attr("cx",
speedScale(animationState.speedMultiplier));

        }));

    // --- CHART DRAWING ---

    // Main content group for chart elements (bars, lines). This group is translated by the
    sidebar scroll.

    const contentGroup = chart.append("g").attr("class", "schedule-content-group");

    const yOffset = document.getElementById('svg-
container').getBoundingClientRect().top + margin.top;

    // Map task IDs to their vertical position based on the sidebar layout.

    const elementGeometry = new Map();

```

```

document.querySelectorAll('.element-row').forEach(elRow => {

    const taskId = parseInt(elRow.dataset.taskId);

    const rect = elRow.getBoundingClientRect();

    const centerY = rect.top + rect.height / 2;

    const barHeight = rect.height * 0.8;

    const barY = (centerY - barHeight / 2) - yOffset;

    elementGeometry.set(taskId, { y: barY, height: barHeight });

});

// Add workstation labels and separator lines.

document.querySelectorAll('.workstation-title').forEach(title => {

    const rect = title.getBoundingClientRect();

    const centerY = rect.top + rect.height / 2;

    const lineY = centerY - yOffset;

    contentGroup.append("line").attr("x1", -margin.left).attr("x2", width +
margin.right).attr("y1", lineY).attr("y2", lineY).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("stroke-width",
2).attr("stroke-opacity", 0.3); // Separator line.

    const workstationMatch = title.textContent.match(/\d+/);

    if (workstationMatch) { // Label.

        contentGroup.append("text").attr("class", "workstation-schedule-label").attr("x", -
10).attr("y", lineY + 2).attr("text-anchor", "end").attr("dominant-baseline",
"hanging").style("font-size", "14px").style("font-weight", "bold").style("fill",
getComputedStyle(root).getPropertyValue('--accent').trim()).text(` WS
${workstationMatch[0]}` );

    }

});

// D3 scales for mapping data to visual properties.

const xScale = d3.scaleLinear().range([0, width]); // Time -> X position.

```

```

    const modelColors = d3.scaleOrdinal().domain([1, 2,
3]).range([getComputedStyle(root).getPropertyValue('--super-color').trim(),
getComputedStyle(root).getPropertyValue('--ultra-color').trim(),
getComputedStyle(root).getPropertyValue('--mega-color').trim()]); // Model ID -> color.

    // --- PERFORMANCE OVERLAYS ---

    // Add utilization bars and bottleneck highlighting.

    try{

        const metrics = calculateMetrics({ dailyDemand: +dailyDemandInput.value,
opHours: +opHoursInput.value, numEmployees: +numEmployeesInput.value },
{ laborCost: +laborCostInput.value });

        if (metrics && Array.isArray(metrics.workstations) && metrics.workstations.length > 0)
        {

            // Find the bottleneck workstation (longest cycle time).

            const bottleneckWS = metrics.workstations.reduce((max, ws) => (ws.cycleTime >
(max.cycleTime || 0) ? ws : max), metrics.workstations[0]);

            document.querySelectorAll('.workstation-title').forEach(title => {

                const wsMatch = title.textContent && title.textContent.match(/\d+/);

                if (!wsMatch) return;

                const wsId = wsMatch[0];

                const wsInfo = metrics.workstations.find(w => String(w.id) === String(wsId));

                if (!wsInfo) return;

                const rect = title.getBoundingClientRect();

                const lineY = (rect.top + rect.height / 2) - yOffset;

                // Highlight the bottleneck row.

                if (wsInfo === bottleneckWS) {

                    contentGroup.append('rect').attr('x', -margin.left).attr('y', lineY).attr('width',
width + margin.right + margin.left).attr('height', rect.height + 8).attr('fill',
getComputedStyle(root).getPropertyValue('--failure-color').trim()).attr('opacity',
0.12).lower();

```

```

    }

    // Calculate and display utilization percentage.

    const totalOpMinutes = opHours * 60;

    const productiveMinutes = (wsInfo.cycleTime || 0) *
(metrics.throughputUnitsPerDay || 0);

    const actualUtilization = totalOpMinutes > 0 ? (productiveMinutes /
totalOpMinutes) : 0;

    contentGroup.append('text').attr('class', 'ws-efficiency-label').attr('x', 65).attr("y",
lineY + 13).attr('text-anchor', 'start').style('font-size', '11px').style('font-weight',
'600').style('fill', getComputedStyle(root).getPropertyValue('--accent').trim()).text(` Util:
${(actualUtilization * 100).toFixed(1)}%` );

    // Draw the utilization bar.

    const barWidth = 50;

    const barHeight = 4;

    contentGroup.append('rect').attr('class', 'ws-utilization-bar-bg').attr('x', 8).attr("y",
lineY + 7).attr('width', barWidth).attr('height', barHeight).attr('fill',
getComputedStyle(root).getPropertyValue('--idle-color').trim()).attr('stroke',
getComputedStyle(root).getPropertyValue('--white').trim()).attr('stroke-width', 0.5).attr('rx',
1); // Background.

    contentGroup.append('rect').attr('class', 'ws-utilization-bar').attr('x', 8).attr("y",
lineY + 7).attr('width', barWidth * actualUtilization).attr('height', barHeight).attr('fill',
getComputedStyle(root).getPropertyValue('--primary').trim()).attr('rx', 1); // Foreground.

    });

    }

    } catch (e) { console.warn('Could not render performance overlays:', e); }

    // --- TIME AXIS & MARKERS ---

    const timeGridGroup = chart.append("g").attr("class", "time-grid"); // Group for grid
lines.

    const timeAxis = chart.append("g").attr("class", "time-axis").attr("transform",
`translate(0, ${height - 10})` ); // Group for the bottom time axis.

```

```
svg.append("text").attr("class", "time-axis-label").attr("x", margin.left + width /
2).attr("y", containerHeight - 15).attr("text-anchor", "middle").style("font-size",
"12px").style("font-weight", "bold").style("fill", getComputedStyle(root).getPropertyValue('--
accent').trim()).text("Time (Hours:Minutes)"); // Axis label.
```

```
const timeMarker = chart.append("line").attr("x1", 0).attr("x2", 0).attr("y1", -
margin.top).attr("y2", height + margin.bottom).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent').trim()).attr("stroke-width", 2); //
Vertical line for current time.
```

```
timeMarker.append("title").text("Current Simulation Time"); // Tooltip for the time
marker.
```

```
// --- TOOLTIP & TASK BARS ---
```

```
const scheduleTooltip = createTooltip('schedule-tooltip'); // Create a reusable tooltip.
```

```
// Helper function to get product type name from model ID.
```

```
const getProductTypeName = (modelId) => ({ 1: 'Super', 2: 'Ultra', 3: 'Mega' })[modelId] ||
'Unknown';
```

```
// Helper function to format time duration nicely.
```

```
const formatDuration = (minutes) => (minutes < 1) ? `${(minutes * 60).toFixed(0)}s` :
`${minutes.toFixed(2)}m`;
```

```
// Bind task data and create the Gantt bars.
```

```
contentGroup.append("g").attr("class", "task-bars")

.selectAll(".bar").data(tasks).enter().append("rect")

.attr("class", "bar")

.attr("y", d => elementGeometry.get(d.taskId)?.y || -100) // Set Y position based on
element geometry map.

.attr("height", d => elementGeometry.get(d.taskId)?.height || 0) // Set height similarly.

.attr("fill", d => modelColors(d.modelId)) // Set fill color based on product model.

.attr("stroke", getComputedStyle(root).getPropertyValue('--
accent').trim()).attr("stroke-width", 1).attr("rx", 2).attr("ry", 2) // Styling.

.style("opacity", d => (activeProductFilters[d.modelId] ? 0.9 : 0.0)) // Set initial opacity
based on filters.
```

```

        .style("display", d => (activeProductFilters[d.modelId] ? "block" : "none")) // Set initial
display based on filters.

        .style("transform", d => (activeProductFilters[d.modelId] ? "scale(1)" : "scale(0.95)"))

        .style("cursor", "pointer")

        .on("mouseenter", function (event, d) { // Tooltip mouseover behavior.

            d3.select(this).style("opacity", 1).style("stroke-width", 2); // Highlight bar.

            const productType = getProductTypeName(d.modelId);

            const duration = formatDuration(d.endTime - d.startTime);

            const startTime = `${Math.floor(d.startTime / 60).toString().padStart(2,
'0')}:${Math.floor(d.startTime % 60).toString().padStart(2, '0')}`;

            const endTime = `${Math.floor(d.endTime / 60).toString().padStart(2,
'0')}:${Math.floor(d.endTime % 60).toString().padStart(2, '0')}`;

            // Populate and show tooltip.

            scheduleTooltip.html(`

                <div class="tooltip-header" style="color:
${modelColors(d.modelId)};">${productType} Refrigerator</div>

                <div class="tooltip-row"><span>Element:</span>
<strong>${d.taskId}</strong></div>

                <div class="tooltip-row"><span>Workstation:</span>
<strong>${d.workstationId}</strong></div>

                <div class="tooltip-row"><span>Duration:</span>
<strong>${duration}</strong></div>

                <div class="tooltip-row"><span>Start:</span> <strong>${startTime}</strong></div>

                <div class="tooltip-row"><span>End:</span> <strong>${endTime}</strong></div>

            `).style("opacity", 1);

        })

        .on("mousemove", function (event) { // Update tooltip position.

```

```

        scheduleTooltip.style("left", (event.pageX + 15) + "px").style("top", (event.pageY - 10)
+ "px");
    })

    .on("mouseleave", function () { // Hide tooltip and de-highlight bar.

        d3.select(this).style("opacity", 0.9).style("stroke-width", 1);

        scheduleTooltip.style("opacity", 0);

    });

// --- ANIMATION LOOP ---

const maxTime = tasks.length > 0 ? d3.max(tasks, d => d.endTime) : (opHours * 60);
const totalSimDurationMinutes = maxTime;

// Initialize the global animation state for this tab.
animationState.schedule = {

    isRunning: true,

    lastFrameTime: performance.now(),

    totalSimTimeMins: 0,

    frameId: null,

    isPaused: false

};

/**

* The main animation loop, called via requestAnimationFrame.

* @param {number} currentTime - The current timestamp provided by the browser.

*/

function animationLoop(currentTime) {

    if (!animationState.schedule.isRunning) return; // Exit if stopped.

    // Calculate time passed since last frame.

    const speedMultiplier = animationState.speedMultiplier;

```

```

const realDeltaMs = currentTime - animationState.schedule.lastFrameTime;
animationState.schedule.lastFrameTime = currentTime;
// Advance simulation time if not paused.
if (!isPaused && !animationState.schedule.isPaused) {
  const simDeltaMs = realDeltaMs * 60 * speedMultiplier;
  animationState.schedule.totalSimTimeMins += simDeltaMs / 60000;
}
// Stop the animation if the simulation time exceeds the total duration.
const elapsedSimTimeMinutes = animationState.schedule.totalSimTimeMins;
if (elapsedSimTimeMinutes > totalSimDurationMinutes) {
  animationState.schedule.isRunning = false;
  const finalHours = String(Math.floor(totalSimDurationMinutes / 60)).padStart(2, '0');
  const finalMinutes = String(Math.floor(totalSimDurationMinutes % 60)).padStart(2,
'0');
  clockDisplay.text(` ${finalHours}:${finalMinutes} `); // Display final time.
  return;
}
// Update clock and counters.
const h = String(Math.floor(elapsedSimTimeMinutes / 60)).padStart(2, '0');
const m = String(Math.floor(elapsedSimTimeMinutes % 60)).padStart(2, '0');
clockDisplay.text(` ${h}:${m} `);
const completedSuper = tasks.filter(t => t.endTime <= elapsedSimTimeMinutes &&
t.modelId === 1 && t.taskId === 31).length;
const completedUltra = tasks.filter(t => t.endTime <= elapsedSimTimeMinutes &&
t.modelId === 2 && t.taskId === 31).length;
const completedMega = tasks.filter(t => t.endTime <= elapsedSimTimeMinutes &&
t.modelId === 3 && t.taskId === 31).length;

```



```

superCounter.text(` Super: ${completedSuper}`);
ultraCounter.text(` Ultra: ${completedUltra}`);
megaCounter.text(` Mega: ${completedMega}`);

// Update the time domain of the x-axis to create the scrolling effect.
const viewStartTime = elapsedSimTimeMinutes;
xScale.domain([viewStartTime, viewStartTime + currentViewWindow]);

// Update the position and width of all task bars based on the new xScale.
contentGroup.selectAll(".bar")
  .attr("x", d => xScale(d.startTime))
  .attr("width", d => Math.max(0, xScale(d.endTime) - xScale(d.startTime)));

// Update the time grid lines.
const gridTicks = xScale.ticks(20);
const gridLines = timeGridGroup.selectAll(".grid-line").data(gridTicks);
gridLines.enter().append("line").attr("class", "grid-line")
  .merge(gridLines)
  .attr("x1", d => xScale(d)).attr("x2", d => xScale(d)).attr("y1", 0).attr("y2", height)
  .attr("stroke", getComputedStyle(root).getPropertyValue('--
accent').trim()).attr("stroke-width", 0.5).attr("stroke-dasharray", "2,2").style("opacity", 0.6);
gridLines.exit().remove();

// Redraw the bottom time axis.
const timeTickFormat = (d) => `${Math.floor(d / 60).toString().padStart(2,
'0')}:${Math.floor(d % 60).toString().padStart(2, '0')}`;

timeAxis.call(d3.axisBottom(xScale).ticks(10).tickFormat(timeTickFormat).tickSizeOuter(0))
  .selectAll("text").style("font-size", "11px").style("font-weight", "500");

// Request the next frame.
animationState.schedule.frameId = requestAnimationFrame(animationLoop);

```

```

}

// --- FINALIZATION ---

// Trigger a scroll event to correctly position the content group initially.
workstationList.dispatchEvent(new Event('scroll'));

// Start the animation loop.

animationState.schedule.frameId = requestAnimationFrame(animationLoop);

// --- LEGEND ---

const legendX = containerWidth - 200;

const legendY = containerHeight - 180;

const legend = svg.append("g").attr("transform", `translate(${legendX + 30}, ${legendY - 20})`);

legend.append("rect").attr("width", 150).attr("height", 140).attr("fill",
getComputedStyle(root).getPropertyValue('--white')).attr("stroke",
getComputedStyle(root).getPropertyValue('--accent')).attr("rx", 5); // Legend box

legend.append("text").text("Schedule Legend").attr("x", 10).attr("y", 20).style("font-
weight", "bold").attr("fill", getComputedStyle(root).getPropertyValue('--accent')); // Title

const legendItems = [

  { label: "Super Product", color: getComputedStyle(root).getPropertyValue('--super-
color') },

  { label: "Ultra Product", color: getComputedStyle(root).getPropertyValue('--ultra-
color') },

  { label: "Mega Product", color: getComputedStyle(root).getPropertyValue('--mega-
color') },

  { label: "Bottleneck WS", color: getComputedStyle(root).getPropertyValue('--failure-
color'), type: "bg" },

  { label: "Utilization Bar", color: getComputedStyle(root).getPropertyValue('--primary'),
type: "bar" }

];

// Create an entry for each item in the legend.

```

```

legendItems.forEach((item, i) => {

    const yPos = 45 + i * 20;

    if (item.type === "bg") {

        legend.append("rect").attr("x", 10).attr("y", yPos - 8).attr("width", 10).attr("height",
10).attr("fill", item.color).attr("opacity", 0.3);

    } else if (item.type === "bar") {

        legend.append("rect").attr("x", 10).attr("y", yPos - 2).attr("width", 10).attr("height",
4).attr("fill", item.color).attr("rx", 1);

    } else {

        legend.append("rect").attr("x", 10).attr("y", yPos - 8).attr("width", 10).attr("height",
10).attr("fill", item.color);

    }

    legend.append("text").text(item.label).attr("x", 25).attr("y", yPos + 2).style("font-size",
"11px").attr("fill", getComputedStyle(root).getPropertyValue('--accent'));

});

}

// Expose the public draw method to be called from the main script.

return {

    draw: draw

};

})();

```